



VNiVERSiDAD
D SALAMANCA

Creating Electronic Books with Code and Artificial Intelligence Assistants

Course for the Faculties of Sciences and Chemical Sciences

Álvaro Lozano Murciego and André Sales Mendes

Universidad de Salamanca

1st Edition • 2026

CONTENTS

I	Tutorial	3
1	1. What is a TeachBook?	5
1.1	Attribution and origin	6
1.2	What goes in and what comes out	6
1.3	Why this is useful for teaching	6
1.4	What this template adds on top of the original base	7
1.5	What a TeachBook is NOT	8
1.6	Practical advice to start	8
1.7	Bibliography for this page	8
2	2. Workflow	9
2.1	Book Development Cycle	9
2.2	File Structure	9
2.3	Directory Tree	11
3	3. Editing with AI	13
3.1	How to Ask the AI for Help	13
4	4. My first book from scratch	17
4.1	Step 0. Prepare the basic requirements	17
4.2	Step 1. Create a repository from the template	18
4.3	Step 2. Enable GitHub Actions and GitHub Pages	19
4.4	Step 3. Clone the repository in Antigravity	20
4.5	Step 4. Ask the agent to configure the environment	21
4.6	Step 5. Make a first edit with the agent	22
4.7	Step 6. View the local preview	23
4.8	Step 7. Generate the PDFs	24
4.9	Step 8. Publish the changes	25
5	5. The “X as code” Philosophy	27
5.1	What is “X as code”?	27
5.2	Why is this philosophy interesting?	28
5.3	Code as a bridge to the final result	28
5.4	Why this fits so well with AI	29
5.5	Reproducibility, not just automation	29
5.6	The mindset shift	30
II	Examples by Degree	31
6	Degree in Physics	33

6.1	Physics Example: Harmonic Oscillator	33
6.2	Drawing Circuits with SchemDraw	34
6.3	Advanced CircuitikZ for Precise Circuits	35
6.4	Technical Diagrams with Kroki for Physics	37
6.5	Ohm's Law: Linear Fit of V-I Data	39
6.6	Harmonic Oscillator: Simple Harmonic Motion	41
7	Degree in Mathematics	43
7.1	Mathematics Example: Symbolic Calculus	43
7.2	Quadratic Equation	43
7.3	Newton's Method for Finding Roots	44
7.4	2D Linear Transformations	46
7.5	Planar Point Sets with Many Unit Distances	48
8	Degree in Statistics	53
8.1	Statistics Example: Data Generation	53
8.2	Linear Regression with Synthetic Data	53
8.3	Estimating π with Monte Carlo	55
8.4	Practice Questions: Statistics	56
9	Degree in Computer Science	59
9.1	Entity-Relationship Diagram with Kroki (Mermaid)	59
9.2	UML Class Diagram with Kroki (Mermaid)	60
9.3	Diagrams with Kroki for Computer Science	63
9.4	Algorithmic Complexity: Visual Comparison	66
10	Degree in Geology	69
10.1	Topographic Profile	69
10.2	Stratigraphic Column	70
10.3	The Rock Cycle with Kroki (Mermaid)	71
11	Chemical Engineering Degree	73
11.1	Process Flow Diagram	73
11.2	McCabe-Thiele Diagram	74
11.3	Recycle Solved with <code>fsolve</code>	77
11.4	Concentration Gradient in a Catalyst Particle	78
11.5	Fast Reading of the Moody Chart	80
12	Chemistry Degree	83
12.1	pH Curve in a Buffer Solution	83
12.2	Michaelis-Menten Kinetics	85
12.3	Interactive Periodic Table	86
12.4	Interactive IR Spectrum	87
12.5	Nernst Equation	87
III	Cross-disciplinary Examples	89
13	Cross-disciplinary Examples	91
13.1	Example families	91
13.2	The common idea	92
13.3	Growth and Saturation Models	92
13.4	Questionnaire Analysis	93
13.5	Text Corpus Frequencies	94
13.6	Rubrics and Learning Sequences	95

13.7	Simulated Data and Variable Relationships	97
13.8	Balances and Conservation	98
IV	Examples from Other Books	101
14	Examples from Other Books	103
V	Basic Content	105
15	Basic Content	107
15.1	What you will find here	107
15.2	How to use this section	108
16	MyST Markdown: The Basics	109
16.1	Essential Syntax	109
16.2	Directives	110
16.3	Math	110
16.4	Tables	111
16.5	Code Blocks	111
17	Figures and Images	113
17.1	Centered image with alt text	113
17.2	Numbered figure with caption	114
17.3	Cross-reference to a figure	114
17.4	Margin figure	114
17.5	Useful parameters	115
18	Equations and Mathematics	117
18.1	Inline math	117
18.2	Display equation (unnumbered)	117
18.3	Numbered equation with label	117
18.4	Cross-reference to an equation	118
18.5	Common math symbols	118
19	Citations and Bibliography	119
19.1	Bibliography file	119
19.2	Textual citation: {cite:t}	119
19.3	Parenthetical citation: {cite:p}	120
19.4	Multiple citations	120
19.5	Printing the bibliography	120
19.6	Workflow	121
20	Cross-References	123
20.1	Label a section	123
20.2	Reference a section	123
20.3	Label and reference figures	123
20.4	Label and reference equations	124
20.5	Label and reference tables	124
20.6	Roles summary	125
21	Tables	127
21.1	Simple Markdown table	127
21.2	Numbered table with caption	127

21.3	List-table directive	128
21.4	Tips for readable tables	129
22	Admonitions and Dropdowns	131
22.1	Note (<code>{note}</code>)	131
22.2	Warning (<code>{warning}</code>)	131
22.3	Tip (<code>{tip}</code>)	132
22.4	Important (<code>{important}</code>)	132
22.5	Caution (<code>{caution}</code>)	132
22.6	Custom admonition	132
22.7	Dropdown (collapsible content)	133
22.8	Teaching use cases	133
23	Tabs and Cards	135
23.1	Simple tabs (<code>{tab-set}</code> / <code>{tab-item}</code>)	135
23.2	Grouped tabs (<code>{tab-set}</code> / <code>{tab-item}</code>)	136
23.3	Tabs for procedure steps	136
23.4	Cards (<code>{card}</code>)	137
24	Code Blocks	139
24.1	Syntax highlighting	139
24.2	Line numbers	139
24.3	Highlight specific lines	140
24.4	Other languages	140
24.5	Common languages in science	140
25	Videos	143
25.1	YouTube video (HTML + PDF)	143
25.2	How to get a YouTube video ID	143
25.3	Local video (HTML5)	144
25.4	Externally generated videos	144
25.5	Video with description	144
25.6	Recommended complete pattern	144
26	Diagrams with Kroki (Mermaid)	147
26.1	Flowchart (<code>flowchart</code>)	147
27	HTML and PDF Compatibility	151
27.1	Compatibility table	151
27.2	Universal pattern: HTML + LaTeX fallback	151
27.3	Example: video with fallback	152
27.4	Example: diagram with Kroki	152
27.5	Practical rules	152
VI	Interactivity and Assessment	153
28	Interactivity and Assessment	155
28.1	What will you find here?	155
28.2	Thebe + Binder: Live Code	156
28.3	Interactive Notebooks	158
28.4	Interactive Widgets (ipywidgets)	162
28.5	Quizzes with jupyterquiz	163
28.6	Parametric Questions	164
28.7	Exercises with Solutions	165

28.8	Interactive HTML (no frameworks)	167
28.9	H5P (Optional)	168
VII	Web Publication	171
29	Web Publication	173
29.1	GitHub Pages	173
29.2	Own Server	174
VIII	Information	177
30	About	179
30.1	How this book is made	179
30.2	How to reproduce the website	179
30.3	About the authors	180
31	Credits and licenses	181
31.1	Project credits	181
31.2	Acknowledgements	181
31.3	Book license	181
31.4	External resources	181
31.5	Code and tools	182
32	How to Cite	183
32.1	Text Citation	183
32.2	BibTeX	183
32.3	DOI	183
33	Bibliography	185

Welcome to **Creating Electronic Books with Code and Artificial Intelligence Assistants**.

What is this?

This is the course material and a template designed for the teaching staff of the **Faculties of Sciences and Chemical Sciences at USAL** to create interactive teaching books easily. The main idea of this template is for it to be broad enough to cover many use cases and for each student to adapt it to their own course, while also being sufficiently equipped to be used easily with the help of AI assistants (such as GitHub Copilot, Gemini, Claude, Codex, etc.) and a code editor such as VS Code.

Content

In this book you will find:

- *Tutorials* to learn how to use the template
- *Examples by Degree* to see real cases
- Information on *how to cite* and *licenses*

PDF version

You can also download the printable version of the book:

- Download PDF in English
- Descargar PDF en español

Note

This project is designed to be used with **VS Code** and **AI** assistants.

Part I

Tutorial

1. WHAT IS A TEACHBOOK?

A **TeachBook** is a digital teaching book, built from text files, code, and multimedia resources, which are then compiled into a **navigable website** and, in the case of this course, also a **PDF version**.

In simple terms:

- you edit the **source content**
- the system **builds** it
- you get an **attractive, functional, publishable** book

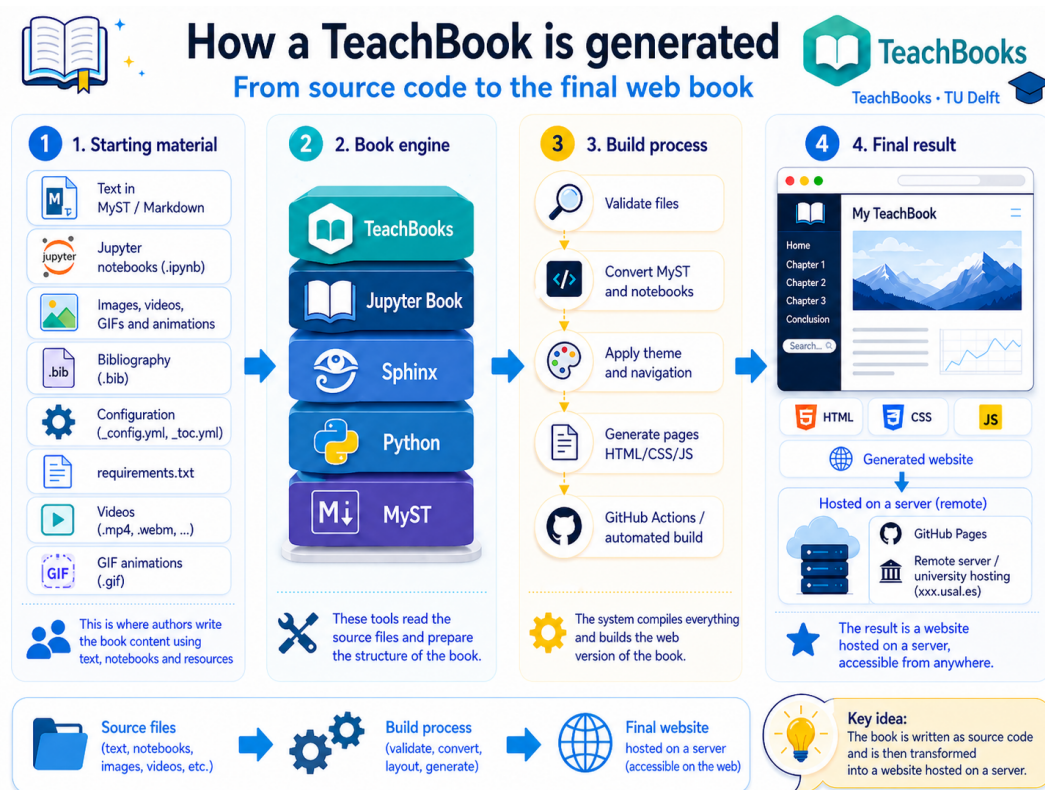


Figure1.1: A TeachBook starts from editable content and produces a publishable teaching website.

1.1 Attribution and origin

This template is built on the original ecosystem of:

- [TeachBooks](#)
- [Jupyter Book](#)

To situate the template, it is useful to cite both the TeachBooks project documentation [[TeachBooksDTeam25](#)] and the reference work for Jupyter Book [[ProjectJupyterBC+25](#)]: the former provides the teaching-oriented approach, while the latter provides the underlying system for building publishable computational books.

This specific adaptation has been prepared for the course entitled “Elaboración de libros electrónicos mediante código y asistentes de Inteligencia Artificial”, taught at the **Faculties of Sciences and Chemical Sciences at the University of Salamanca (USAL)**.

This matters: **we are not starting from scratch**. We are starting from existing tools and previous projects, and building on top of them a template adapted to a specific real teaching use (adapted to the needs of each course, but with reuse potential).

1.2 What goes in and what comes out

The core idea is this:

The following table summarizes the main elements of this section.

Table. What goes in and what comes out.

You start with...	And you get...
.md and .ipynb files	book web pages
images, bibliography, and static assets	enriched content
book configuration (<code>_config_*.yaml</code>)	navigation, theme, language, extensions
table of contents (<code>_toc_*.yaml</code>)	visible book structure

1.3 Why this is useful for teaching

1. **Accessibility**: the book can be read on the web from any device.
2. **Reproducibility**: the content comes from source files that can be versioned.
3. **Maintainability**: updating a page is much easier (thanks to AI agents) than rebuilding a whole set of notes.
4. **Scalability**: you can start with one page or chapter and grow gradually.
5. **Compatibility**: the same project can serve as both a website and a classic PDF book.

1.4 What this template adds on top of the original base

On top of TeachBooks and Jupyter Book, this project adds several practical improvements for teachers and students:

1.4.1 1. Multilingual structure

The book is prepared to work in:

- Spanish
- English

with synchronized indexes and content. More languages could be added easily.

1.4.2 2. PDF export

Besides the website, this template prepares a PDF output designed for printing or offline distribution. Not all multimedia and interactive content can be exported, but the rest of the valuable content can, making it possible to have a traditional version of the book.

1.4.3 3. Simple automation scripts

You do not need to learn a complicated workflow on day one. The project includes SKILLS to:

- prepare the environment (install dependencies, configure variables, etc.)
- build the website (generate the navigable book locally so you can review it)
- open live preview
- export PDF
- convert PDF to Markdown (to reuse content you already have in PDF from other subjects or courses)
- etc.

1.4.4 4. AI agent integration

The template is prepared so an agent can help you:

- create content
- reorganize chapters
- add multimedia
- maintain the structure of the book
- etc.

1.5 What a TeachBook is NOT

It is not:

- a PowerPoint presentation
- a React-style web app with heavy JS frameworks
- a custom-designed website (although the theme can be customized, that is not the main goal)

The main idea is that the content remains **editable, structured, and reusable**.

1.6 Practical advice to start

Do not try to build a huge book on the first day.

Start small:

- a cover page
- one short chapter
- one image
- one equation
- one section with two or three pages

Once that works well and you have been able to publish it, expand.

1.7 Bibliography for this page

2. WORKFLOW

This is the recommended workflow for your TeachBook:

1. **Write content** in Markdown files (`.md`) or Notebooks (`.ipynb`). In the case of Notebooks, it is important to execute them beforehand so that the book keeps the cell outputs. The AI agent can help you write the content and execute the Notebooks.
2. **Preview** with `python scripts/launch_preview.py` to see the same build that will be published. In this course, you can ask the AI agent to do it for you, because it has the information it needs.
3. **Save versions** with Git (`commit`). The AI agent will help you save changes in an orderly way.
4. **Publish** to GitHub (`push`) so the website updates automatically through GitHub Pages.

2.1 Book Development Cycle

1. Modify or add content in Markdown files or Notebooks.
2. Build the book and start a local server to preview the changes.
3. Generate the PDF versions of the book when needed. That process first creates LaTeX and then compiles the PDFs linked from the website.
4. If everything is correct, commit the changes with Git.
5. Publish the changes to GitHub to deploy the website.
6. If you find errors or want to improve something, go back to step 1 and repeat the cycle.

Fig. 2.1 summarizes the full process: write content, preview it, review the result, version the changes, and publish the website.

2.2 File Structure

To work with the book, we need to understand the file and folder structure. The content is organized by language, and each language has its own table of contents and configuration. This makes it possible to maintain versions in several languages with the same content or with content adapted to each language. We follow the same philosophy as the original TeachBooks project, with a few adaptations to make it easier to use with AI assistants.

The Spanish content is organized in `book/es/`. The English content is organized in `book/en/`. Each language has its own table of contents (`_toc_*.yaml`) and configuration (`_config_*.yaml`), but the structure must remain coherent across languages.

Main files and folders:

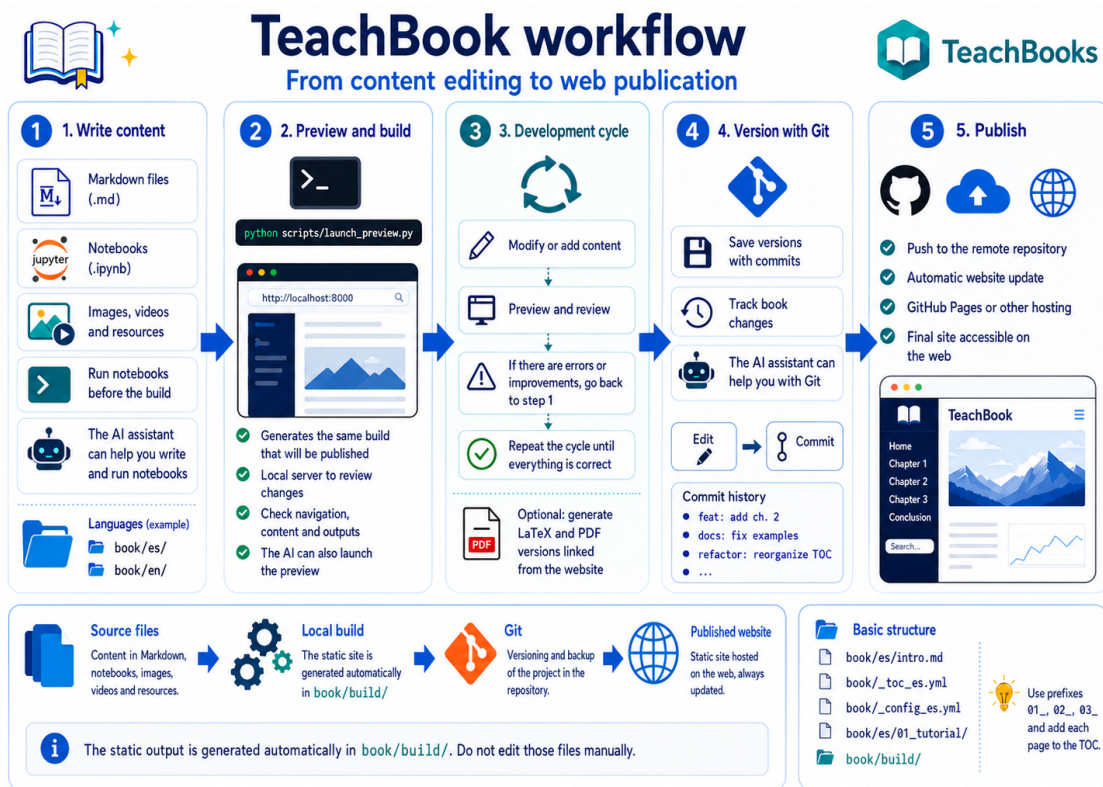


Figure2.1: TeachBook workflow from content editing to web publication.

- `book/es/intro.md`: main page of the Spanish version.
- `book/en/intro.md`: main page of the English version.
- `book/_toc_es.yml` and `book/_toc_en.yml`: book tables of contents; they define the order of chapters and sections.
- `book/_config_es.yml` and `book/_config_en.yml`: language-specific configuration files.
- `book/_static/`: shared images, videos, styles, scripts, PDFs, and references.
- `book/es/01_tutorial/` and `book/en/01_tutorial/`: folders for the initial tutorial.

Good practices for keeping things organized:

- Use numerical prefixes in folders and files (`01_`, `02_`, `03_`) to reflect the sequence.
- Add each new file to the TOC of the corresponding language; if it is not in the TOC, it will not appear in the navigation.
- Keep names clear and consistent, for example `03_experimental_activity.md`.
- When you add or change content in Spanish, also review the equivalent English page.

The build results are generated in `book/_build/html/`, which is the static version of the book published on GitHub Pages. You do not need to edit anything in that folder, because it is generated automatically from the source files.

The `book/_static/` folder is intended for static files such as images, videos, CSS styles, JavaScript scripts, PDFs, and references. You can place your images there and reference them from Markdown files using relative paths. If you do not know how to do that, the AI agent can help you place the images and reference them correctly.

2.3 Directory Tree

```
teachbook_sciences_template/
├── book/
│   ├── _config_es.yml
│   ├── _config_en.yml
│   ├── _toc_es.yml
│   ├── _toc_en.yml
│   ├── _static/
│   ├── _build/
│   ├── es/
│   │   ├── intro.md
│   │   └── 01_tutorial/
│   │       ├── 01_que_es_un_teachbook.md
│   │       └── 02_flujo_trabajo.md
│   └── en/
│       ├── intro.md
│       └── 01_tutorial/
│           ├── 01_what_is_a_teachbook.md
│           └── 02_workflow.md
└── scripts/
    └── launch_preview.py
```


3. EDITING WITH AI

This course is aimed at people with any level of programming experience. The possibilities opened up by AI assistants are enormous, because they allow people with no programming experience to create content more easily and people with programming experience to work faster and more efficiently.

You can edit this book in two ways:

- **Manually:** by editing Markdown files (`.md`) or Notebooks (`.ipynb`) directly with a text editor or an IDE such as VS Code. This also involves running scripts to build and preview the book, committing changes with Git, and pushing them to GitHub to publish updates.
- **With AI:** by using an AI assistant to help you write content, generate code, review spelling, create graphs, and so on. You can ask the AI to perform specific tasks or help you maintain the structure of the book while you write. We are no longer talking about code autocompletion, but about an assistant that understands the context of the book and helps you create quality content without needing to know how to program.

For this to work well, the project must be well structured and the AI must have all the information it needs to help you. That is why this template is designed to be easy to use with AI assistants, with a clear structure and simple scripts for building and publishing the book.

Vibe Coding: a development methodology that emphasizes a fluid programming experience through integrated AI assistants. It consists of writing code intuitively, letting AI agents edit the code and propose solutions while you keep control of the creative flow. The main problem with this methodology is that it can lead to excessive dependence on AI, which could affect code quality and the programmer's ability to solve problems independently. However, when used in a balanced way, it can significantly increase productivity and creativity.

As shown in [Fig. 3.1](#), the term was coined by Andrej Karpathy in a post that framed this way of working as useful for quick, exploratory projects.

In addition, figures such as Linus Torvalds, closely associated with well-crafted code, have shown interest in this new way of programming, while also raising concerns about the quality of AI-generated code and the need to keep human control over the process. [Fig. 3.2](#) illustrates that combination of practical interest and technical judgement.

3.1 How to Ask the AI for Help

This template is designed for use with AI assistants such as **GitHub Copilot** or **Antigravity**.

You can ask it things like:

- “Explain this Python code to me”
- “Generate a graph of a sine function”
- “Review the spelling of this paragraph”
- “Add a mathematical equation in LaTeX format”



Figure3.1: Andrej Karpathy's post introducing the term *vibe coding*.



Figure3.2: Linus Torvalds' comment on an integration done with help from Google Antigravity.

Or even more complex tasks, such as:

- Starting from a folder with your resources in the repository (PDFs, images, articles, videos), helping you write a complete book chapter that follows the structure and style you have defined.
- Translating the new chapter you have just written into another language while preserving the book's format and structure.
- Running all the commands needed to build the project, deploy it locally for preview, commit the changes with Git, and push them to GitHub to publish the updates.
- Regenerating the book PDF in several languages.
- Publishing the book.

As mentioned earlier, this project contains several SKILLS. You can think of them as cheat sheets or mini-tutorials for specific tasks that AI agents have available. They are located in the `.github/skills/` folder and are kept synchronized in other folders so that the project is compatible with other AI agents such as Claude, Codex, and others.

Taking this idea one step further, you can create your own SKILLS **by asking the assistant to generate them itself following the project's approach**. For example, if you want a SKILL for generating graphs with a library specific to your discipline, or for reviewing the spelling of a paragraph according to a particular style guide, you can ask the AI to generate that SKILL using the format of the existing project SKILLS. This way, you can build a set of custom SKILLS adapted to your needs and to the needs of your project.

4. MY FIRST BOOK FROM SCRATCH

This page shows a first complete workflow for creating a new book from this template: create the repository, open it on your computer, ask the agent for help, preview locally, generate PDFs, and publish the website.

The goal is not to learn Git, Python, LaTeX, or GitHub in depth before starting. The goal is to understand what is happening on each screen, what you can ask the agent to do, and where to look when you need to check that everything worked.

Note

The GIFs on this page are sped up so the process can be shown compactly. The initial environment setup and PDF generation can take much longer, especially the first time the LaTeX engine is installed.

4.1 Step 0. Prepare the basic requirements

Before working with the template, check that the computer has the basic tools ready: a GitHub account, Python installed, Git available, and an editor with an AI agent, such as Antigravity or VS Code. As shown in [Fig. 4.1](#), this step sets up the starting point before creating the repository.

GitHub will be the place where the book is stored on the internet. Python will run the scripts included in the template. Git is the tool that records the history of changes and sends them to GitHub. Antigravity or VS Code will be the workspace: you will open the book folder there, ask the agent to perform tasks, and review the files.

It is also important to configure Git for its first use. If you have never done it on that computer, open a terminal and run:

```
git config --global user.name "Your name"
git config --global user.email "your.email@example.com"
```

These details are attached to commits. A commit is a snapshot of the project at a specific moment. You do not write a password or token here: only your name and email address. The email should be associated with your GitHub account if you want GitHub to recognize those changes as yours.

If you do not know whether Git is already configured, you can check it with:

```
git config --global user.name
git config --global user.email
```

If those commands return your name and email, you can continue. If they return nothing, configure Git with the previous commands. This step usually only needs to be done once per computer.

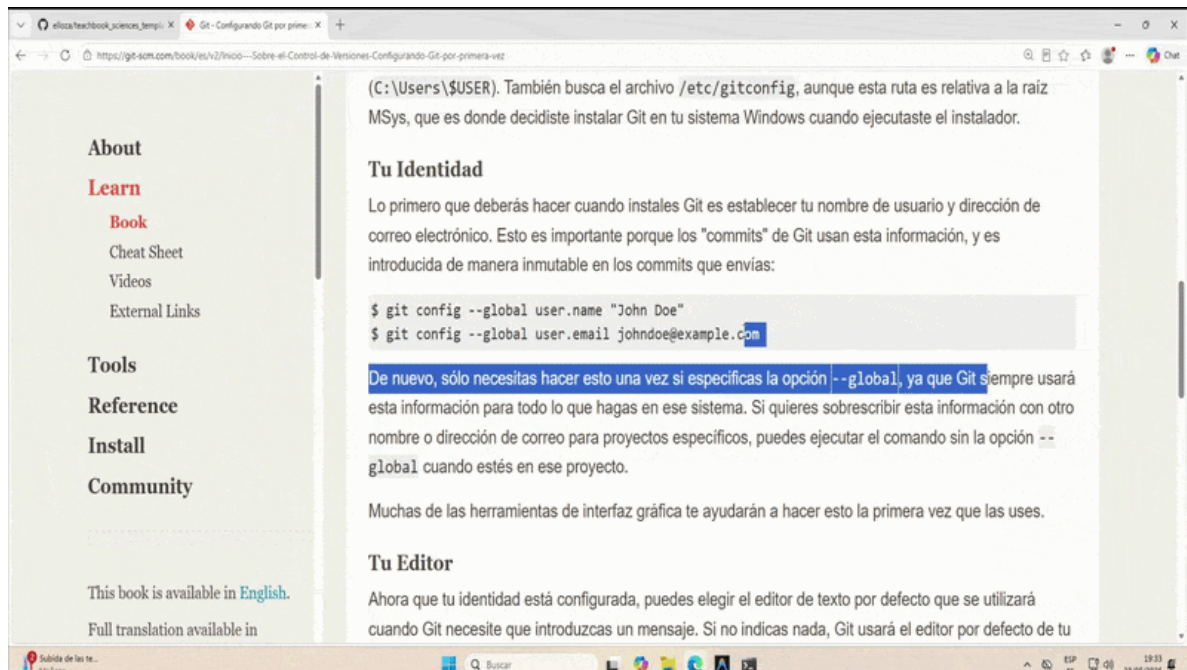


Figure4.1: Basic requirements: GitHub, Python, configured Git, and an editor with an AI agent.

4.2 Step 1. Create a repository from the template

The next step is to open the template URL on GitHub and use **Use this template** to create your own repository. In this first walkthrough we will make it **public**, as shown in Fig. 4.2, because that simplifies the initial publication with GitHub Pages. Private repositories can be considered later.

A repository is the project folder inside GitHub. When you use the template button, GitHub does not modify the original repository: it creates an independent copy in your account. That copy will be your book. You can change the title, remove examples, add chapters, and publish the website without affecting anyone else.

When GitHub asks for the repository details, pay special attention to three fields:

- **Owner:** the account or organization where the book will be created.
- **Repository name:** the technical name of the repository. Avoid spaces and unusual characters; for example, `my-first-book`.
- **Public:** for this first exercise, public is recommended because GitHub Pages is easier to activate.

After creating the repository, GitHub will take you to a page whose address looks similar to:

```
https://github.com/your-user/my-first-book
```

Keep that page in mind: it is the main GitHub page for your project. From there you can copy the clone URL, open **Settings**, inspect **Actions**, and check whether the website has been published correctly.

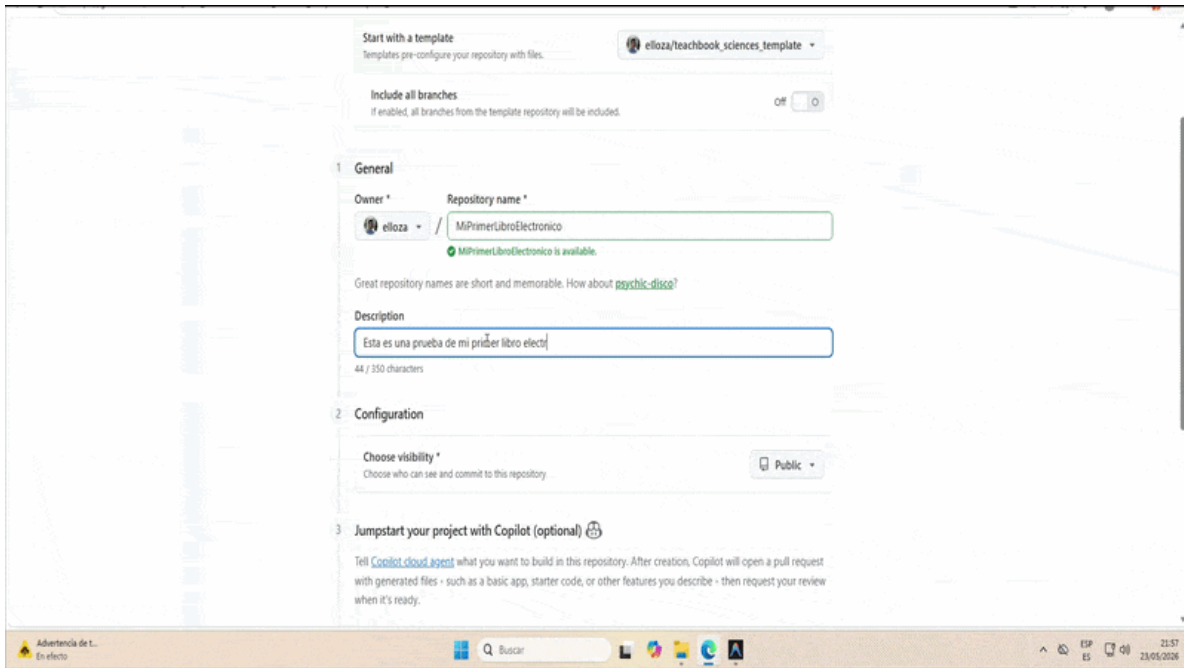


Figure4.2: Creation of a new public repository using GitHub's template button.

4.3 Step 2. Enable GitHub Actions and GitHub Pages

After creating the repository, allow GitHub to run the project workflows and publish the website. Fig. 4.3 shows the **GitHub Pages** configuration using **GitHub Actions** as the source.

GitHub Actions is GitHub's automation system. In this template it does in the cloud what you could do on your computer: install the environment, check the project, generate the PDFs, build the website, and publish it. GitHub Pages is the service that takes that built website and makes it available at a public URL.

To configure it, open your GitHub repository and go to:

```
Settings -> Pages -> Build and deployment -> Source -> GitHub Actions
```

If this is the first use of Actions in that account or repository, GitHub may ask for an additional confirmation to enable workflows. That detail may not appear in the GIF, but it must be accepted so the book can be built and published automatically.

The public website URL does not always appear immediately. It usually appears after the first successful deployment. There are two places where you can find it:

- In **Settings -> Pages**, GitHub usually shows a link to the published website once a successful deployment exists.
- In the **Actions** tab, open the deployment workflow run and look for the deployment summary. When everything is green, there is often a link to the published site.

The typical GitHub Pages URL for a project repository has this form:

```
https://your-user.github.io/my-first-book/
```

If the repository belongs to an organization, the first part changes to the organization name. If you later use a custom domain, such as `libro.usal.es`, that URL can be different.

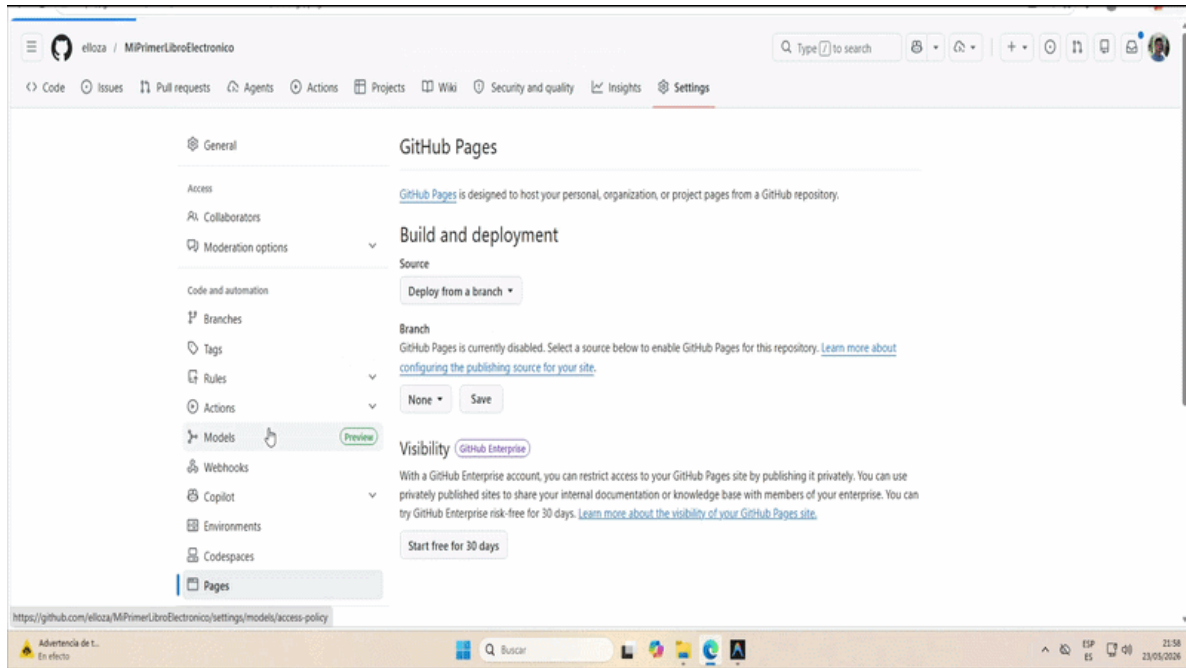


Figure4.3: Activation of GitHub Pages with GitHub Actions as the publication source.

4.4 Step 3. Clone the repository in AntigraVity

With the repository created on GitHub, copy its URL and clone it from AntigraVity. Fig. 4.4 shows the general flow: open the editor, choose the clone option, and paste the repository URL.

Cloning means downloading a complete copy of the repository to your computer. It is not just downloading a ZIP file: after cloning, the local folder remains connected to GitHub. That lets you work locally and, when you are satisfied, upload the changes with `commit` and `push`.

To find the URL you need, go back to the main page of the repository on GitHub and click the green **Code** button. In the **HTTPS** tab you will see an address similar to:

```
https://github.com/your-user/my-first-book.git
```

Copy that URL and paste it into AntigraVity's clone option. The editor will ask where on your computer you want to store the project. Choose a folder that is easy to find, such as `Documents`, `Desktop`, or a folder dedicated to your courses.

At this point GitHub may ask for authentication. That is expected: the editor needs permission to download the repository and, later, to push changes back. If a browser window or GitHub sign-in screen appears, follow the authentication flow. Do not paste passwords into project files.

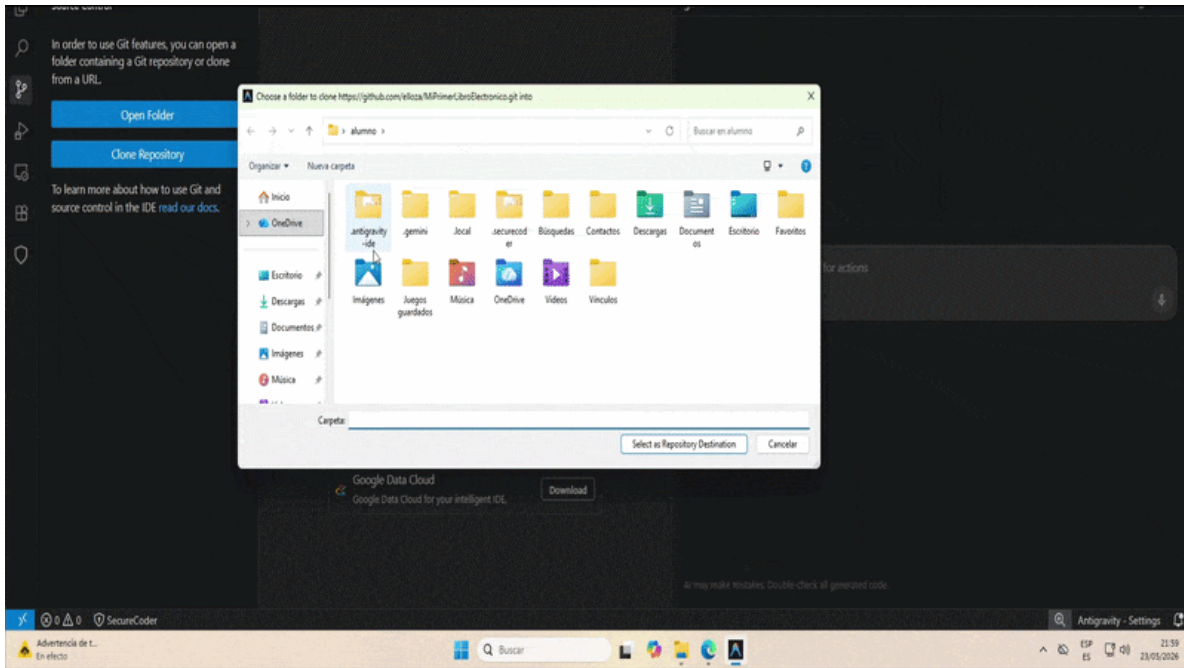


Figure4.4: Repository cloned from GitHub in Antigravity.

4.5 Step 4. Ask the agent to configure the environment

When the project is opened for the first time, ask the agent to prepare the environment. It is sensible to use a low-cost model, for example **Gemini 3.1 Pro (Low)** if available, because this task is fairly mechanical and may require several interactions.

A useful prompt would be:

```
Configure this project for the first time following the repository instructions.
Use the .venv environment and continue with the required installations.
```

The `.venv` environment is a local folder where the Python tools required by this book are installed. It prevents the project dependencies from being mixed with other programs on the computer. For a non-technical user, the key point is this: if the agent uses `.venv`, the project stays organized and the installation is easier to repeat on another machine.

As shown in Fig. 4.5, the agent will run commands and may ask several times whether it can continue. During this first setup, the usual answer is yes. It may ask for permission to install `uv`, create `.venv`, install dependencies, prepare LaTeX, or synchronize skills. Those questions are expected.

This step takes longer than the GIF. It depends on the connection, the computer, and whether Python, Git, or `uv` were already installed. During installation, avoid closing the editor or shutting down the computer. If it looks like nothing is happening for a few minutes, check whether the agent is waiting for confirmation in the terminal.

When it finishes, the project should normally have a `.venv` folder and the agent should be able to run the verification scripts. If something fails, do not try to fix it manually by copying random commands: ask the agent to read the error, review `AGENTS.mcd`, and run the setup again cleanly.

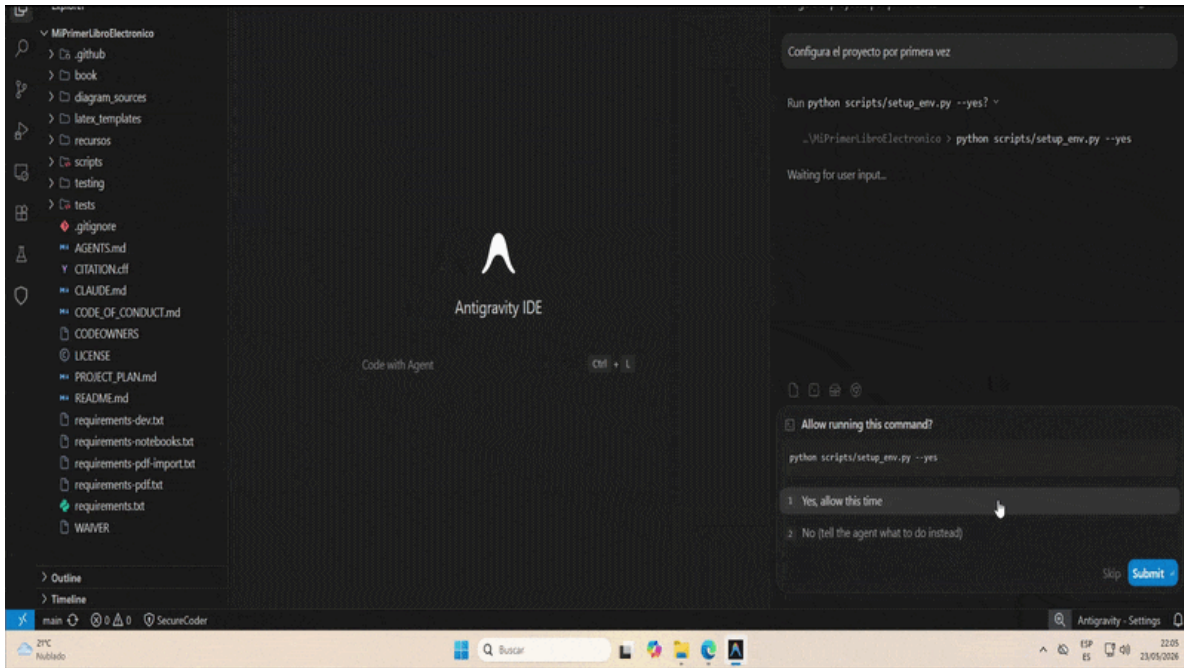


Figure 4.5: Initial project environment setup through the agent.

4.6 Step 5. Make a first edit with the agent

When the environment is ready, ask for a first real change. In the example shown in Fig. 4.6, the agent is asked to hide the current template chapters and leave only a sample chapter about the **Central Limit Theorem**, written with LaTeX and MyST.

This first change helps you learn the workflow. You do not need to start by writing the whole book. It is better to request a small modification, check that it appears in the local website, and then publish it. That way you understand the full cycle before investing many hours in content.

The request should clearly state that the change must be made in both languages:

Temporarily hide the current template chapters and leave only one sample chapter explaining the Central Limit Theorem with LaTeX and MyST. Do it in Spanish and English and keep the TOCs synchronized.

There are two important ideas in that prompt. The first is **hide**, not delete. The agent can remove sections from the menu by commenting or adjusting the TOCs, but it is useful to keep the example files because they may serve as references later. The second is **in Spanish and English**. This template is designed for multiple languages, so every visible change should have its equivalent in the configured languages.

It is also useful to ask the agent to validate the result:

When you finish, check encoding, multi-language structure, and build the website.

If the agent modifies a Spanish page but forgets the English one, the book can become inconsistent. That is why the automatic checks are part of the normal workflow, not an extra for advanced users.

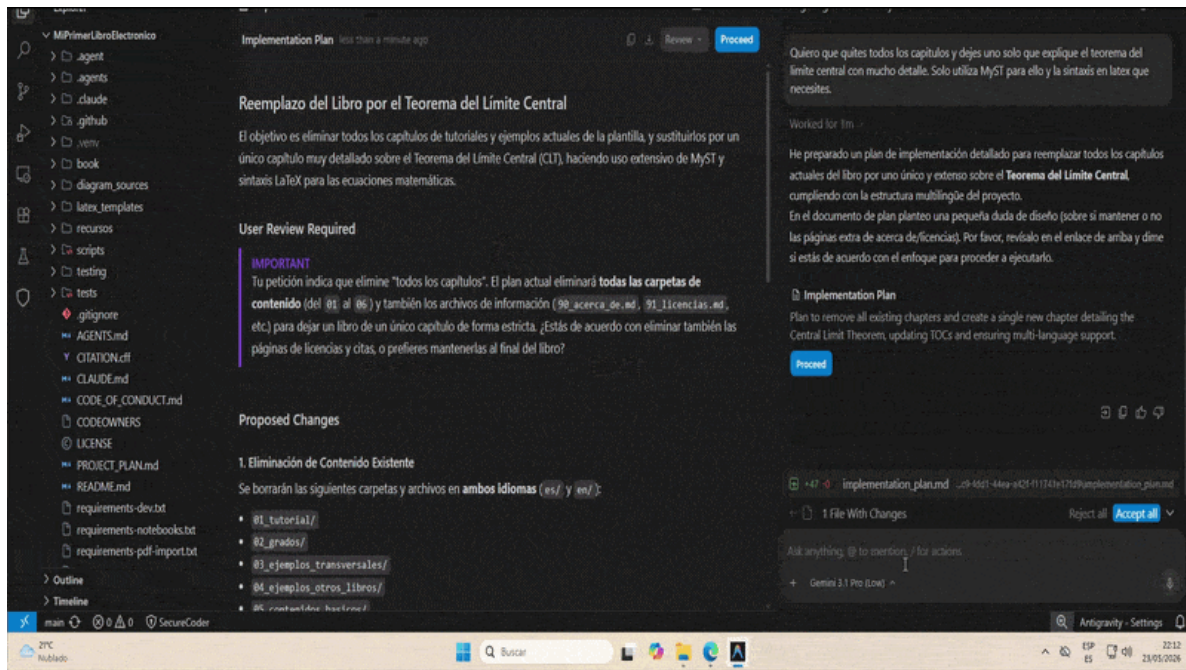


Figure 4.6: First assisted edit: hide example content and leave a minimal chapter in both languages.

4.7 Step 6. View the local preview

After editing, ask the agent to launch a local preview. Fig. 4.7 shows how the agent starts a server on the computer and then reports the URL where the book can be opened.

You can ask for it simply:

I want to see a local preview of the book. Start the server and tell me the exact URL.

The URL is usually local, for example `http://127.0.0.1:8000` or `http://localhost:8000`. That means the website is not published on the internet yet: it is being served from your own PC so you can review changes before uploading them. That address only works on your computer while the local server is running.

It is important to distinguish these two URLs:

- `http://localhost:8000` or `http://127.0.0.1:8000`: local preview on your PC.
- `https://your-user.github.io/my-first-book/`: website published on GitHub Pages.

If port 8000 is already in use, the agent may choose another one, such as 8001 or 8080. That is fine: open exactly the URL the agent gives you. If you see an old version, ask the agent to rebuild the book and restart the preview. In this template, the preview should be built with the same workflow as production, so you do not review an outdated website.

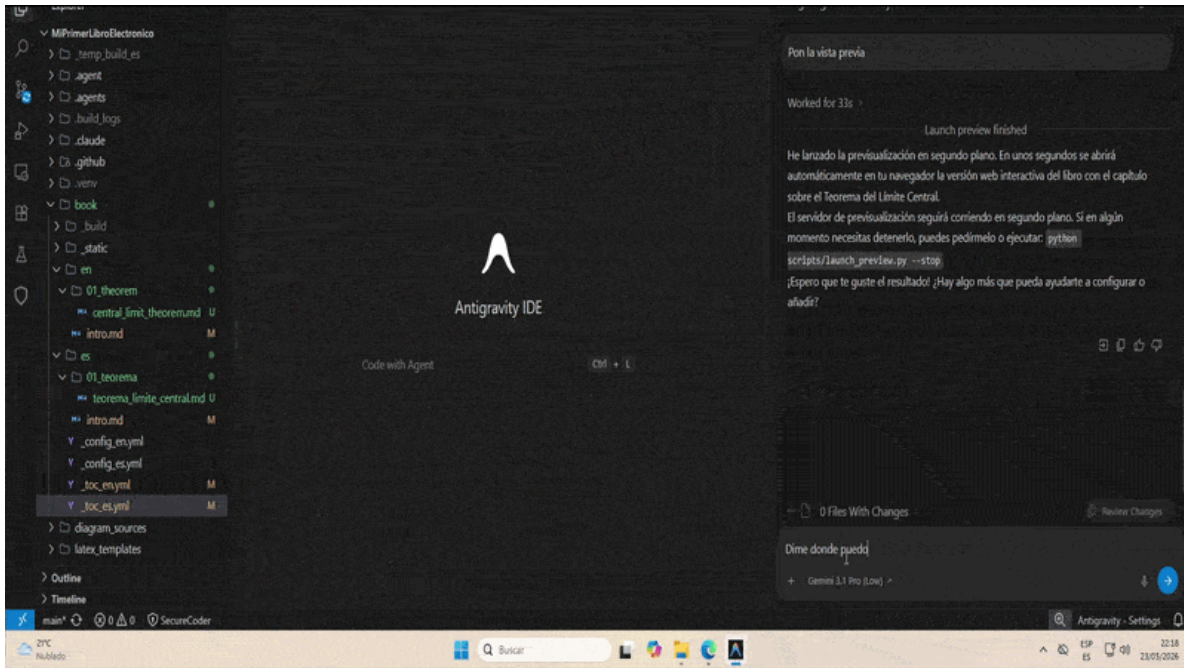


Figure4.7: Local book preview through a server running on the user's own computer.

4.8 Step 7. Generate the PDFs

When the local website looks right, ask the agent to generate the book PDFs. Fig. 4.8 shows the flow in compressed form, but this process may take longer than the GIF, especially the first time.

You can ask:

Generate the book PDFs in all languages and check that they were created correctly.

The template normally generates one PDF per language and places them in `book/_static/`, for example:

```
book/_static/teachbook_es.pdf
book/_static/teachbook_en.pdf
```

The reason for the initial wait is that the project needs a LaTeX engine to convert the book to PDF. LaTeX is the system that produces academic documents with good typography, equations, indexes, and references. You do not need to learn LaTeX to use the template, but the engine that compiles the document must be installed.

During the first generation it may look like many things are being installed. That is normal. Later runs are usually faster because the installation has already been prepared. If it fails, the important thing is to keep the error message and ask the agent to review it. It is not advisable to delete folders or reinstall tools manually without a diagnosis.

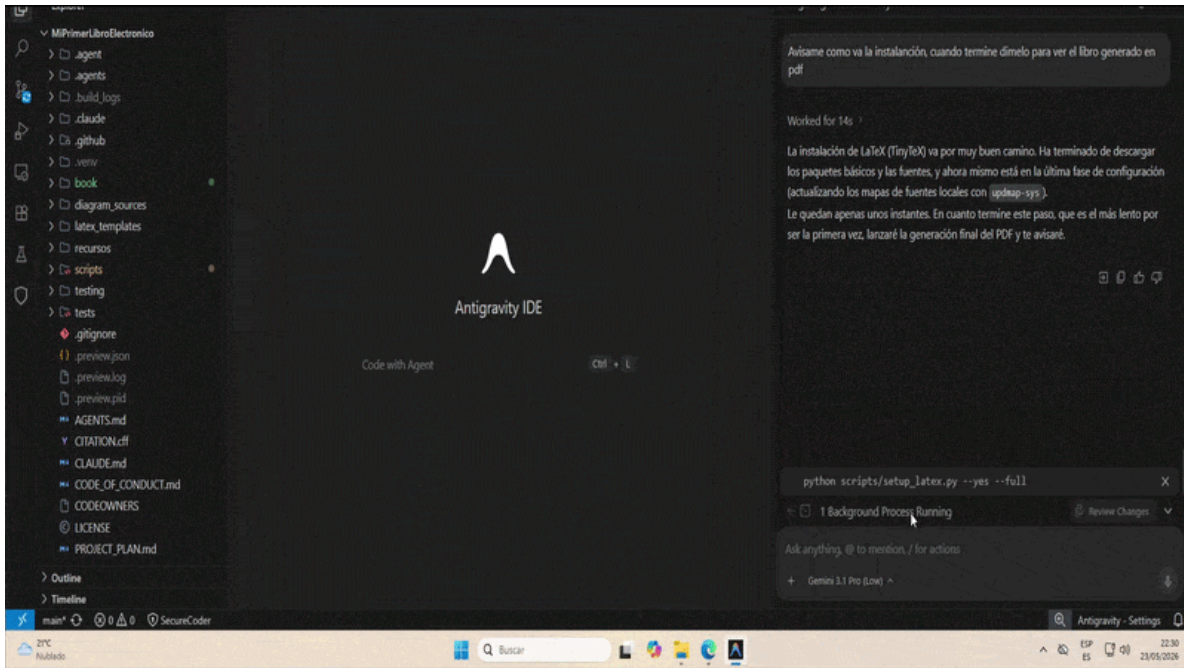


Figure4.8: Book PDF generation, including the initial LaTeX installation when required.

4.9 Step 8. Publish the changes

Finally, ask the agent to publish the changes. As shown in Fig. 4.9, publishing in this template means a complete Git cycle: add files, create a commit, and push to GitHub.

A good request would be:

Save the changes, commit, push, and check that the GitHub Pages deployment works.

The commit saves a specific version of the project with a descriptive message. The push uploads that version to GitHub. When it reaches GitHub, the deployment workflow starts. That workflow builds the book in the cloud, regenerates the PDFs, and publishes the website.

After the push, open your repository on GitHub and go to the **Actions** tab. There you will see a new workflow run. While it is running, it may appear in yellow or with a progress icon. When it finishes correctly, it appears in green. If it appears in red, the deployment failed and you need to open that run to read the error.

To find the published website URL, check these places:

- **Settings -> Pages:** after a successful deployment, GitHub usually shows the public site URL here.
- **Actions -> deployment run:** the run that published the website may include a deployment link.
- **Usual URL pattern:** if you do not use a custom domain, it will look similar to `https://your-user.github.io/my-first-book/`.

Keep in mind that the published website does not always update in the browser in the same second. If the workflow is green but you still see old content, wait a little and reload the page. You can also open a private browser window to avoid cache.

Because this template generates the PDFs first and then builds the website, the download buttons or links on the site should point to the updated PDFs. That is why manually uploading only HTML is not recommended: the full deployment keeps the website and printable documents synchronized.

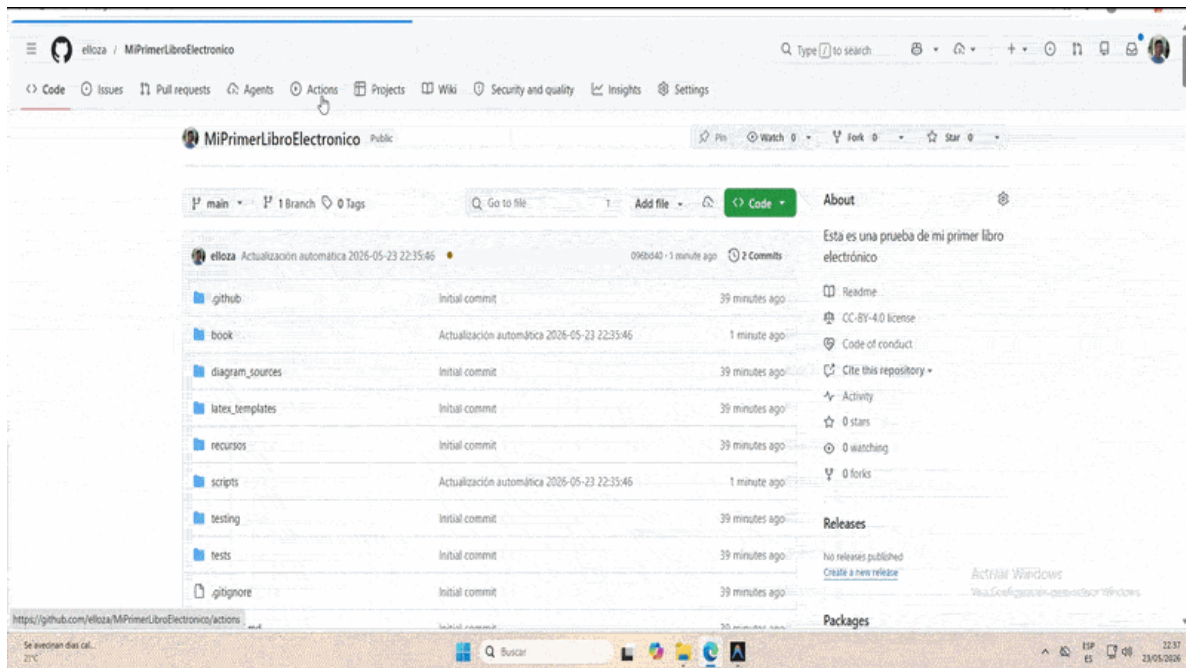


Figure4.9: Book publication through commit, push, and automatic deployment with GitHub Actions.

At the end of this walkthrough you have the basic full cycle: create the repository, edit with the agent, review locally, generate PDFs, and publish the website. From that point on, daily work consists of repeating the same pattern with smaller changes: write, preview, validate, and publish.

5. THE “X AS CODE” PHILOSOPHY

One of the important ideas we want you to take from this course is the possibility of editing and generating teaching artifacts through programming languages and markup languages. This is what we call the “X as code” philosophy. It means using code as a medium to describe and generate educational resources, so that we can benefit from the fact that LLMs can understand and manipulate that code to help us create quality materials without manually editing every artifact.

5.1 What is “X as code”?

When we talk about “X as code”, we mean the idea that an educational resource (X) can be described and generated through a programming language or a markup language. Instead of creating a diagram, plot, animation, or quiz directly in a visual tool, we describe it with code. Then that code is compiled or executed to generate the final artifact.

The idea is to use programming languages and markup languages as **intermediate languages** for generating teaching artifacts: images, diagrams, animations, videos, simulations, quizzes, tables, notebooks, and PDFs.

In other words: working with **X as code**.

- *Diagrams as code*: diagrams written as text.
- *Figures as code*: plots and figures generated from data or formulas.
- *Animations as code*: educational videos rendered from a programmed scene.
- *Quizzes as code*: questions, solutions, and exercise banks defined as structured text.
- *Books as code*: a complete book compiled from Markdown files, notebooks, and configuration.

Key idea

A teaching artifact generated from code is not edited “by hand” at the end of the process. It is described with an intermediate language, where AI assistants can understand the structure and content, and then the final result is generated through a compilation or execution process.

5.2 Why is this philosophy interesting?

One of the problems with generating content using AI tools is that the final result is often an image, PDF, or video that cannot be edited easily. If you want to change something, you have to regenerate the artifact from scratch or edit it manually, which can be complicated and time-consuming.

In the end, the advantage AI offers for generating content quickly is lost if the result is not editable. In contrast, if the artifact is generated from code, you can ask the AI to edit that code to make changes, generate new versions, or adapt the content to different contexts.

Having this intermediate language is sometimes key for AI to help you generate quality content. For example, if you want to generate a flowchart, you can describe it with Mermaid or Kroki code. The AI can understand that description and help you modify, translate, or adapt it without needing to edit the final image.

5.3 Code as a bridge to the final result

When you draw a diagram directly in a visual tool, the result is locked inside that interface. If you want to change ten arrows, translate labels, or adapt the style, you have to edit it manually again.

When you describe that same diagram with code, an intermediate layer appears:

As shown in Fig. 5.1, the diagram is versioned as a static image.

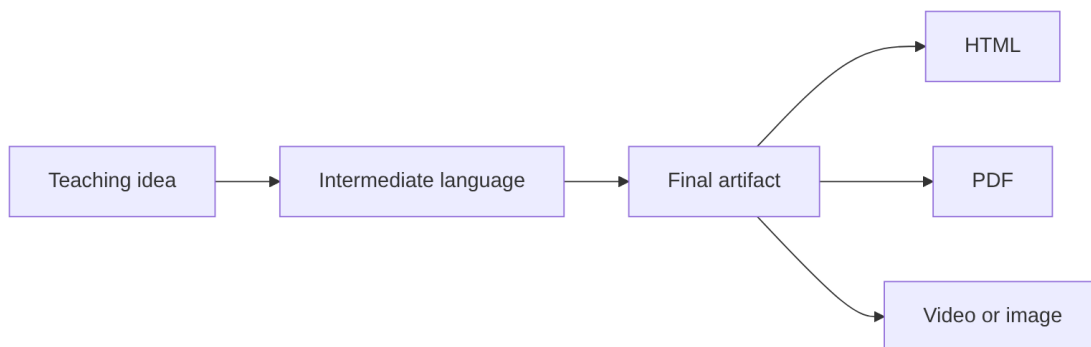


Figure5.1: Diagram: Code as a bridge to the final result.

The following table summarizes the main elements of this section.

Table. What artifacts can be generated from code?.

Final artifact	Intermediate language	Teaching use case
Diagram	Mermaid/Kroki, GraphViz, PlantUML	Concept map, experiment flow, process schema
Scientific plot	Python + matplotlib	Represent real or simulated data
Electrical circuit	Schemdraw or CircuitikZ	Generate consistent schematics for Physics problems
Animation or video	External tool + MP4	Explain a transformation, a wave, or a proof step by step without loading the base installation
Table or dataset	Python + pandas	Create clean tables from raw data
Quiz	MyST, JupyterQuiz, structured Markdown	Review questions with collapsible solutions
Executable notebook	Jupyter Notebook	Simulation, reproducible calculation, guided practice
Web book and PDF	Jupyter Book/TeachBooks	Publish the same material as a website, download, and printable version

5.4 Why this fits so well with AI

AI does not “understand” a final image the way it understands a code-based description. If you give it a screenshot of a diagram, it can suggest changes; if you give it the Mermaid or Kroki code that generates it, it can **make** those changes.

The practical advantage

We do not ask AI to edit pixels. We ask it to edit instructions. Then the system recompiles those instructions and produces the final artifact.

This makes it possible to ask for things like:

- “Turn this flowchart into a state diagram”.
- “Generate an English version while keeping the same structure”.
- “Adapt the example so it uses Biology data instead of Physics data”.
- “Prepare the visual script for a 20-second animation showing this function growing”.
- “Create three versions of the quiz with different difficulty levels”.

The key is that the final result is not handcrafted every time. It is produced from an editable recipe.

5.5 Reproducibility, not just automation

Working “as code” does not mean making things more complicated. It means every artifact has a clear recipe:

1. **Input:** data, formulas, text, or a teaching idea.
2. **Intermediate language:** Markdown, Python, Mermaid, LaTeX, etc.
3. **Compilation:** the system transforms that description.
4. **Output:** image, video, website, PDF, notebook, or quiz.

If you change the recipe, you regenerate the artifact. If something fails, you fix the recipe. If another teacher wants to adapt it, they do not start from scratch: they modify the recipe.

5.6 The mindset shift

The question stops being: “Which program should I use to draw this?”

The question becomes: “Which intermediate language best describes this artifact?”

Not every material needs code. But when a resource will be revised, translated, versioned, reused, or generated with AI support, it is worth asking whether it can be expressed as **X as code**.

That is the teaching-as-code philosophy: using code not as an end in itself, but as a **generative medium** between the teaching intention and the final artifact.

Part II

Examples by Degree

In this section, you will find examples adapted to Physics Degree courses. The goal is to show how to integrate:

- Complex mathematical formulas
- Plots of physical simulations
- Experimental data analysis code

6.1 Physics Example: Harmonic Oscillator

This notebook simulates a simple harmonic oscillator.

```
import numpy as np
import matplotlib.pyplot as plt

# Parameters
k = 1.0 # Spring constant
m = 1.0 # Mass
omega = np.sqrt(k / m)
t = np.linspace(0, 20, 100)

# Position
x = np.cos(omega * t)

# Plot
plt.figure(figsize=(8, 4))
plt.plot(t, x)
plt.title('Simple Harmonic Oscillator')
plt.xlabel('Time (s)')
plt.ylabel('Position (m)')
plt.grid(True)
plt.show()
```

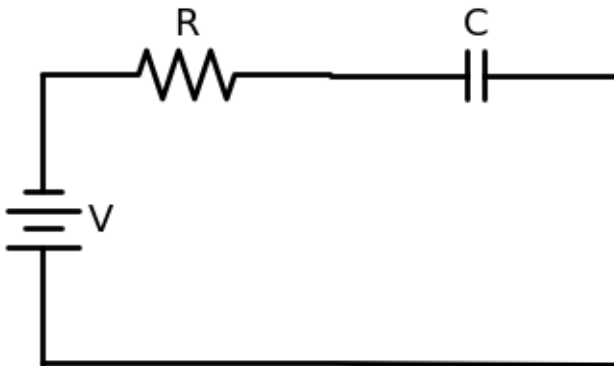
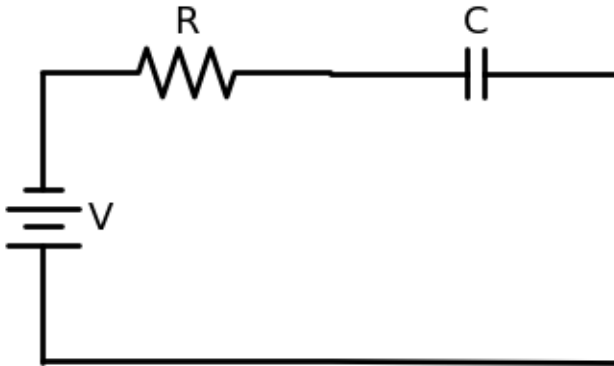
6.2 Drawing Circuits with SchemDraw

In this example we draw a series RC circuit (battery + resistor + capacitor) using the `schemdraw` library. This type of circuit is fundamental in electronics and appears in low-pass filters, timers, and coupling circuits.

```
import schemdraw
import schemdraw.elements as elm

with schemdraw.Drawing() as d:
    battery = d.add(elm.Battery().up().label('V', loc='bot'))
    d.add(elm.Resistor().right().label('R'))
    d.add(elm.Capacitor().right().label('C'))
    d.add(elm.Line().down())
    d.add(elm.Line().tox(battery.start).left())

d.draw()
```



6.2.1 Circuit Components

- **Battery (V):** provides the electromotive force that drives the current.
- **Resistor (R):** limits current flow and dissipates energy as heat.
- **Capacitor (C):** stores energy as an electric field between its plates.

The time constant of the circuit is $\tau = RC$, which determines the charging and discharging speed of the capacitor.

```
R = 1000    # 1 kOhm
C = 1e-6    # 1 uF
```

(continues on next page)

(continued from previous page)

```
tau = R * C
print(f'Time constant = RC = {tau*1e3:.2f} ms')
print(f'Time for full charge (~5 tau) = {5*tau*1e3:.2f} ms')
```

```
Time constant = RC = 1.00 ms
Time for full charge (~5 tau) = 5.00 ms
```

6.2.2 Conclusion

With `schemdraw` we can generate clean, professional circuit diagrams. The time constant $\tau = RC$ is the key parameter: after a time 5τ the capacitor is considered fully charged ($\approx 99.3\%$ of the source voltage).

6.3 Advanced CircuitikZ for Precise Circuits

CircuitikZ is the advanced option for drawing circuits with LaTeX-quality output. If you want highly precise schematics, similar to technical notes or scientific papers, this is the right tool.

Practical recommendation

Use **SchemDraw** for most teaching examples. Use **CircuitikZ** when you need a more formal and precise finish.

6.3.1 Recommended workflow in TeachBook

In this project, CircuitikZ is not inserted directly as `raw latex` in the page. Instead:

1. You write the circuit in a `.tex` file
2. You convert it to an image with a script
3. You insert the image with `{figure}`

This makes it work in both HTML and PDF.

6.3.2 Example source file

File `rc_circuit.tex`:

```
\begin{circuitikz}
\draw
  (0,0) to[battery1,l=$V$] (0,3)
  to[R,l=$R$] (3,3)
  to[C,l=$C$] (3,0)
  -- (0,0);
\end{circuitikz}
```

6.3.3 Command to render it

```
python scripts/render_circuitikz.py rc_circuit.tex book/_static/generated/rc_circuit.  
→png
```

6.3.4 Insert it in a MyST page

```
```{figure} ../../../../_static/generated/rc_circuit.png  
:alt: RC circuit generated with CircuitikZ
:width: 70%
:align: center
:name: fig-advanced-circuitikz-1

RC circuit generated with CircuitikZ.
```

```
Rendered result

The {numref}`fig-advanced-circuitikz-2` visually summarizes this part of the
→explanation.

```{figure} ../../../../_static/generated/rc_circuit_circuitikz.png  
:alt: RC circuit generated with CircuitikZ  
:width: 70%  
:align: center  
:name: fig-advanced-circuitikz-2  
  
RC circuit generated from a `.tex` file with CircuitikZ and converted to a PNG image.
```

The advantage of this workflow is that the final circuit behaves like a normal image: it appears on the website, is included in the PDF, and does not require the reader to have LaTeX installed.

6.3.5 When to use CircuitikZ

- formal electrical schematics
- advanced electronics notes
- material with a technical-paper style finish

6.3.6 When to use SchemDraw instead

- quick classroom examples
- introductory material
- situations where you want to edit the circuit with less friction

6.3.7 Prerequisite

If you do not have Tectonic installed yet:

```
python scripts/setup_latex.py --yes
```

6.4 Technical Diagrams with Kroki for Physics

Kroki is useful for **block diagrams, signal flow, timing diagrams, and conceptual schematics**. For **electrical circuits with standard symbols**, the recommended tool is **CircuitikZ rendered as an image**.

6.4.1 When to use each tool

- Use **Kroki** for conceptual explanations, functional blocks, timing, instrumentation, and signal flow.
- Use **WaveDrom through Kroki** for digital signals, buses, clocks, and timing protocols.
- Use **CircuitikZ** for resistors, capacitors, sources, switches, and formal electrical schematics.
- Use **SchemDraw** when you want to build simple circuits from Python inside a notebook.

6.4.2 1. Mermaid: instrumentation chain

As shown in Fig. 6.1, the diagram is versioned as a static image.

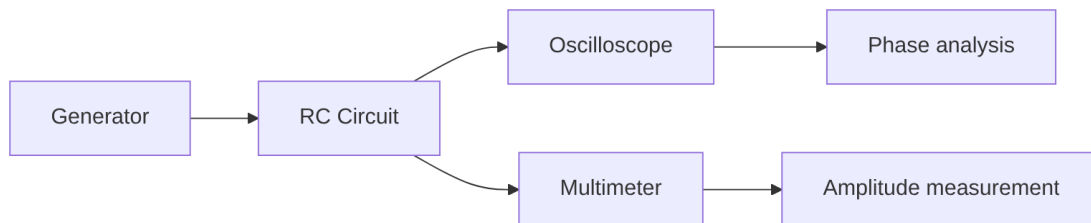


Figure 6.1: Diagram: Mermaid: instrumentation chain.

As shown in Fig. 6.2, the diagram is versioned as a static image.

The Fig. 6.3 visually summarizes this part of the explanation.

```

\begin{circuitikz}
\draw
  (0,0) to[V, l=$5\,\mathrm{V}$] (0,3)
  to[R, l=$R_1$] (3,3)
  to[C, l=$20\,\mu\mathrm{F}$] (3,0)
  -- (0,0);
\end{circuitikz}
  
```

i Important

Kroki supports TikZ, but the public Kroki service does not guarantee that the `circuitikz` package is available. For robust teaching circuits, this project renders CircuitikZ locally as an image.

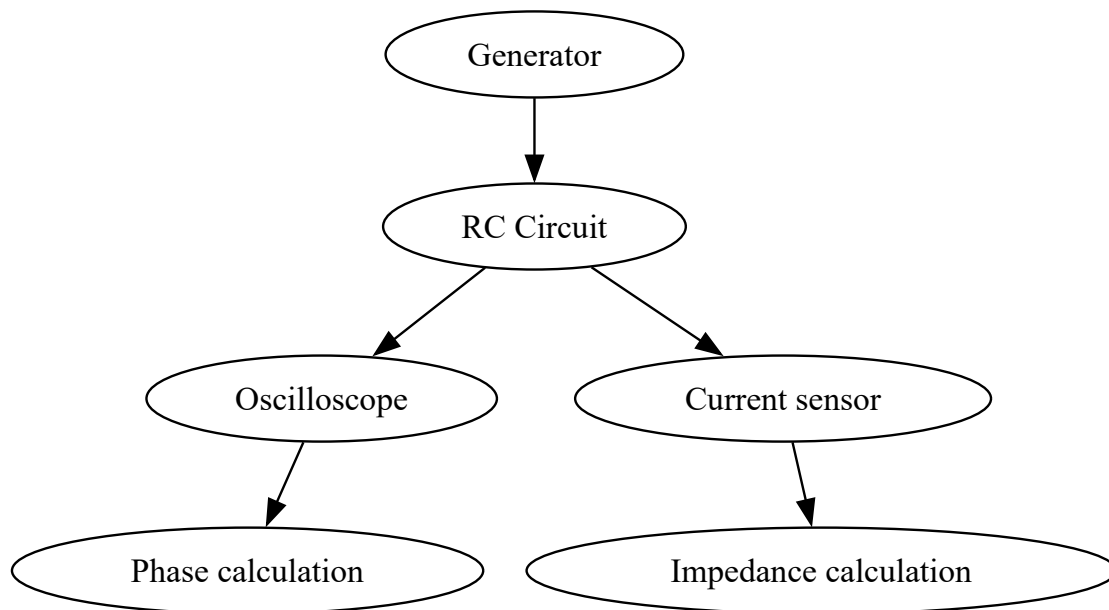


Figure6.2: Diagram: GraphViz: signal flow in an experimental setup.

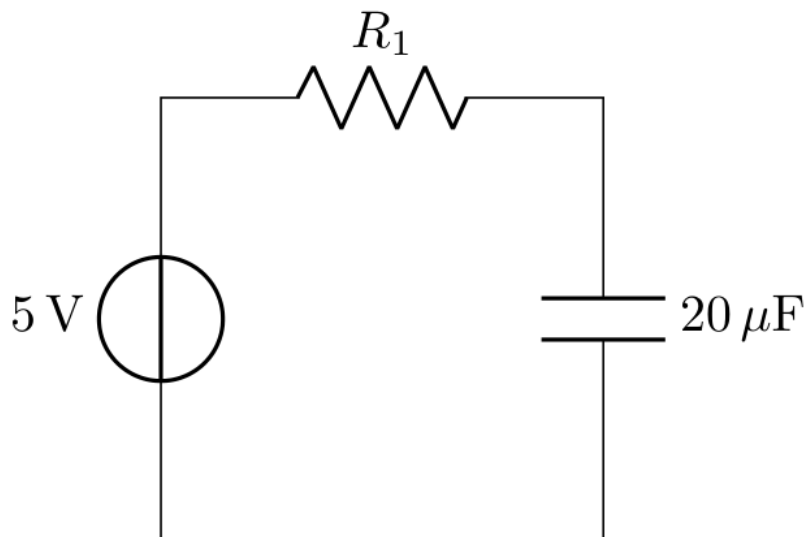


Figure6.3: RC circuit generated from CircuitikZ code.

6.4.3 4. Wavedrom: input and output signals

As shown in Fig. 6.4, the diagram is versioned as a static image.

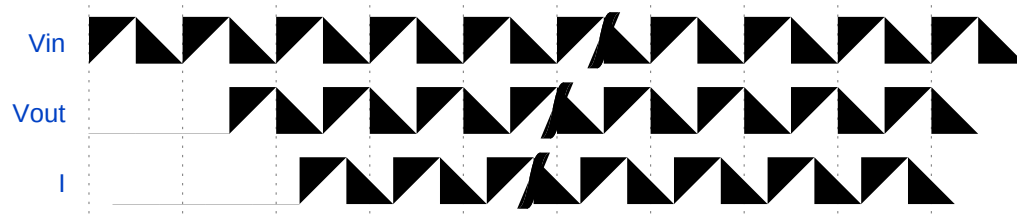


Figure6.4: Diagram: Wavedrom: input and output signals.

As shown in Fig. 6.5, the diagram is versioned as a static image.

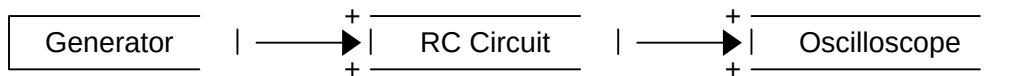


Figure6.5: Diagram: Dita: quick experimental bench sketch.

- **Kroki** = system-level visual explanation, blocks, flows, and timing diagrams.
- **CircuitikZ** = formal circuit with standard symbols and LaTeX-quality output.
- **SchemDraw** = programmable circuit from Python, ideal for teaching notebooks.

6.5 Ohm's Law: Linear Fit of V-I Data

We generate synthetic voltage-current data with noise and fit a linear regression $V = IR$ to verify Ohm's Law. We calculate the R^2 coefficient to evaluate the quality of the fit.

```
import numpy as np
import matplotlib.pyplot as plt

R_true = 220.0 # True resistance in
np.random.seed(42)

I = np.linspace(0.001, 0.05, 30)
V_true = R_true * I
noise = np.random.normal(0, 0.3, size=V_true.shape)
V_measured = V_true + noise

coef = np.polyfit(I, V_measured, 1)
R_fitted = coef[0]
V_fit = np.polyval(coef, I)

SS_res = np.sum((V_measured - V_fit) ** 2)
SS_tot = np.sum((V_measured - np.mean(V_measured)) ** 2)
R2 = 1 - SS_res / SS_tot

print(f'True resistance:      {R_true:.1f}')
```

(continues on next page)

(continued from previous page)

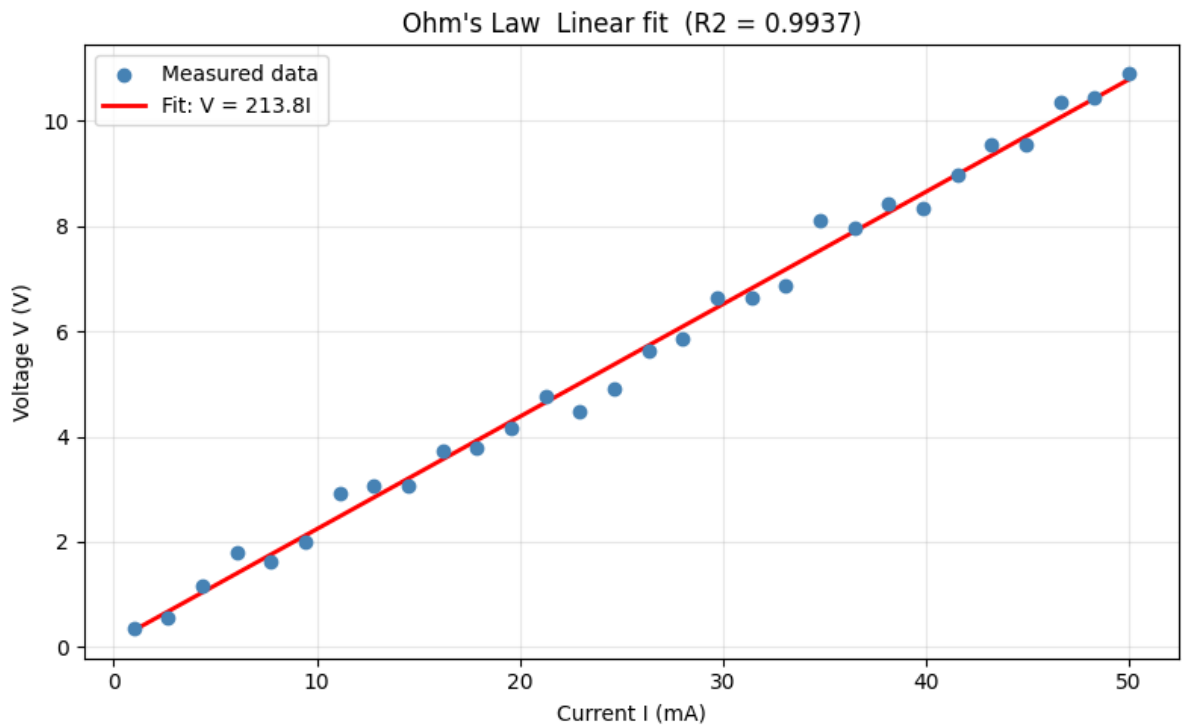
```
print(f'Fitted resistance:   {R_fitted:.1f} ')
print(f'R2 = {R2:.6f}')
```

```
True resistance:   220.0
Fitted resistance:  213.8
R2 = 0.993681
```

6.5.1 Visualization

The plot shows experimental data (dots) and the fitted line. An R^2 close to 1 indicates that the linear model explains the variability in the data.

```
plt.figure(figsize=(8, 5))
plt.scatter(I * 1000, V_measured, color='steelblue', label='Measured data', zorder=3)
plt.plot(I * 1000, V_fit, 'r-', linewidth=2, label=f'Fit: V = {R_fitted:.1f}I')
plt.xlabel('Current I (mA)')
plt.ylabel('Voltage V (V)')
plt.title(f"Ohm's Law Linear fit (R2 = {R2:.4f})")
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```



6.5.2 Conclusion

The linear fit recovers the resistance value with great accuracy. The $R^2 \approx 1$ coefficient confirms that the $V = IR$ relationship is linear, validating Ohm's Law for this synthetic dataset.

6.6 Harmonic Oscillator: Simple Harmonic Motion

Simple harmonic motion describes systems that oscillate periodically:

$$x(t) = A \cos(\omega t + \varphi)$$

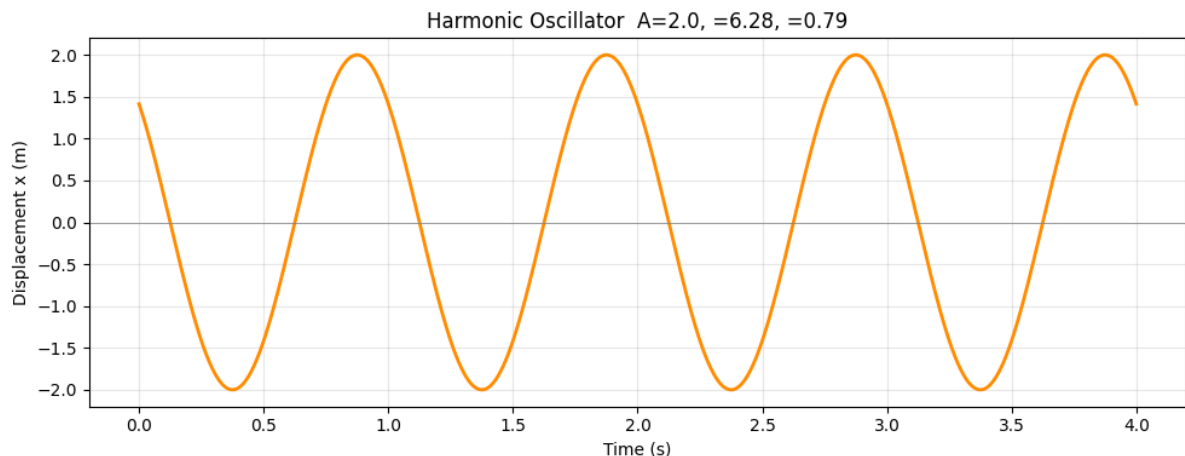
where A is the amplitude, ω the angular frequency, and φ the initial phase. By modifying these parameters we can explore different oscillatory behaviors.

```
import numpy as np
import matplotlib.pyplot as plt

A = 2.0           # Amplitude (m)
omega = 2*np.pi  # Angular frequency (rad/s)  1 Hz
phi = np.pi/4    # Initial phase (rad)

t = np.linspace(0, 4, 1000)
x = A * np.cos(omega * t + phi)

plt.figure(figsize=(10, 4))
plt.plot(t, x, linewidth=2, color='darkorange')
plt.axhline(0, color='gray', linewidth=0.5)
plt.xlabel('Time (s)')
plt.ylabel('Displacement x (m)')
plt.title(f'Harmonic Oscillator A={A}, ={\omega:.2f}, ={\phi:.2f}')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```



6.6.1 Effect of Parameters

You can experiment by changing A , ω , and ϕ in the cell above:

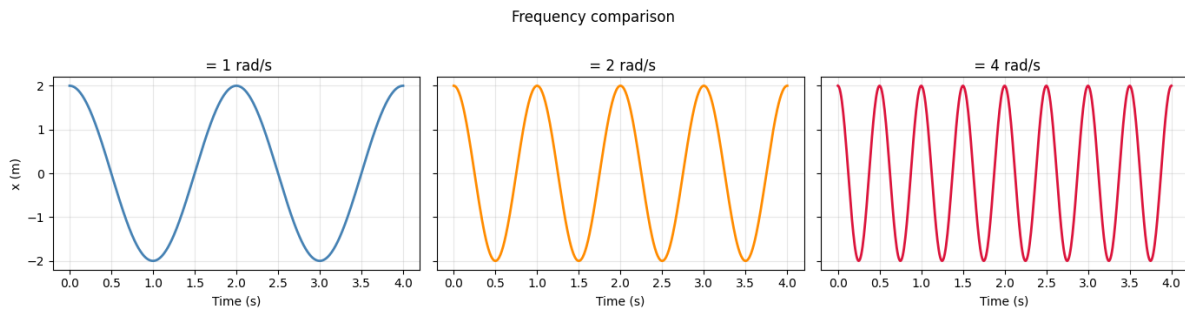
- **Amplitude (A):** controls the maximum height of the oscillation.
- **Angular frequency (ω):** controls the oscillation speed ($T = 2\pi/\omega$).
- **Initial phase (ϕ):** shifts the curve horizontally.

```
A = 2.0
phi = 0
t = np.linspace(0, 4, 500)

fig, axes = plt.subplots(1, 3, figsize=(14, 3.5), sharey=True)

for ax, omega_val, color in zip(axes, [np.pi, 2*np.pi, 4*np.pi],
                                ['steelblue', 'darkorange', 'crimson']):
    x = A * np.cos(omega_val * t + phi)
    ax.plot(t, x, linewidth=2, color=color)
    ax.set_title(f' = {omega_val/np.pi:.0f} rad/s')
    ax.set_xlabel('Time (s)')
    ax.grid(True, alpha=0.3)

axes[0].set_ylabel('x (m)')
plt.suptitle('Frequency comparison', y=1.02)
plt.tight_layout()
plt.show()
```



6.6.2 Conclusion

The simple harmonic oscillator is the basic model for oscillatory phenomena. Real systems such as pendulums (for small angles), springs, and LC circuits follow this behavior. The angular frequency $\omega = \sqrt{k/m}$ connects the motion to the physical properties of the system.

Examples for mathematics courses. Priorities here include:

- Rigor in mathematical notation (LaTeX)
- Visual proofs
- Symbolic algorithms (SymPy)

7.1 Mathematics Example: Symbolic Calculus

We will use SymPy to differentiate and integrate functions.

```
from sympy import symbols, diff, integrate, sin, exp

x = symbols('x')
f = exp(-x) * sin(x)

# Derivative
df = diff(f, x)
print(f"Derivative: {df}")

# Indefinite Integral
int_f = integrate(f, x)
print(f"Integral: {int_f}")
```

7.2 Quadratic Equation

The quadratic equation is a useful example for combining mathematical explanation, formulas, and tables in a page that works in both HTML and PDF.

We start from the general form:

$$ax^2 + bx + c = 0, \quad a \neq 0$$

The behavior of its solutions depends on the discriminant:

$$\Delta = b^2 - 4ac$$

When $\Delta \geq 0$, the real solutions are computed with:

$$x = \frac{-b \pm \sqrt{\Delta}}{2a}$$

7.2.1 Interpreting the discriminant

Value of Δ	Type of solutions	Geometric interpretation
$\Delta > 0$	Two distinct real solutions	The parabola intersects the x -axis at two points
$\Delta = 0$	One repeated real solution	The parabola is tangent to the x -axis
$\Delta < 0$	Two complex conjugate solutions	The parabola does not intersect the x -axis

7.2.2 Example

For the equation:

$$x^2 - 5x + 6 = 0$$

we have $a = 1$, $b = -5$, and $c = 6$. Therefore:

$$\Delta = (-5)^2 - 4 \cdot 1 \cdot 6 = 1$$

Since $\Delta > 0$, there are two real solutions:

$$x_1 = \frac{5+1}{2} = 3, \quad x_2 = \frac{5-1}{2} = 2$$

Tip

This static example is intentionally lightweight: it requires no video dependencies and exports reliably to both the website and the PDF.

7.3 Newton's Method for Finding Roots

The Newton-Raphson method is an iterative algorithm for finding roots of $f(x) = 0$:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

We apply it to the equation $x^3 - 2x - 5 = 0$, which has a real root near $x = 2$.

```
import numpy as np
import matplotlib.pyplot as plt

def f(x):
    return x**3 - 2*x - 5

def df(x):
    return 3*x**2 - 2

x0 = 2.0
steps = [x0]
for _ in range(5):
    x_new = steps[-1] - f(steps[-1]) / df(steps[-1])
    steps.append(x_new)
```

(continues on next page)

(continued from previous page)

```
for i, x in enumerate(steps):
    print(f'x_{i} = {x:.10f}    f(x) = {f(x):.2e}')
```

```
x_0 = 2.0000000000    f(x) = -1.00e+00
x_1 = 2.1000000000    f(x) = 6.10e-02
x_2 = 2.0945681211    f(x) = 1.86e-04
x_3 = 2.0945514817    f(x) = 1.74e-09
x_4 = 2.0945514815    f(x) = -8.88e-16
x_5 = 2.0945514815    f(x) = -8.88e-16
```

7.3.1 Convergence Visualization

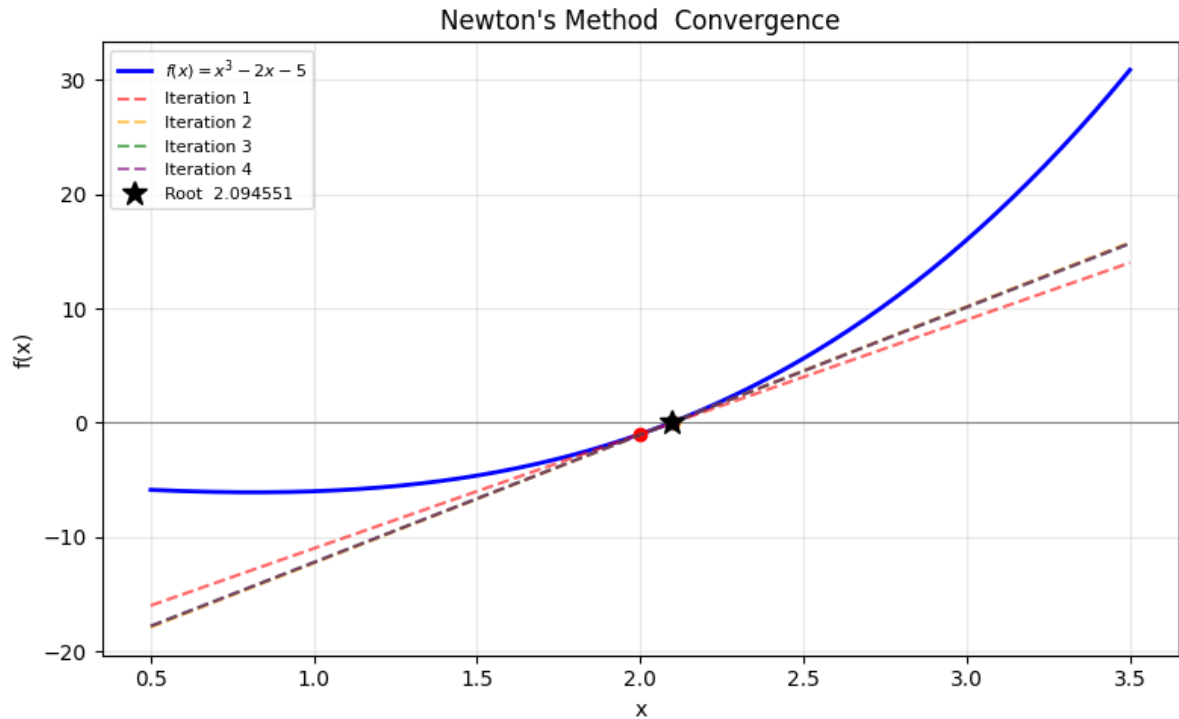
The plot shows the function $f(x)$ and the tangent lines used at each iteration. Each tangent intersects the x -axis at the next approximation of the root.

```
x_plot = np.linspace(0.5, 3.5, 300)

plt.figure(figsize=(8, 5))
plt.plot(x_plot, f(x_plot), 'b-', linewidth=2, label=r'$f(x) = x^3 - 2x - 5$')
plt.axhline(0, color='gray', linewidth=0.8)

colors = ['red', 'orange', 'green', 'purple']
for i in range(min(4, len(steps)-1)):
    xi = steps[i]
    tangent = f(xi) + df(xi) * (x_plot - xi)
    plt.plot(x_plot, tangent, '--', color=colors[i], alpha=0.6,
             label=f'Iteration {i+1}')
    plt.plot(xi, f(xi), 'o', color=colors[i], markersize=6)

plt.plot(steps[-1], 0, 'k*', markersize=12, label=f'Root {steps[-1]:.6f}')
plt.xlabel('x')
plt.ylabel('f(x)')
plt.title("Newton's Method Convergence")
plt.legend(fontsize=8)
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```



7.3.2 Conclusion

Newton's method converges very quickly (quadratic convergence): in just 4-5 iterations we reach a precision of 10^{-10} . A good starting point is required, and $f'(x_n) \neq 0$ must hold at each step.

7.4 2D Linear Transformations

A linear transformation in $\mathbb{R}^2$ is represented by a 2×2 matrix. Applied to the unit square, it lets us visualize how the transformation distorts space. We explore three classic examples: rotation, scaling, and shear.

```
import numpy as np
import matplotlib.pyplot as plt

square = np.array([[0,0],[1,0],[1,1],[0,1],[0,0]]).T

theta = np.pi / 4
R_rot = np.array([[np.cos(theta), -np.sin(theta)],
                  [np.sin(theta),  np.cos(theta)]])
R_scl = np.array([[2, 0], [0, 0.5]])
R_shr = np.array([[1, 0.5], [0, 1]])

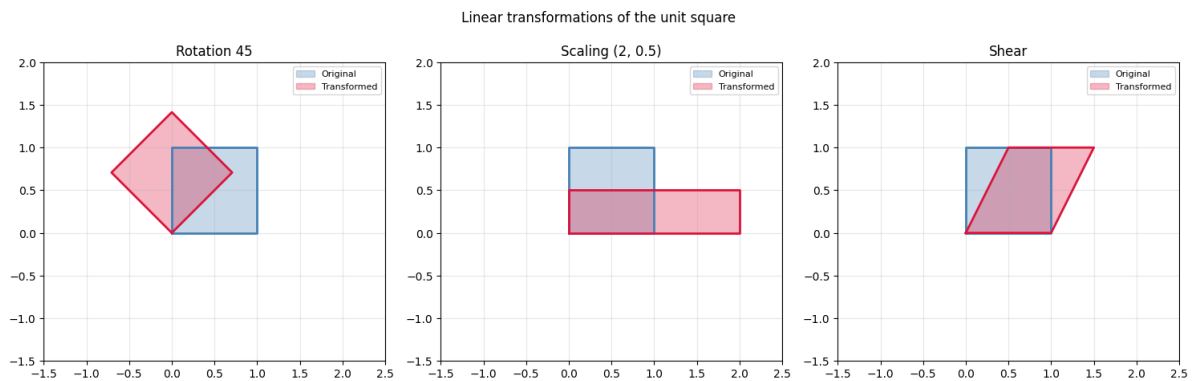
transforms = [
    (R_rot, 'Rotation 45'),
    (R_scl, 'Scaling (2, 0.5)'),
    (R_shr, 'Shear'),
]
```

(continues on next page)

(continued from previous page)

```
fig, axes = plt.subplots(1, 3, figsize=(15, 4.5))
for ax, (M, title) in zip(axes, transforms):
    transformed = M @ square
    ax.fill(square[0], square[1], alpha=0.3, color='steelblue', label='Original')
    ax.plot(square[0], square[1], 'steelblue', linewidth=2)
    ax.fill(transformed[0], transformed[1], alpha=0.3, color='crimson', label=
↳ 'Transformed')
    ax.plot(transformed[0], transformed[1], 'crimson', linewidth=2)
    ax.set_xlim(-1.5, 2.5)
    ax.set_ylim(-1.5, 2.0)
    ax.set_aspect('equal')
    ax.set_title(title)
    ax.legend(fontsize=8)
    ax.grid(True, alpha=0.3)

plt.suptitle('Linear transformations of the unit square', y=1.02)
plt.tight_layout()
plt.show()
```



7.4.1 Interpretation

- **Rotation:** preserves areas and angles (orthogonal transformation, $\det = 1$).
- **Scaling:** stretches one axis and compresses another ($\det = 2 \times 0.5 = 1$).
- **Shear:** shifts one axis proportionally to the other ($\det = 1$, preserves area).

```
for M, name in transforms:
    print(f'{name}: det = {np.linalg.det(M) :.2f}')
```

```
Rotation 45: det = 1.00
Scaling (2, 0.5): det = 1.00
Shear: det = 1.00
```

7.4.2 Conclusion

The matrix determinant tells us whether the transformation preserves area ($|\det| = 1$), expands it ($|\det| > 1$), or contracts it ($|\det| < 1$). Visualizing the effect on the unit square is the most intuitive way to understand any linear transformation.

7.5 Planar Point Sets with Many Unit Distances

This page explains the manuscript *Planar Point Sets with Many Unit Distances* [OpenAI26] from the perspective of a mathematics course. The goal is not to reproduce every technical detail from algebraic number theory, but to make the architecture of the proof clear: an arithmetic construction in number fields eventually produces finite planar point sets with many pairs at distance exactly one.

7.5.1 The Unit-Distance Problem

Let $P \subset \mathbb{R}^2$ be a finite point set. Define

$$\nu(P) = \#\{\{p, q\} \subset P : \|p - q\|_2 = 1\},$$

the number of unordered pairs of points of P at Euclidean distance one. Also define

$$\nu(n) = \max_{|P|=n} \nu(P),$$

the largest possible number of unit distances among n planar points.

The conjecture discussed in the manuscript goes back to Erdős. In one asymptotic form, it predicted an absolute constant C such that, for all sufficiently large n ,

$$\nu(n) \leq n^{1+C/\log \log n}. \quad (7.1)$$

The main result of the manuscript asserts the opposite: there is a fixed constant $\delta > 0$ and infinitely many values of n for which

$$\nu(n) \geq n^{1+\delta}. \quad (7.2)$$

Equation (7.2) is not merely a small improvement over a known estimate. It says that, along an infinite sequence of sizes, the number of unit distances grows with an exponent strictly larger than one.

Conceptual reading

The argument is not an elementary recipe for drawing a small example by hand. It is an existence proof: it builds planar point sets from infinite towers of number fields and then projects them to the plane.

7.5.2 Proof Map

The proof has two halves. The first is geometric: if we have enough algebraic norm-one elements, they can be interpreted as translations of length one. The second is arithmetic: it constructs number fields in which many such elements exist.

Table. Logical map of the argument.

Block	Main object	Contribution
Planar problem	Finite set $P \subset \mathbb{R}^2$	We want many pairs at distance 1
Geometry of numbers	Minkowski lattice in $\mathbb{C}^f$	Counts points and translations inside a bounded window
Arithmetic	Fields $L, K = L(i)$ and split primes	Produces many elements u with $u c(u) = 1$
Projection	One complex coordinate $\mathbb{C}^f \rightarrow \mathbb{C}$	Turns algebraic translations into planar unit segments

7.5.3 Admissible Data

The geometric part starts with an admissible datum:

- a totally real field L of degree $f = [L : \mathbb{Q}]$;
- the CM field $K = L(i)$, with relative complex conjugation c ;
- distinct rational primes $q_1, \dots, q_t$, all congruent to 1 modulo 4 and splitting completely in L .

If $Q = \prod_b q_b$, the complete splitting of these primes produces many conjugate pairs of prime ideals in K :

$$\{P_s, cP_s\}, \quad s = 1, \dots, m, \quad m = tf.$$

For each binary vector $\varepsilon \in \{0, 1\}^m$, choose an ideal

$$A_\varepsilon = \prod_{\varepsilon_s=1} P_s \prod_{\varepsilon_s=0} cP_s.$$

There are 2^m choices but only $h(K)$ ideal classes. By the pigeonhole principle, many of those choices lie in the same ideal class. Comparing one such choice with another gives elements

$$u_\varepsilon = \frac{\alpha_\varepsilon}{c(\alpha_\varepsilon)}$$

which satisfy

$$u_\varepsilon c(u_\varepsilon) = 1. \quad (7.3)$$

Since L is totally real, under any complex embedding $\sigma : K \hookrightarrow \mathbb{C}$ the automorphism c becomes ordinary complex conjugation. Therefore (7.3) implies

$$|\sigma(u_\varepsilon)| = 1.$$

This is the key idea: the algebraic elements constructed in the proof have length one in every relevant complex coordinate.

7.5.4 Minkowski Window and Counting

Choose a Minkowski embedding

$$\Phi : K \longrightarrow \mathbb{C}^f, \quad \Phi(x) = (\sigma_1(x), \dots, \sigma_f(x)).$$

The fractional ideal $D^{-1}\mathcal{O}_K$, with $D = Q^2$, becomes a lattice $\Lambda \subset \mathbb{C}^f$. The elements $u \in U$ constructed above belong to this lattice and have every coordinate of modulus one.

Now take a window

$$B_R = \{z \in \mathbb{C}^f : \|z\|_\infty \leq R\},$$

which is a product of discs of radius R . For a translate $a + \Lambda$, consider the points

$$X_a = (a + \Lambda) \cap B_R.$$

The manuscript averages over the torus $\mathbb{C}^f/\Lambda$. The intuition is simple: if the lattice is shifted at random, the expected number of points inside the window is proportional to the volume of B_R , and the expected number of pairs $(x, x + u)$ with $u \in U$ depends on the overlap of B_R with $B_R - u$.

If $\gamma = t \log 2 - \log H > 0$, this averaging gives a translate with many pairs:

$$E_a \geq e^{\gamma f/2} |X_a|. \quad (7.4)$$

Then $X_a \subset \mathbb{C}^f$ is projected to the first complex coordinate. The projection is injective on the lattice coset being used: if two points have the same first coordinate, their difference comes from an algebraic integer with one zero embedding, and hence it is zero. This gives a planar set

$$P = \pi_1(X_a) \subset \mathbb{C} \simeq \mathbb{R}^2.$$

Every pair counted in (7.4) projects to a segment of length one, because the first coordinate of each $u \in U$ has modulus one.

7.5.5 Controlling the Size of P

To turn the estimate into a statement about $n = |P|$, one must control how many points fit inside the window. The manuscript uses a packing argument: two distinct lattice points cannot be too close in every coordinate at once, because the product of their coordinate differences is controlled by the algebraic norm.

This gives a uniform exponential upper bound

$$|P| \leq e^{Bf}$$

for a constant B fixed by the parameters. Together with the unit-pair estimate, this gives

$$\nu(P) \geq \frac{1}{2} |P| e^{\gamma f/2}.$$

Since $|P| \leq e^{Bf}$, the factor $e^{\gamma f/2}$ can be rewritten as a positive power of $|P|$. Absorbing constants for large degree yields

$$\nu(P) \geq |P|^{1+\delta}$$

for some $\delta > 0$.

7.5.6 Arithmetic Construction of the Fields

The arithmetic step produces a sequence of totally real fields

$$F = F_0 \subset F_1 \subset F_2 \subset \dots$$

with degrees $f_j = [F_j : \mathbb{Q}]$ tending to infinity. The tower is everywhere unramified and has Galois groups of 3-power order. This matters for two reasons:

1. because the tower is unramified, the root discriminant stays controlled;
2. because the proof works with pro-3 groups, the generator-relation count can keep the tower infinite.

The starting point is a cyclic cubic field F built from cubic characters and cyclotomic fields. Rational primes q_i are then chosen by Chebotarev so that they split completely in every level of the tower. To impose this splitting, the proof kills Frobenius classes lying in the Frattini subgroup. The Golod-Shafarevich inequality ensures that, even after adding those relations, the resulting pro-3 group remains infinite.

Finally, one sets

$$K_j = F_j(i).$$

The root discriminant of K_j is bounded independently of j , and a Minkowski class-number estimate gives

$$h(K_j) \leq H^{f_j}.$$

The parameter t is chosen on the order of ℓ^2 , while $\log H$ grows only like $O(\ell \log \ell)$. Therefore, for large ℓ ,

$$t \log 2 - \log H > 0.$$

This inequality supplies the positive parameter $\gamma > 0$ used in the geometric part.

Student takeaway

The proof looks abstract because it combines combinatorial geometry, lattices, number fields, and pro- p groups. A useful way to read it is to follow the gains and losses: one gains 2^{tf} choices of ideals, loses a controlled class-number factor, and then turns the remaining exponential gain into many planar unit distances.

7.5.7 Bibliographic Context

The original problem and the classical unit-distance bounds are connected to Erdős, Kővári-Sós-Turán, Spencer-Szemerédi-Trotter, Székely, Ágoston-Pálvolgyi, and Guth-Katz [ErdHos46, GK15, KHovariSosTuran54, SSzemerédiT84, Szekely97, AgostonPalvolgyi22]. Variants for norms and related convexity problems appear in work by Brass, Brass-Moser-Pach, Eisenbrand-Pach-Rothvoß-Sopher, Bília-Buchin-Fulek-Kiyomi-Okamoto-Tanigawa-Tóth, Matoušek, Alon-Bucić-Sauermann, Greilhuber-Schildkraut-Tidor, Valtr, and Erdős-Fishburn [ABucicS25, Bra96, BMP05, BíliaBF+10, EP RothvossS08, ErdHosF97, GST25, Matoušek11, Val05]. The algebraic part uses standard tools from class field towers, pro- p theory, Chebotarev, cyclotomic fields, discriminants, and class numbers [Dav00, DdSMS99, GS64, GS65, HM01, HMR21, Koc02, Lan94, Neu99, NSW08, RZ10, Sha63, Sha66, Tsc26, Was97].

7.5.8 Page References

DEGREE IN STATISTICS

Examples oriented towards data analysis and probability. Highlights:

- DataFrame handling (Pandas)
- Statistical visualization (Seaborn)
- Probabilistic models

8.1 Statistics Example: Data Generation

We generate a normal distribution and visualize its histogram.

```
import numpy as np
import matplotlib.pyplot as plt

data = np.random.normal(0, 1, 1000)

plt.hist(data, bins=30, alpha=0.7, color='green', density=True)
plt.title('Normal Distribution Histogram')
plt.xlabel('Values')
plt.ylabel('Density')
plt.show()
```

8.2 Linear Regression with Synthetic Data

We generate bivariate data with an underlying linear relationship and Gaussian noise. We fit a linear regression model using least squares and evaluate the fit quality with R^2 and residual analysis.

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(12)
n = 50
x = np.linspace(2, 12, n)
y_true = 3.5 * x + 10
y = y_true + np.random.normal(0, 5, n)

coef = np.polyfit(x, y, 1)
y_fit = np.polyval(coef, x)
residuals = y - y_fit
```

(continues on next page)

(continued from previous page)

```

SS_res = np.sum(residuals**2)
SS_tot = np.sum((y - np.mean(y))**2)
R2 = 1 - SS_res / SS_tot

print(f'Fitted equation: y = {coef[0]:.3f}x + {coef[1]:.3f}')
print(f'R2 = {R2:.4f}')

```

```

Fitted equation: y = 3.350x + 9.963
R2 = 0.7609

```

8.2.1 Scatter plot with regression line

```

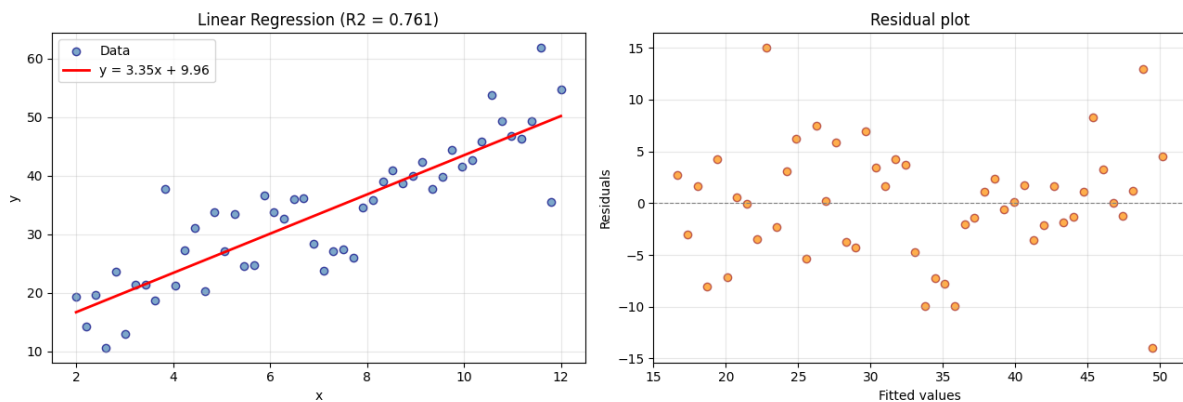
fig, axes = plt.subplots(1, 2, figsize=(13, 4.5))

axes[0].scatter(x, y, alpha=0.7, color='steelblue', edgecolors='navy', label='Data')
axes[0].plot(x, y_fit, 'r-', linewidth=2, label=f'y = {coef[0]:.2f}x + {coef[1]:.2f}')
axes[0].set_xlabel('x')
axes[0].set_ylabel('y')
axes[0].set_title(f'Linear Regression (R2 = {R2:.3f})')
axes[0].legend()
axes[0].grid(True, alpha=0.3)

axes[1].scatter(y_fit, residuals, alpha=0.7, color='darkorange', edgecolors='brown')
axes[1].axhline(0, color='gray', linewidth=0.8, linestyle='--')
axes[1].set_xlabel('Fitted values')
axes[1].set_ylabel('Residuals')
axes[1].set_title('Residual plot')
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

```



8.2.2 Conclusion

The residual plot allows us to verify regression assumptions: if residuals are randomly scattered around zero, the linear model is adequate. A pattern in the residuals would indicate a missing nonlinear term.

8.3 Estimating π with Monte Carlo

The Monte Carlo method estimates π by throwing random points in a unit square $[0, 1] \times [0, 1]$. The fraction that falls inside the quarter circle (radius 1) approximates $\pi/4$:

$$\pi \approx 4 \cdot \frac{N_{\text{inside}}}{N_{\text{total}}}$$

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(42)
N = 5000
x = np.random.uniform(0, 1, N)
y = np.random.uniform(0, 1, N)

inside = x**2 + y**2 <= 1
pi_est = 4 * np.sum(inside) / N

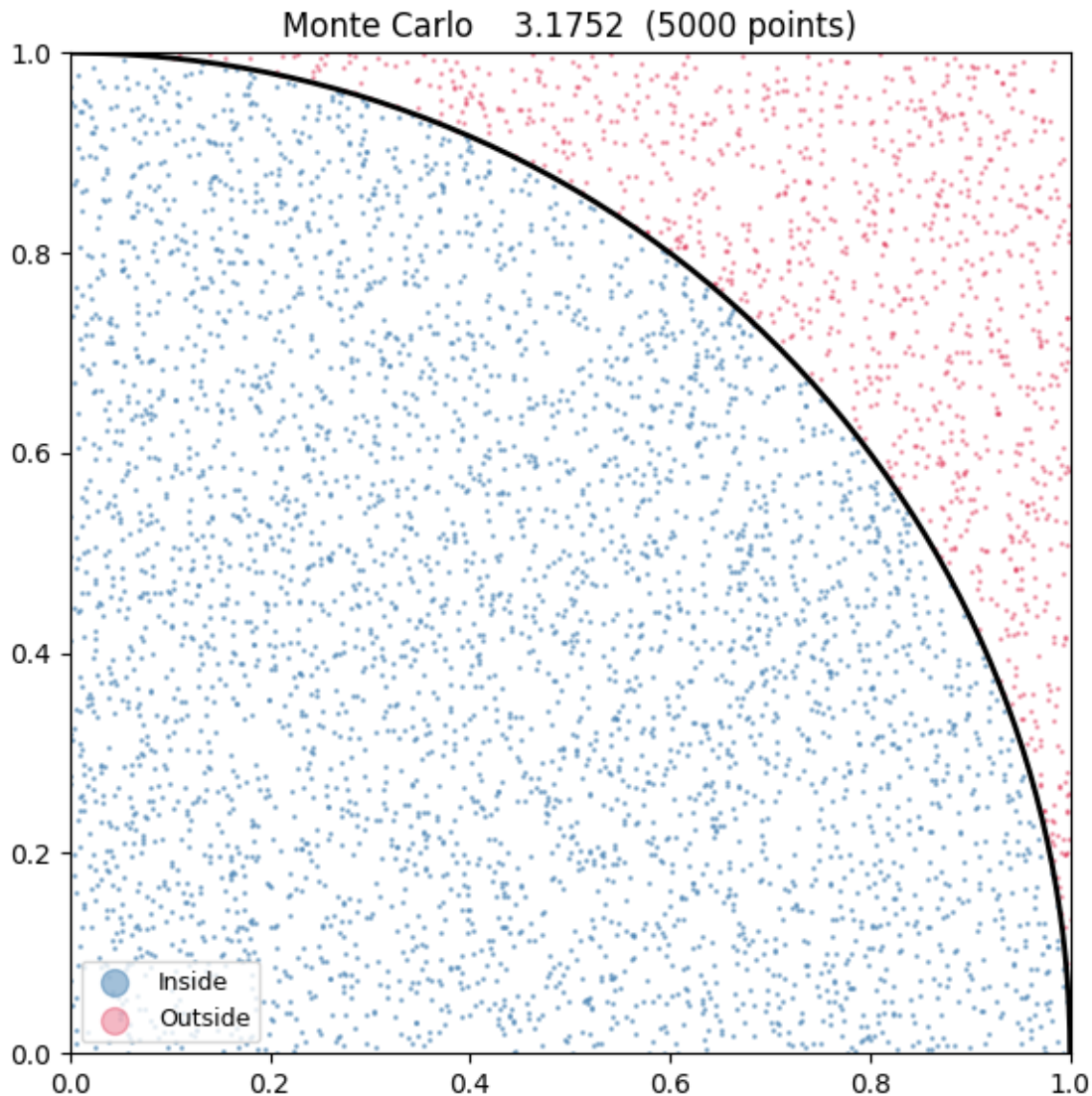
print(f'Total points:      {N}')
print(f'Inside circle:     {np.sum(inside)}')
print(f' estimate:          {pi_est:.6f}')
print(f'True value:          {np.pi:.6f}')
print(f'Absolute error:      {abs(pi_est - np.pi):.6f}')
```

```
Total points:      5000
Inside circle:     3969
 estimate:          3.175200
True value:        3.141593
Absolute error:     0.033607
```

8.3.1 Point Visualization

Blue points fall inside the quarter circle, red points outside. The black curve is the boundary $x^2 + y^2 = 1$.

```
plt.figure(figsize=(6, 6))
plt.scatter(x[inside], y[inside], s=1, color='steelblue', alpha=0.5, label='Inside')
plt.scatter(x[~inside], y[~inside], s=1, color='crimson', alpha=0.3, label='Outside')
theta = np.linspace(0, np.pi/2, 200)
plt.plot(np.cos(theta), np.sin(theta), 'k-', linewidth=2)
plt.xlim(0, 1)
plt.ylim(0, 1)
plt.gca().set_aspect('equal')
plt.title(f'Monte Carlo {pi_est:.4f} ({N} points)')
plt.legend(markerscale=10, fontsize=9)
plt.tight_layout()
plt.show()
```



8.3.2 Convergence as N Increases

The estimate improves as we increase the number of points. The error decreases as $O(1/\sqrt{N})$, characteristic of Monte Carlo methods.

8.4 Practice Questions: Statistics

Test your knowledge of fundamental statistics concepts. Expand each question to reveal the answer.

Question 1 — p-value

What exactly does a p-value of 0.03 mean in a hypothesis test?

A p-value of 0.03 means that, **if the null hypothesis H_0 were true**, the probability of observing a result as extreme or more extreme than the one obtained would be 3%.

Important: It is NOT the probability that H_0 is true. It is the probability of the data (or more extreme) given H_0 . Since $0.03 < 0.05$ (the typical significance level), we would reject H_0 .

Question 2 — Confidence interval

Interpret a 95% confidence interval for a mean: [12.3, 15.7].

It does NOT mean the population mean has a 95% probability of being in that interval. The correct frequentist interpretation is:

If we repeat the experiment many times, 95% of the intervals constructed with the same method will contain the true parameter value.

For this particular interval, the population mean is either inside or outside — there is no intermediate probability. What has 95% confidence is the **construction method**.

Question 3 — Normal distribution

What proportion of data falls within ± 2 standard deviations of the mean in a normal distribution?

Using the empirical rule (68-95-99.7 rule):

The following table summarizes the main elements of this section.

Table. Practice Questions: Statistics.

Range	Proportion
$\mu \pm 1\sigma$	$\approx 68.27\%$
$\mu \pm 2\sigma$	$\approx 95.45\%$
$\mu \pm 3\sigma$	$\approx 99.73\%$

Therefore, approximately **95.45%** of data falls within $\mu \pm 2\sigma$.

Question 4 — Correlation vs. Causation

If two variables have a correlation of $r = 0.85$, can you conclude that one causes the other?

No. Correlation does not imply causation. A strong correlation only indicates linear association. Possible explanations include:

1. X **causes** Y — possible but not proven by r alone.
2. Y **causes** X — the direction could be reversed.
3. A **confounding variable** Z causes both.
4. **Coincidence** — especially with small datasets.

Establishing causation requires experimental studies (randomized controlled trials), not just observational data.

Question 5 — Type I and Type II Errors

Define Type I and Type II errors. Which one is directly controlled by the α level?

The following table summarizes the main elements of this section.

Table. Practice Questions: Statistics.

Type	Occurs when...	Called
Type I Error	We reject H_0 when it is true	False positive
Type II Error	We fail to reject H_0 when it is false	False negative

The **significance level** α directly controls the probability of committing a Type I error: $P(\text{Type I}) = \alpha$.

The probability of a Type II error is denoted β , and the power of a test is $1 - \beta$. Decreasing α generally increases β .

Question 6 — Central Limit Theorem

State the Central Limit Theorem and explain its practical importance.

The **Central Limit Theorem (CLT)** states that, for sufficiently large samples ($n \geq 30$, approximately), the distribution of the sample mean $\bar{X}$ approaches a normal distribution:

$$\bar{X} \sim N\left(\mu, \frac{\sigma^2}{n}\right)$$

regardless of the original population distribution.

Practical importance: It justifies using the normal distribution to construct confidence intervals and perform hypothesis tests about the mean, even when the population is not normal. It is the foundation of much of inferential statistics.

DEGREE IN COMPUTER SCIENCE

In this section, you will find examples adapted to Computer Science Degree courses. The goal is to show how to integrate:

- Database diagrams (E-R model)
- UML class diagrams
- Algorithmic complexity analysis with executable code

Applicable subjects

This block can be adapted to programming, databases, and software engineering courses.

9.1 Entity-Relationship Diagram with Kroki (Mermaid)

E-R diagrams are fundamental in database design. With Kroki and `:type: mermaid`, you can include them directly in your book and make them work in both HTML and PDF.

9.1.1 Example: University database

Code:

```
```${kroki}
:type: mermaid
:align: center

erDiagram
 STUDENT {
 int id PK
 string name
 string email
 }
 COURSE {
 int id PK
 string name
 int credits
 }
 PROFESSOR {
 int id PK
 string name
```

(continues on next page)

(continued from previous page)

```

 string department
 }
 ENROLLMENT {
 int id PK
 float grade
 string term
 }

 STUDENT ||--o{ ENROLLMENT : "enrolls in"
 COURSE ||--o{ ENROLLMENT : "has"
 PROFESSOR ||--o{ COURSE : "teaches"
 ...

```

#### Result:

As shown in Fig. 9.1, the diagram is versioned as a static image.

- **Relationship types:** Use `||--o{` (one-to-many), `||--||` (one-to-one), `}o--o{` (many-to-many).
- **Primary keys:** Mark attributes with PK.
- **Foreign keys:** Mark attributes with FK.

### 9.1.2 Cardinality reference

The following table summarizes the main elements of this section.

**Table. Cardinality reference.**

Notation	Meaning
`	
`	
}o--o{	Many to many

## 9.2 UML Class Diagram with Kroki (Mermaid)

Class diagrams are essential for modeling object-oriented systems. With Kroki and `:type: mermaid`, you can represent classes, attributes, methods, and inheritance in both HTML and PDF.

### 9.2.1 Example: Geometric Shape Hierarchy

#### Code:

```

```{kroki}
:type: mermaid
:align: center

classDiagram
    class Shape {

```

(continues on next page)

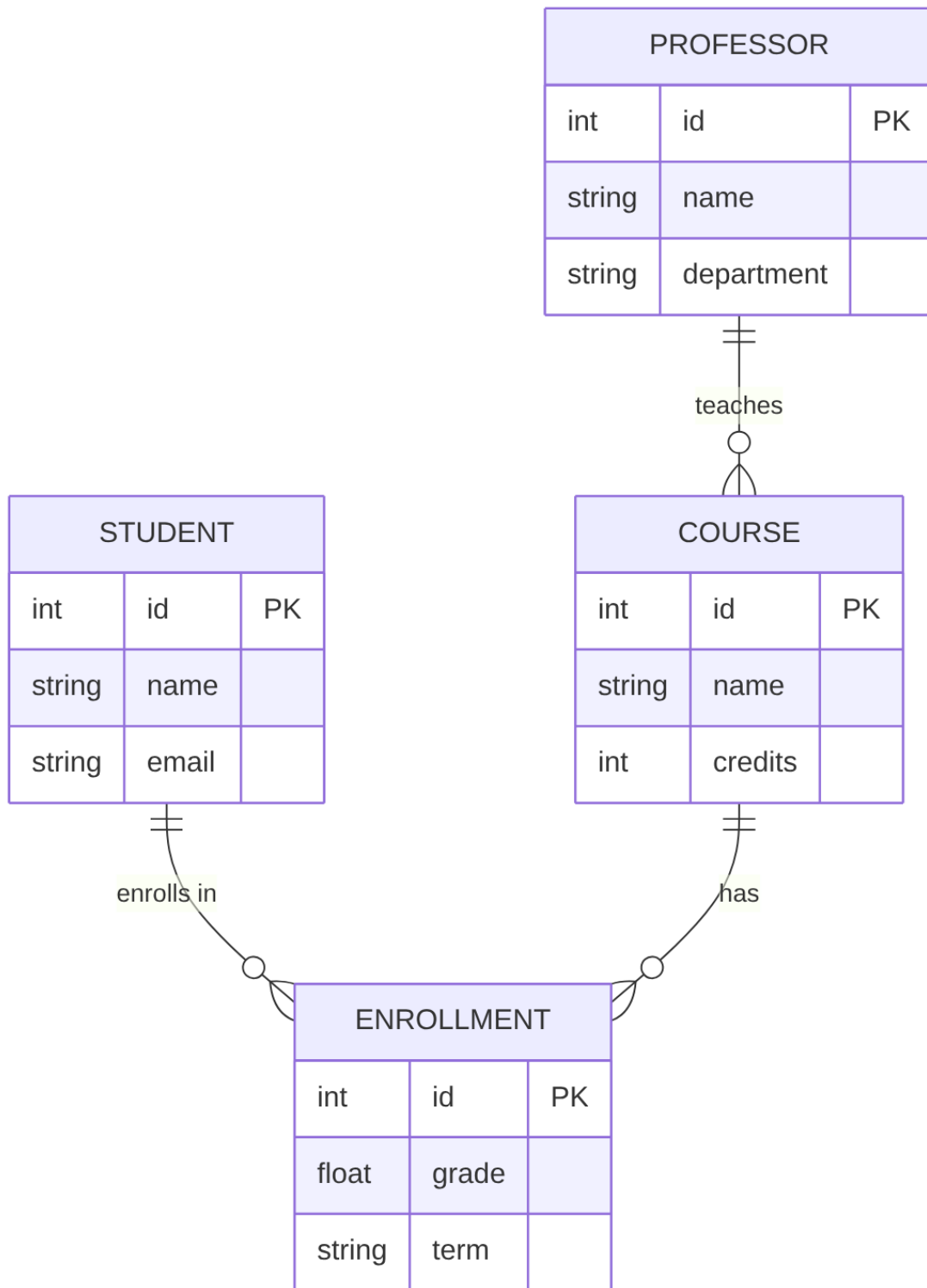


Figure9.1: Diagram: Example: University database.

(continued from previous page)

```

    <<abstract>>
    +String color
    +area() float
    +perimeter() float
}

class Circle {
    +float radius
    +area() float
    +perimeter() float
}

class Rectangle {
    +float width
    +float height
    +area() float
    +perimeter() float
}

class Triangle {
    +float base
    +float height
    +area() float
    +perimeter() float
}

Shape <|-- Circle
Shape <|-- Rectangle
Shape <|-- Triangle
...

```

Result:

As shown in [Fig. 9.2](#), the diagram is versioned as a static image.

- **Methods:** Listed in the second section, with return type.
- **Abstract class:** Indicated with <<abstract>>.

9.2.2 Relationship types

The following table summarizes the main elements of this section.

Table. Relationship types.

Notation	Meaning
< --	Inheritance (is a)
*--	Composition
o--	Aggregation
-->	Association
..>	Dependency

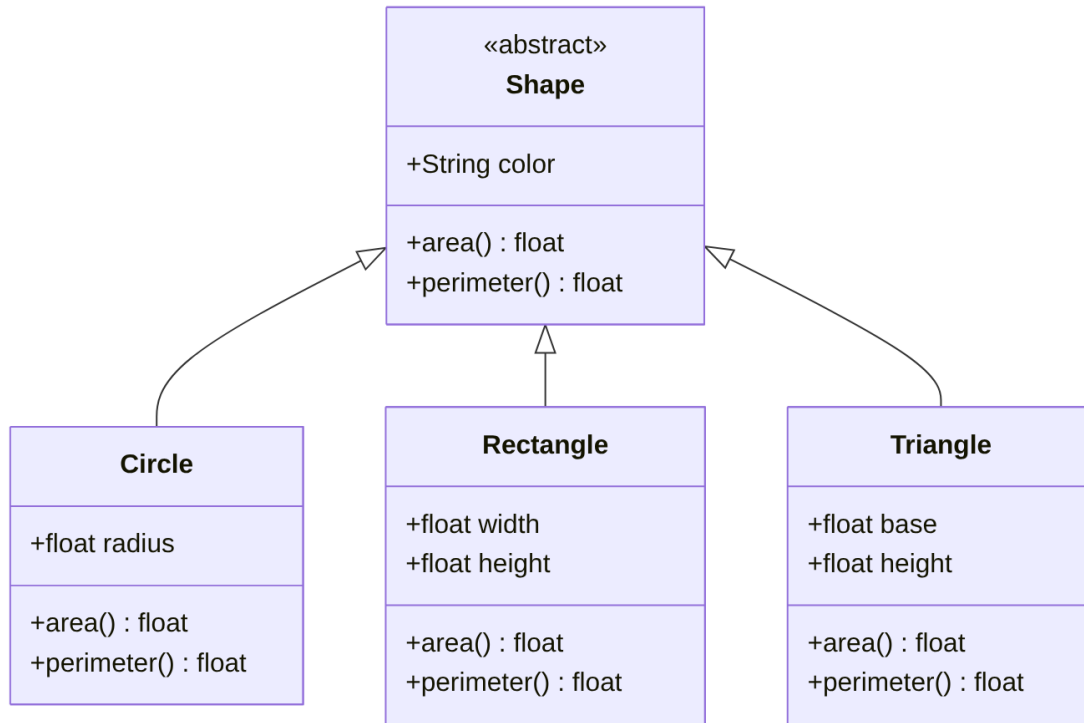


Figure9.2: Diagram: Example: Geometric Shape Hierarchy.

9.3 Diagrams with Kroki for Computer Science

Kroki lets you generate many useful diagram types in Computer Science without depending on browser-side JavaScript. All the following examples work in **HTML and PDF**.

9.3.1 1. Mermaid: layered architecture

As shown in Fig. 9.3, the diagram is versioned as a static image.

As shown in Fig. 9.4, the diagram is versioned as a static image.

As shown in Fig. 9.5, the diagram is versioned as a static image.

As shown in Fig. 9.6, the diagram is versioned as a static image.

```
## 5. Wavedrom: digital protocol timing
```

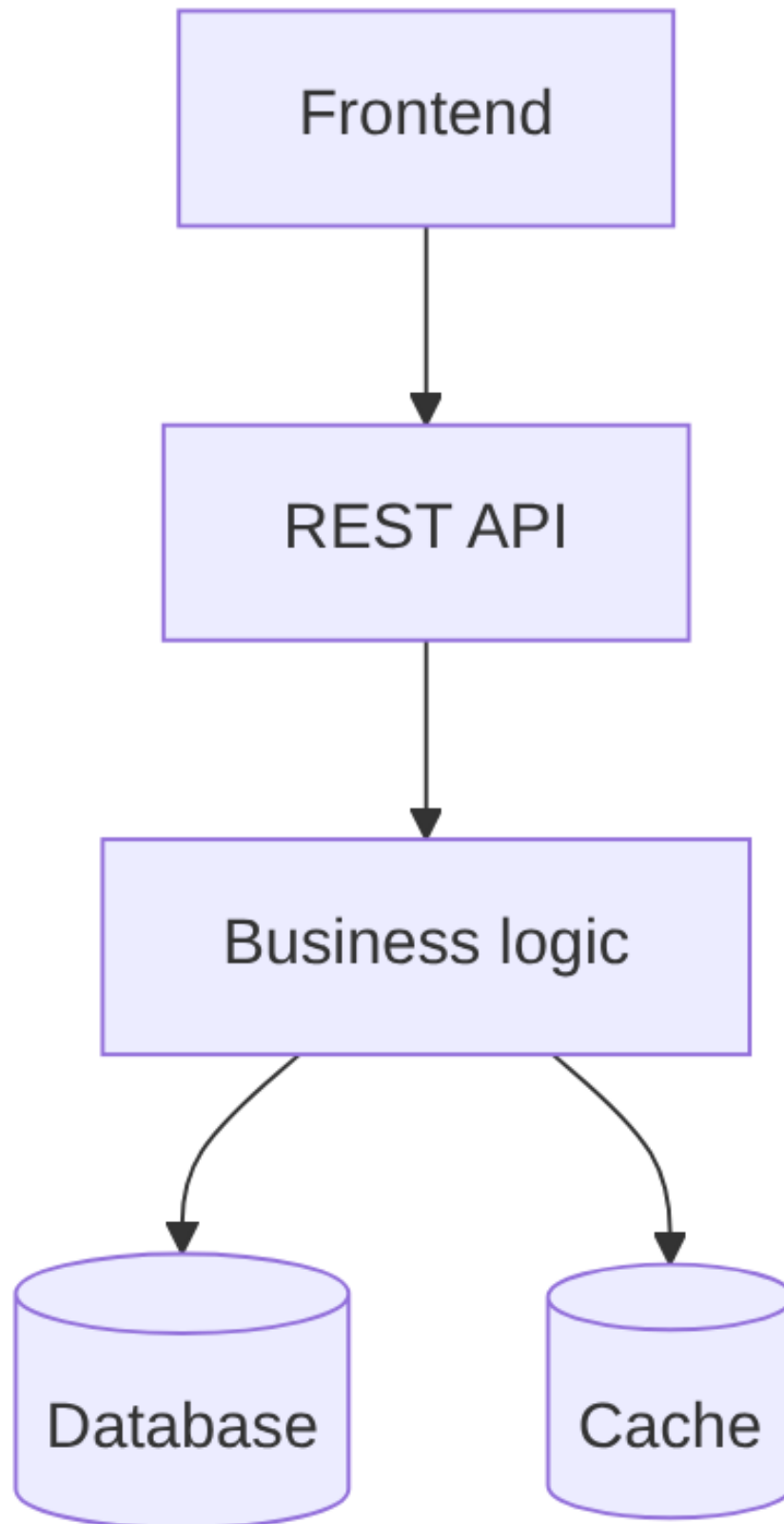
As shown in {numref}`fig-diagram-02-degrees-computer-science-degree-kroki-diagrams-05`, the diagram is versioned as a static image.

```

```{figure} ../../../_static/generated/diagrams/en/02_degrees_computer_science_degree_
 kroki_diagrams_05.svg
:name: fig-diagram-02-degrees-computer-science-degree-kroki-diagrams-05
:alt: Diagram: Wavedrom: digital protocol timing
:width: 90%
:align: center

```

(continues on next page)



**Figure9.3:** Diagram: Mermaid: layered architecture.



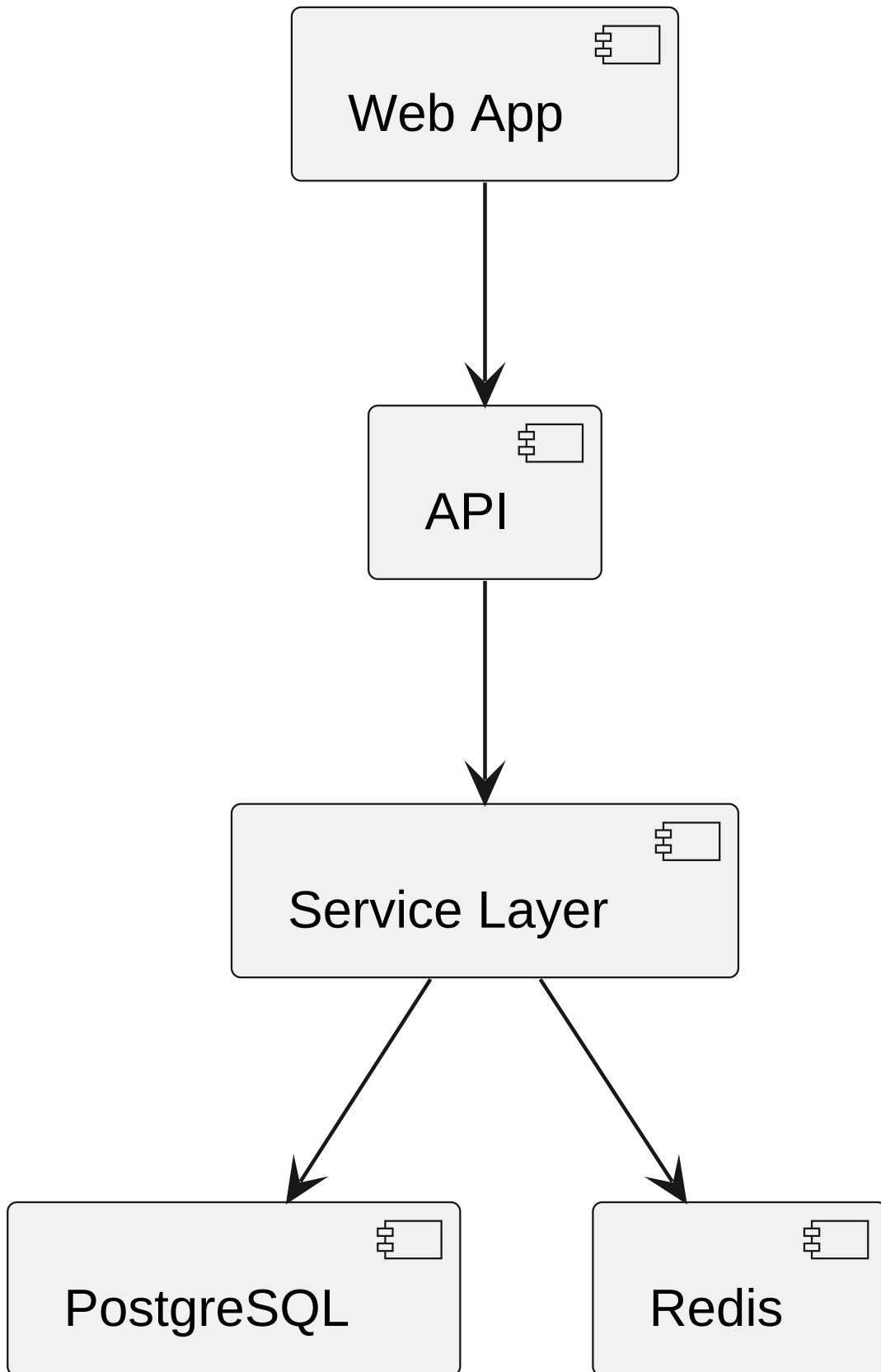


Figure9.4: Diagram: PlantUML: system components.

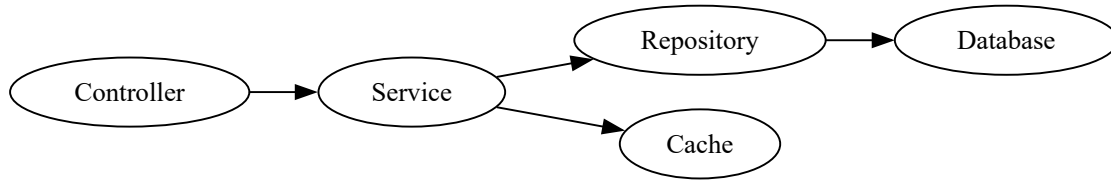


Figure 9.5: Diagram: GraphViz: dependency graph.

#### Container View: Teaching Platform

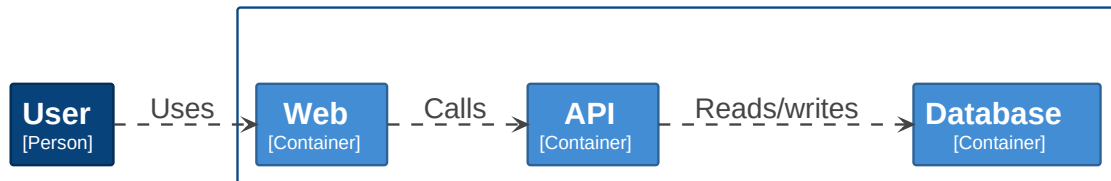


Figure 9.6: Diagram: Structurizr: simplified C4 view.

(continued from previous page)

Diagram: Wavedrom: digital protocol timing.

- architecture
- UML
- dependency graphs
- protocols
- C4 diagrams

If the user asks for a fast and portable diagram, Kroki is a great option.

## 9.4 Algorithmic Complexity: Visual Comparison

How does execution time grow as we increase data size? Let's compare three classic complexities:

- $O(n)$  — linear
- $O(n \log n)$  — linearithmic
- $O(n^2)$  — quadratic

```

import numpy as np
import matplotlib.pyplot as plt
import time

def measure_time(func, data):
 start = time.perf_counter()
 func(data)
 return time.perf_counter() - start

def linear_search(lst):
 target = lst[-1]
 for x in lst:

```

(continues on next page)

(continued from previous page)

```

 if x == target:
 return x

def sort_data(lst):
 return sorted(lst)

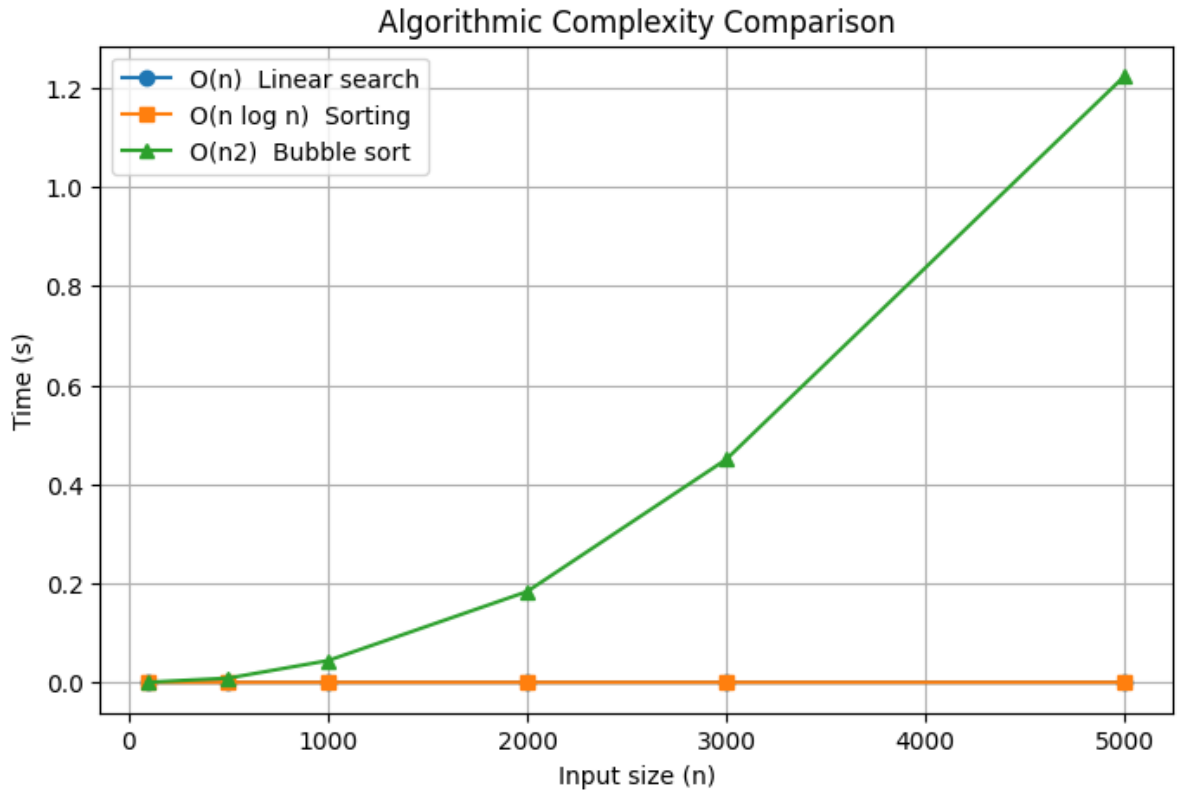
def bubble_sort(lst):
 n = len(lst)
 for i in range(n):
 for j in range(0, n - i - 1):
 if lst[j] > lst[j + 1]:
 lst[j], lst[j + 1] = lst[j + 1], lst[j]

sizes = [100, 500, 1000, 2000, 3000, 5000]
t_on, t_onlogn, t_on2 = [], [], []

for n in sizes:
 data = list(range(n, 0, -1))
 t_on.append(measure_time(linear_search, data[:]))
 t_onlogn.append(measure_time(sort_data, data[:]))
 t_on2.append(measure_time(bubble_sort, data[:]))

plt.figure(figsize=(8, 5))
plt.plot(sizes, t_on, 'o-', label='O(n) Linear search')
plt.plot(sizes, t_onlogn, 's-', label='O(n log n) Sorting')
plt.plot(sizes, t_on2, '^-', label='O(n^2) Bubble sort')
plt.xlabel('Input size (n)')
plt.ylabel('Time (s)')
plt.title('Algorithmic Complexity Comparison')
plt.legend()
plt.grid(True)
plt.show()

```



### 9.4.1 Expected result

The plot clearly shows how  $O(n^2)$  grows much faster than  $O(n)$  and  $O(n \log n)$ . This explains why choosing the right algorithm is crucial when working with large datasets.

### 9.4.2 Conclusion

- $O(n)$ : Time grows proportionally to data size.
- $O(n \log n)$ : Moderate growth, typical of efficient sorting algorithms.
- $O(n^2)$ : Explosive growth, unfeasible for large data.

Choosing a good algorithm can mean the difference between seconds and hours of computation.

## DEGREE IN GEOLOGY

In this section, you will find examples adapted to Geology Degree courses. The goal is to show how to integrate:

- Topographic profiles from elevation data
- Stratigraphic columns with executable code
- Rock cycle diagrams

### Applicable subjects

This block can be adapted to cartography, stratigraphy, general geology, geomorphology, and petrology courses.

## 10.1 Topographic Profile

A topographic profile represents elevation variation along a section line. We will generate a synthetic profile and plot it with matplotlib.

```
import numpy as np
import matplotlib.pyplot as plt

distance = np.linspace(0, 10, 500)

elevation = (
 200
 + 80 * np.sin(0.8 * distance)
 + 40 * np.sin(2.5 * distance)
 + 20 * np.random.normal(0, 1, len(distance))
)

plt.figure(figsize=(10, 4))
plt.fill_between(distance, 0, elevation, color='#8B7355', alpha=0.4)
plt.plot(distance, elevation, color='#5C4033', linewidth=1.5)
plt.xlabel('Distance (km)')
plt.ylabel('Elevation (m)')
plt.title('Synthetic topographic profile')
plt.axhline(y=0, color='skyblue', linewidth=0.5)
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

### 10.1.1 Expected result

A profile simulating terrain with elevation variations is produced. The brown fill represents the ground mass and the blue line marks sea level.

### 10.1.2 Conclusion

- Topographic profiles can be created from real data (CSV, GeoTIFF).
- This example uses synthetic data to avoid external file dependencies.
- For real data, simply replace the `distance` and `elevation` arrays.

## 10.2 Stratigraphic Column

A stratigraphic column represents the vertical succession of geological layers. Each layer is drawn as a colored rectangle according to its lithology.

```
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches

layers = [
 {"name": "Limestone", "thickness": 15, "color": "#4A90D9", "age": "Jurassic"},
 {"name": "Sandstone", "thickness": 20, "color": "#E8C84A", "age": "Triassic"},
 {"name": "Clay", "thickness": 10, "color": "#C97B3D", "age": "Triassic"},
 {"name": "Conglomerate", "thickness": 12, "color": "#9B7653", "age": "Permian"},
 {"name": "Shale", "thickness": 18, "color": "#6B6B6B", "age": "Carboniferous"},
 {"name": "Granite", "thickness": 25, "color": "#D44A4A", "age": "Precambrian"}
]

fig, ax = plt.subplots(figsize=(5, 8))

y_base = 0
for layer in layers:
 ax.add_patch(plt.Rectangle((0, y_base), 1, layer["thickness"],
 facecolor=layer["color"], edgecolor="black",
 linewidth=0.8))
 ax.text(0.5, y_base + layer["thickness"] / 2,
 f"{layer['name']}\n({layer['age']}, {layer['thickness']} m)",
 ha='center', va='center', fontsize=8, fontweight='bold')
 y_base += layer["thickness"]

ax.set_xlim(-0.2, 1.4)
ax.set_ylim(-2, y_base + 2)
ax.set_ylabel('Cumulative thickness (m)')
ax.set_title('Synthetic stratigraphic column')
ax.set_xticks([])
ax.invert_yaxis()
plt.tight_layout()
plt.show()
```

### 10.2.1 Expected result

The plot shows stacked layers from bottom to top (oldest to youngest). Each color represents a different lithology: blue for limestone, yellow for sandstone, etc.

### 10.2.2 Conclusion

- You can add more layers by appending entries to the `layers` list.
- Colors are customizable by changing the hex values.
- For real data, replace the list with data read from a CSV file.

## 10.3 The Rock Cycle with Kroki (Mermaid)

The rock cycle describes the transformations between the three main rock types. With Kroki and `:type: mermaid`, we can represent this cycle as a flowchart that works in both HTML and PDF.

### 10.3.1 The cycle

**Code:**

```
```{kroki}
:type: mermaid
:align: center

flowchart LR
    Magma["Magma"]
    Igneous["Igneous Rock"]
    Sedimentary["Sedimentary Rock"]
    Metamorphic["Metamorphic Rock"]

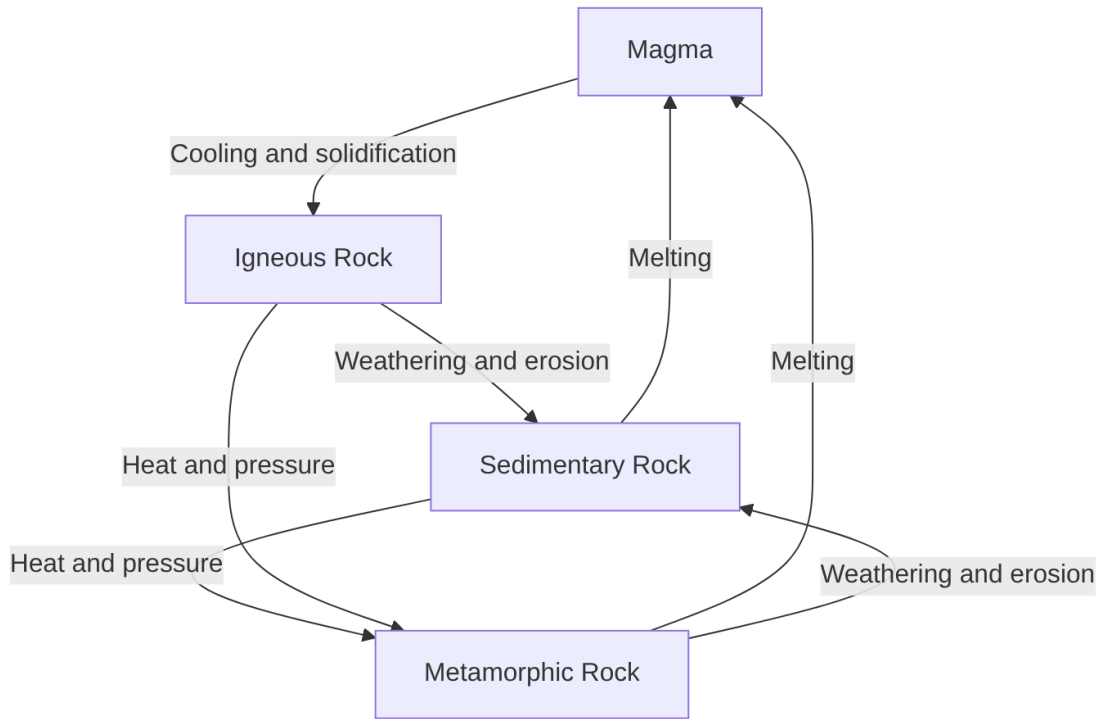
    Magma -->|"Cooling and solidification"| Igneous
    Igneous -->|"Weathering and erosion"| Sedimentary
    Sedimentary -->|"Heat and pressure"| Metamorphic
    Metamorphic -->|"Melting"| Magma
    Igneous -->|"Heat and pressure"| Metamorphic
    Metamorphic -->|"Weathering and erosion"| Sedimentary
    Sedimentary -->|"Melting"| Magma
```
```

**Result:**

As shown in [Fig. 10.1](#), the diagram is versioned as a static image.

The following table summarizes the main elements of this section.

**Table. Transition explanation.**



**Figure10.1:** Diagram: The cycle.

| Transition                | Geological process                                       |
|---------------------------|----------------------------------------------------------|
| Magma → Igneous           | Cooling and solidification at the surface or underground |
| Igneous → Sedimentary     | Weathering, erosion, transport, and sedimentation        |
| Sedimentary → Metamorphic | Heat and pressure (metamorphism) without melting         |
| Metamorphic → Magma       | Complete melting at extreme temperatures                 |
| Igneous → Metamorphic     | Direct metamorphism through heat and pressure            |
| Metamorphic → Sedimentary | Uplift, weathering, and erosion                          |

### 10.3.2 Key point

Any rock type can transform into any other. It is not a linear cycle, but a **transformation network** operating on geological timescales (millions of years).



## CHEMICAL ENGINEERING DEGREE

This section contains examples for Chemical Engineering courses. The cases combine material balances, separation operations, mass transfer, reaction engineering, and technical chart reading.

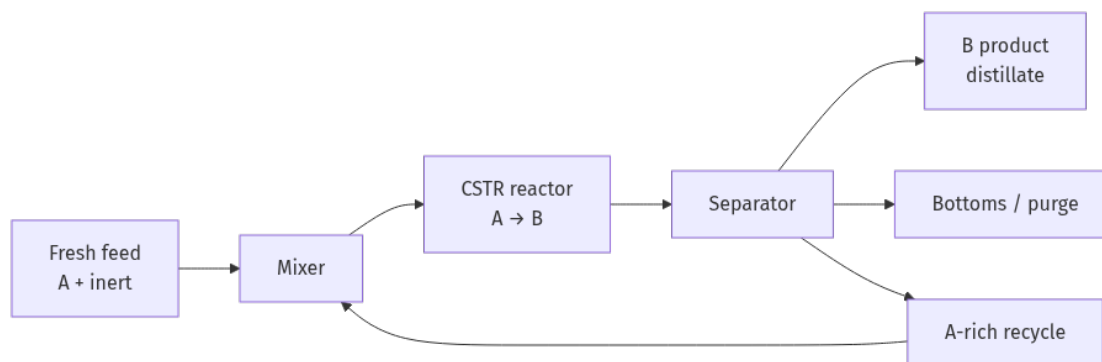
The teaching goal is for students to:

- Turn a written process description into a clear flow diagram.
- Explore how separation designs change when parameters vary.
- Solve recycle streams with simple numerical methods.
- Connect transport, kinetics, and concentration profiles.
- Read professional engineering charts, such as the Moody chart, with computational support.

### 11.1 Process Flow Diagram

A flow diagram helps turn a verbal process description into a structure that can be reviewed, discussed, and improved. For teaching notes, a simplified first draft is often enough: feeds, main unit operations, products, and possible recycle streams.

As shown in Fig. 11.1, the process can be represented with a feed, a mixer, a reactor, a separator, and a recycle stream. The figure is not meant to replace an industrial PFD; it is a quick teaching draft.



**Figure 11.1:** Simplified process flow diagram with recycle.

### 11.1.1 Classroom Use

Students can be asked to identify:

- The inlet and outlet streams.
- The unit where the reaction occurs.
- The stream that is recycled and why it may improve reactant use.
- Which variables would be needed for a full material balance.

#### **i** Short activity

Redraw the diagram to include a purge stream. Explain which operating problem that purge could solve if an inert component accumulates.

## 11.2 McCabe-Thiele Diagram

A McCabe-Thiele diagram estimates the number of stages required for a binary distillation. This example uses constant relative volatility and compares several reflux ratios.

The notebook figure shows the equilibrium curve, the diagonal, operating lines, and the staircase stages. By editing  $R$ ,  $x_F$ ,  $x_D$ ,  $x_B$ , or  $\alpha$ , students can observe how the design changes.

```
import numpy as np
import matplotlib.pyplot as plt

Editable parameters
alpha = 2.45 # relative volatility
xF = 0.45 # feed composition
xD = 0.92 # distillate composition
xB = 0.08 # bottoms composition
R = 2.2 # reflux ratio
max_stages = 60

def y_equilibrium(x, alpha):
 return alpha * x / (1 + (alpha - 1) * x)

def x_equilibrium_from_y(y, alpha):
 return y / (alpha - (alpha - 1) * y)

def y_rectifying(x, R, xD):
 return (R / (R + 1)) * x + xD / (R + 1)

Saturated-liquid feed: q-line is vertical at xF. The stripping line is drawn
through the feed intersection and the bottoms point on the diagonal.
y_feed = y_rectifying(xF, R, xD)
m_strip = (y_feed - xB) / (xF - xB)
b_strip = xB - m_strip * xB

def y_operating(x):
```

(continues on next page)

(continued from previous page)

```

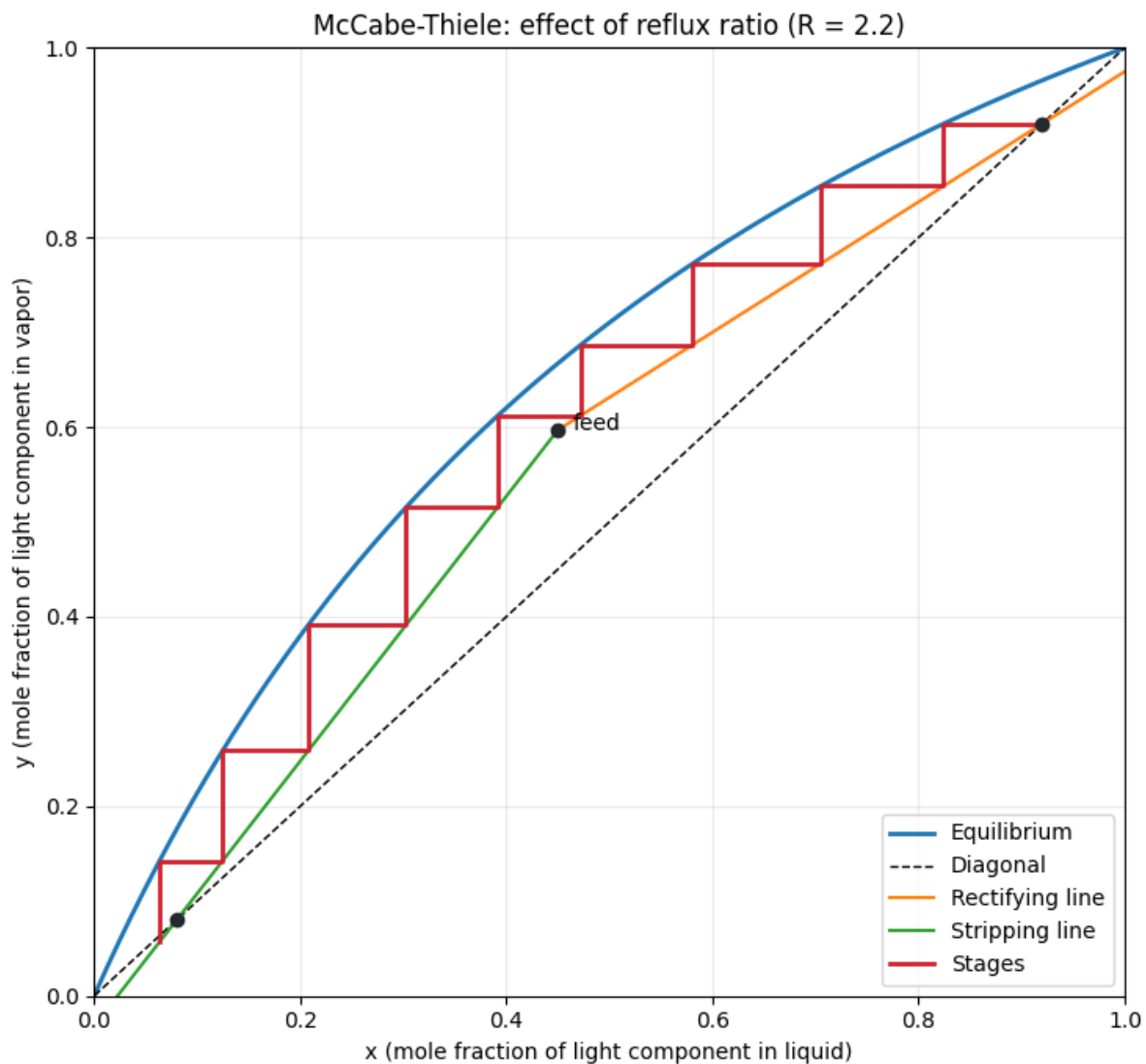
 return y_rectifying(x, R, xD) if x >= xF else m_strip * x + b_strip

Staircase construction.
stair_x = [xD]
stair_y = [xD]
stages = 0
x_current = xD
y_current = xD
while x_current > xB and stages < max_stages:
 x_eq = x_equilibrium_from_y(y_current, alpha)
 stair_x.extend([x_eq, x_eq])
 stair_y.extend([y_current, y_operating(x_eq)])
 x_current = x_eq
 y_current = y_operating(x_eq)
 stages += 1

x = np.linspace(0, 1, 500)
plt.figure(figsize=(7.5, 7))
plt.plot(x, y_equilibrium(x, alpha), label="Equilibrium", linewidth=2)
plt.plot(x, x, "k--", label="Diagonal", linewidth=1)
plt.plot(x[x >= xF], y_rectifying(x[x >= xF], R, xD), label="Rectifying line")
plt.plot(x[x <= xF], m_strip * x[x <= xF] + b_strip, label="Stripping line")
plt.plot(stair_x, stair_y, color="#cf222e", linewidth=2, label="Stages")
plt.scatter([xF, xD, xB], [y_feed, xD, xB], color="#24292f", zorder=5)
plt.text(xF + 0.015, y_feed, "feed")
plt.xlim(0, 1)
plt.ylim(0, 1)
plt.xlabel("x (mole fraction of light component in liquid)")
plt.ylabel("y (mole fraction of light component in vapor)")
plt.title("McCabe-Thiele: effect of reflux ratio (R = " + str(R) + ")")
plt.grid(alpha=0.25)
plt.legend(loc="lower right")
plt.tight_layout()
plt.show()

print(f"Approximate theoretical stages: {stages}")
print(f"xD={xD:.2f}, xF={xF:.2f}, xB={xB:.2f}, alpha={alpha:.2f}, R={R:.2f}")

```



Approximate theoretical stages: 9  
 $x_D=0.92$ ,  $x_F=0.45$ ,  $x_B=0.08$ ,  $\alpha=2.45$ ,  $R=2.20$

### 11.2.1 Reading the Result

As the reflux ratio increases, the rectifying line moves closer to the diagonal and fewer stages are required. The energy cost increases because more liquid is recycled inside the column.

## 11.3 Recycle Solved with fsolve

Many chemical processes include recycle streams. This means that a variable depends on itself: the stream leaving the separator returns to the mixer and changes the reactor inlet.

This example uses a simple reactor model with conversion depending on residence time. The equations are solved with `scipy.optimize.fsolve`.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import fsolve

Editable parameters
F_A0 = 100.0 # fresh molar flow of A (mol/min)
Q0 = 120.0 # fresh volumetric flow (L/min)
C_ref = 1.0 # reference concentration in recycle stream (mol/L)
V = 450.0 # reactor volume (L)
k = 0.32 # first-order kinetic constant (1/min)
recycle_fraction = 0.65

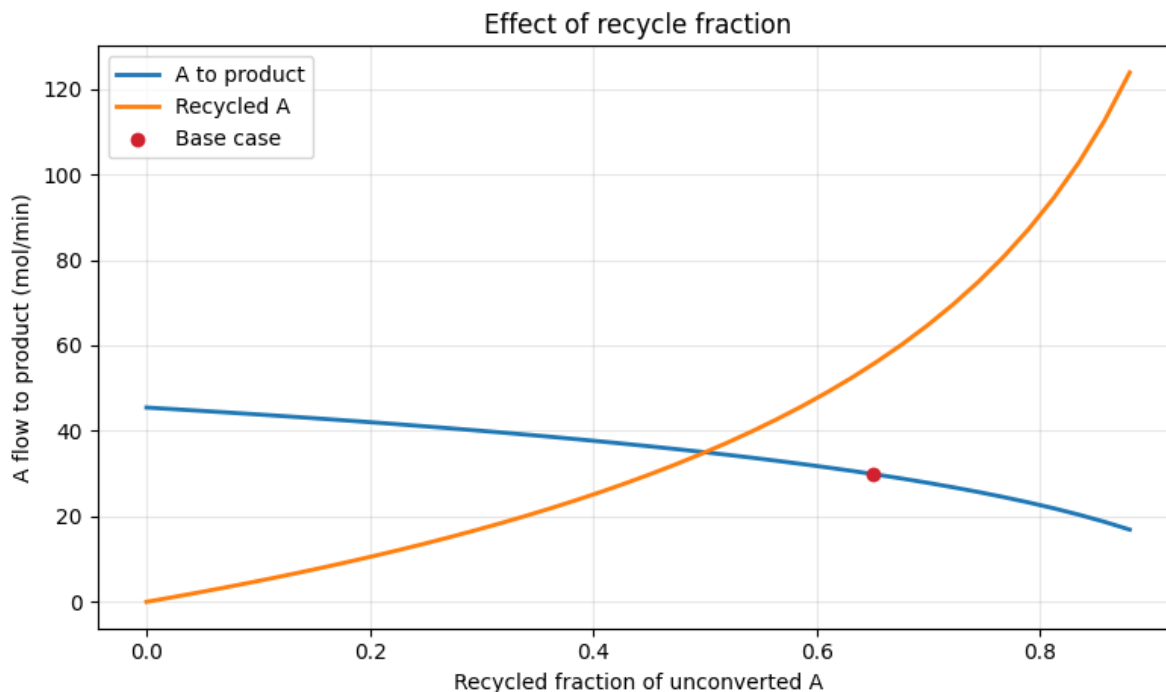
def solve_recycle(recycle_fraction):
 def equations(unknowns):
 F_A_out, F_A_rec = unknowns
 Q_rec = F_A_rec / C_ref
 tau = V / (Q0 + Q_rec)
 X = k * tau / (1 + k * tau)
 eq_reactor = F_A_out - (F_A0 + F_A_rec) * (1 - X)
 eq_separator = F_A_rec - recycle_fraction * F_A_out
 return [eq_reactor, eq_separator]
 F_A_out, F_A_rec = fsolve(equations, [35.0, 20.0])
 F_A_product = F_A_out - F_A_rec
 global_conversion = 1 - F_A_product / F_A0
 return F_A_out, F_A_rec, F_A_product, global_conversion

base = solve_recycle(recycle_fraction)
print("Base-case solution")
print(f"F_A,out reactor = {base[0]:.2f} mol/min")
print(f"F_A,recycle = {base[1]:.2f} mol/min")
print(f"F_A to product = {base[2]:.2f} mol/min")
print(f"Global conversion of fresh A = {100*base[3]:.1f} %")

fractions = np.linspace(0, 0.88, 40)
results = np.array([solve_recycle(fr) for fr in fractions])

plt.figure(figsize=(8, 4.8))
plt.plot(fractions, results[:, 2], label="A to product", linewidth=2)
plt.plot(fractions, results[:, 1], label="Recycled A", linewidth=2)
plt.scatter([recycle_fraction], [base[2]], color="#cf222e", zorder=5, label="Base case")
plt.xlabel("Recycled fraction of unconverted A")
plt.ylabel("A flow to product (mol/min)")
plt.title("Effect of recycle fraction")
plt.grid(alpha=0.3)
plt.legend()
plt.tight_layout()
plt.show()
```

```
Base-case solution
F_A,out reactor = 85.45 mol/min
F_A,recycle = 55.54 mol/min
F_A to product = 29.91 mol/min
Global conversion of fresh A = 70.1 %
```



### 11.3.1 Interpretation

Recycle increases overall reactant use, but it also increases the internal flow rate. In a real design, conversion, equipment size, energy cost, and inert accumulation must be balanced.

## 11.4 Concentration Gradient in a Catalyst Particle

When a reaction occurs inside a porous particle, the reactant concentration may be lower in the interior than at the surface. The profile depends on kinetics, internal diffusion, and external mass transfer from the fluid.

The example uses a first-order spherical model. The plot compares several scenarios to show which resistance dominates.

```
import numpy as np
import matplotlib.pyplot as plt

Editable parameters
R_particle = 1.5e-3 # m
C_bulk = 1.0 # mol/L, used as reference
scenarios = [('Fast transfer', 0.03, 3.5e-06, 0.25), ('Low internal diffusion', 0.03, 7e-07, 0.25), ('Slow external film', 0.006, 3.5e-06, 0.25), ('Fast kinetics', 0.03, 3.5e-06, 0.9)]
```

(continues on next page)

(continued from previous page)

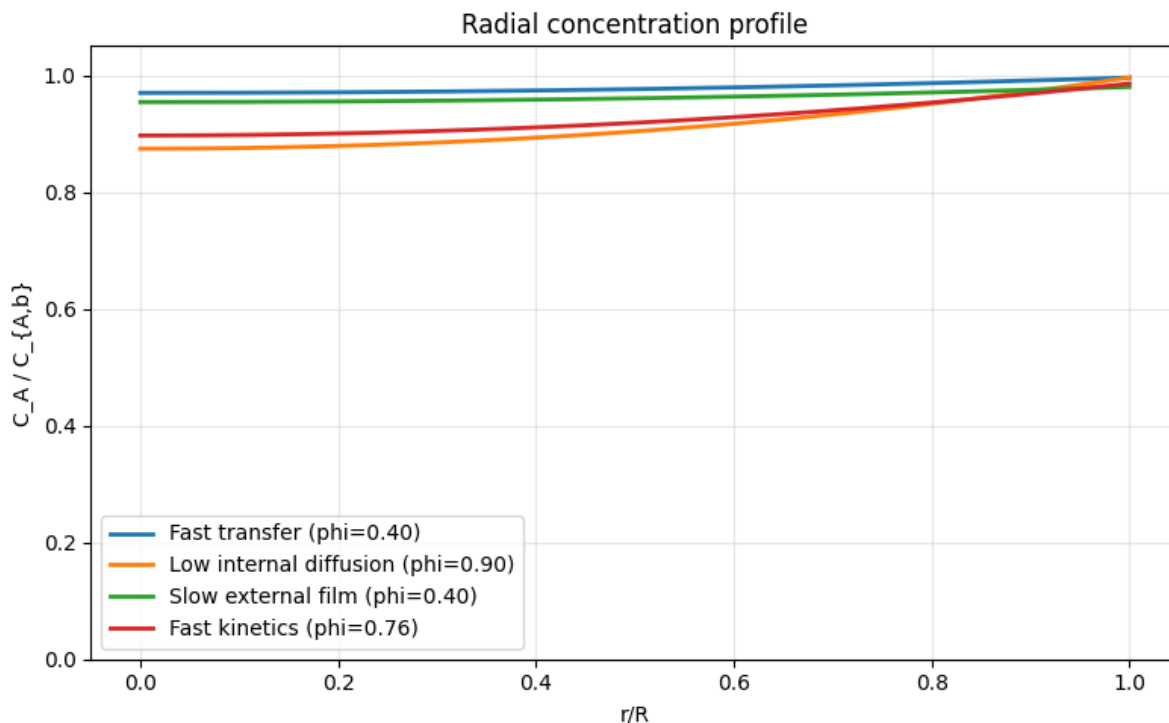
```
def effectiveness_factor(phi):
 if phi < 1e-6:
 return 1.0
 return 3 / phi**2 * (phi / np.tanh(phi) - 1)

def profile(r_over_R, kf, De, k):
 phi = R_particle * np.sqrt(k / De)
 eta = effectiveness_factor(phi)
 C_surface = C_bulk / (1 + eta * k * R_particle / (3 * kf))
 r = np.maximum(r_over_R, 1e-6)
 internal = np.sinh(phi * r) / (r * np.sinh(phi)) if phi > 1e-6 else np.ones_
 like(r)
 internal[0] = phi / np.sinh(phi) if phi > 1e-6 else 1.0
 return C_surface * internal / C_bulk, phi, eta, C_surface / C_bulk

r = np.linspace(0, 1, 250)
plt.figure(figsize=(8, 5))
print("Scenario summary")
for name, kf, De, k in scenarios:
 C, phi, eta, Cs = profile(r, kf, De, k)
 plt.plot(r, C, linewidth=2, label=f"{name} (phi={phi:.2f})")
 print(f"{name}: phi={phi:.2f}, eta={eta:.2f}, C_surface/C_bulk={Cs:.2f}")

plt.xlabel("r/R")
plt.ylabel("C_A / C_{A,b}")
plt.title("Radial concentration profile")
plt.ylim(0, 1.05)
plt.grid(alpha=0.3)
plt.legend()
plt.tight_layout()
plt.show()
```

```
Scenario summary
Fast transfer: phi=0.40, eta=0.99, C_surface/C_bulk=1.00
Low internal diffusion: phi=0.90, eta=0.95, C_surface/C_bulk=1.00
Slow external film: phi=0.40, eta=0.99, C_surface/C_bulk=0.98
Fast kinetics: phi=0.76, eta=0.96, C_surface/C_bulk=0.99
```



### 11.4.1 Interpretation

If internal diffusion is slow or kinetics are fast, a strong gradient appears inside the particle. If external transfer is slow, even the surface concentration drops below the bulk-fluid concentration.

## 11.5 Fast Reading of the Moody Chart

The Moody chart links Reynolds number, relative roughness, and the Darcy friction factor. Instead of reading a curve manually, a correlation can place points on the chart and support discussion of the flow regime.

```
import numpy as np
import matplotlib.pyplot as plt

def friction_factor(Re, eps_rel):
 Re = np.asarray(Re, dtype=float)
 f = np.empty_like(Re)
 laminar = Re < 2300
 f[laminar] = 64 / Re[laminar]
 turbulent = ~laminar
 # Haaland equation for turbulent flow.
 f[turbulent] = (-1.8 * np.log10((eps_rel / 3.7)**1.11 + 6.9 / Re[turbulent]))**-2
 return f

Re = np.logspace(3, 8, 500)
roughness_values = [0.0, 1e-5, 1e-4, 1e-3, 5e-3]
points = [
 (4.0e3, 1e-4, "A"),
```

(continues on next page)



(continued from previous page)

```

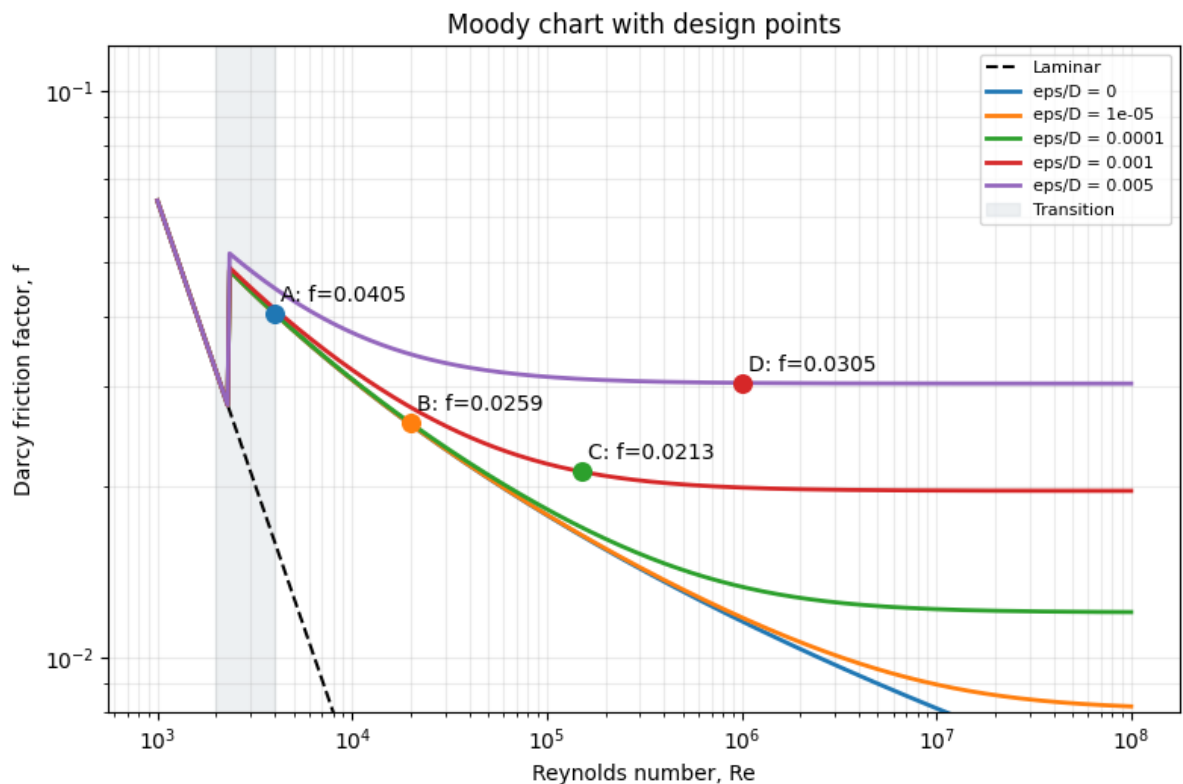
(2.0e4, 1e-4, "B"),
(1.5e5, 1e-3, "C"),
(1.0e6, 5e-3, "D"),
]

plt.figure(figsize=(8, 5.4))
plt.loglog(Re, 64/Re, "k--", linewidth=1.5, label="Laminar")
for eps in roughness_values:
 plt.loglog(Re, friction_factor(Re, eps), linewidth=2, label=f"eps/D = {eps:g}")

for Re_p, eps_p, name in points:
 f_p = friction_factor(np.array([Re_p]), eps_p)[0]
 plt.scatter(Re_p, f_p, s=70, zorder=5)
 plt.text(Re_p * 1.08, f_p * 1.05, f"{name}: f={f_p:.4f}")

plt.axvspan(2000, 4000, color="#d8dee4", alpha=0.45, label="Transition")
plt.xlabel("Reynolds number, Re")
plt.ylabel("Darcy friction factor, f")
plt.title("Moody chart with design points")
plt.grid(True, which="both", alpha=0.25)
plt.ylim(0.008, 0.12)
plt.legend(fontsize=8)
plt.tight_layout()
plt.show()

```



### 11.5.1 Interpretation

In laminar flow the friction factor depends only on Reynolds number. In turbulent flow, relative roughness shifts the curves and can dominate the behavior at high Reynolds numbers.

## CHEMISTRY DEGREE

This section contains examples for Chemistry courses. The cases combine acid-base equilibria, enzyme kinetics, periodic properties, infrared spectroscopy, and electrochemistry.

The teaching goal is for students to:

- Interpret pH curves in buffer systems.
- Compare kinetic models and the effect of experimental parameters.
- Consult chemical properties visually.
- Recognize characteristic bands in IR spectra.
- Apply the Nernst equation to redox pairs under non-standard conditions.

## 12.1 pH Curve in a Buffer Solution

A buffer solution resists pH changes because it contains a weak acid and its conjugate base. This example uses the acetic acid/acetate pair and calculates pH after adding strong acid or strong base.

```
import numpy as np
import matplotlib.pyplot as plt

Editable parameters
pKa = 4.76
n_HA0 = 25.0 # mmol acetic acid
n_A0 = 25.0 # mmol acetate
additions = np.linspace(-22, 22, 220) # negative: strong acid; positive: strong base

pH = []
for added in additions:
 if added >= 0:
 n_HA = n_HA0 - added
 n_A = n_A0 + added
 else:
 n_HA = n_HA0 - added
 n_A = n_A0 + added
 n_HA = max(n_HA, 1e-6)
 n_A = max(n_A, 1e-6)
 pH.append(pKa + np.log10(n_A / n_HA))
pH = np.array(pH)

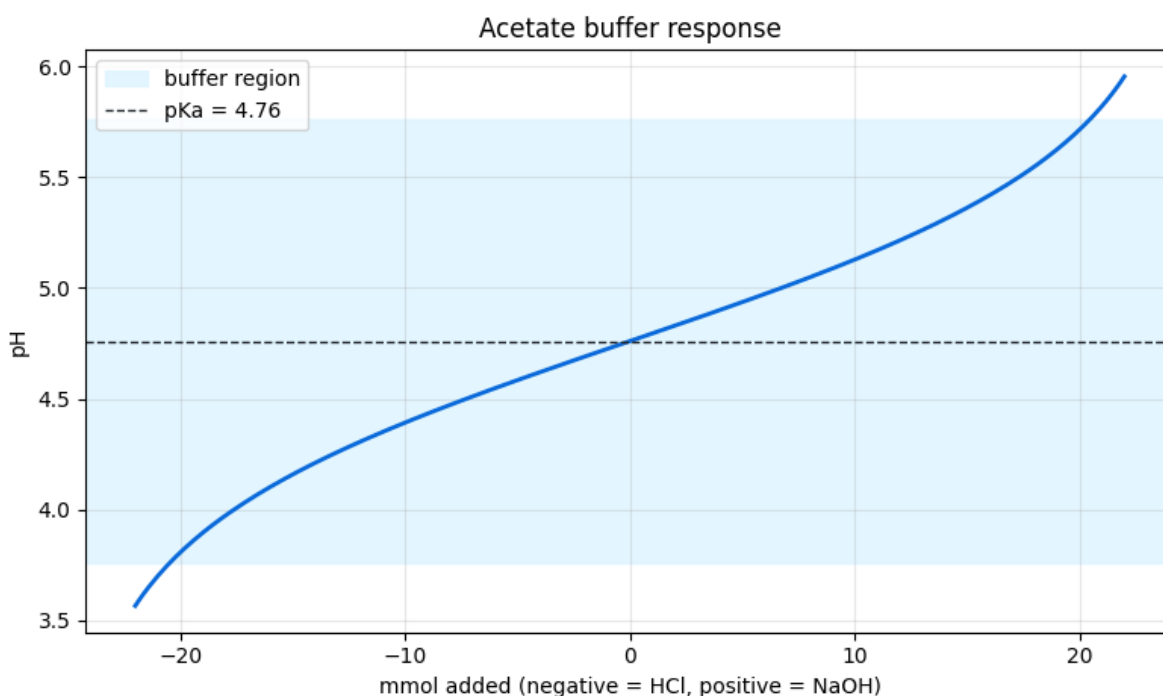
plt.figure(figsize=(8, 4.8))
```

(continues on next page)

(continued from previous page)

```
plt.plot(additions, pH, linewidth=2, color="#0969da")
plt.axhspan(pKa - 1, pKa + 1, color="#ddf4ff", alpha=0.8, label="buffer region")
plt.axhline(pKa, color="#24292f", linestyle="--", linewidth=1, label=f"pKa = {pKa}")
plt.xlabel("mmol added (negative = HCl, positive = NaOH)")
plt.ylabel("pH")
plt.title("Acetate buffer response")
plt.grid(alpha=0.3)
plt.legend()
plt.tight_layout()
plt.show()

print(f"Initial pH = {pKa + np.log10(n_A0/n_HA0) :.2f}")
print(f"pH after adding 10 mmol NaOH = {pH[np.argmin(np.abs(additions - 10))] :.2f}")
print(f"pH after adding 10 mmol HCl = {pH[np.argmin(np.abs(additions + 10))] :.2f}")
```



```
Initial pH = 4.76
pH after adding 10 mmol NaOH = 5.13
pH after adding 10 mmol HCl = 4.39
```

### 12.1.1 Interpretation

The most stable region appears when acid and conjugate base amounts are comparable. Near the ends, the buffer is exhausted and pH changes much more sharply.

## 12.2 Michaelis-Menten Kinetics

Michaelis-Menten kinetics relates the initial rate of an enzyme reaction to substrate concentration. This example compares changes in  $V_{\max}$ ,  $K_m$ , and the presence of a competitive inhibitor.

```
import numpy as np
import matplotlib.pyplot as plt

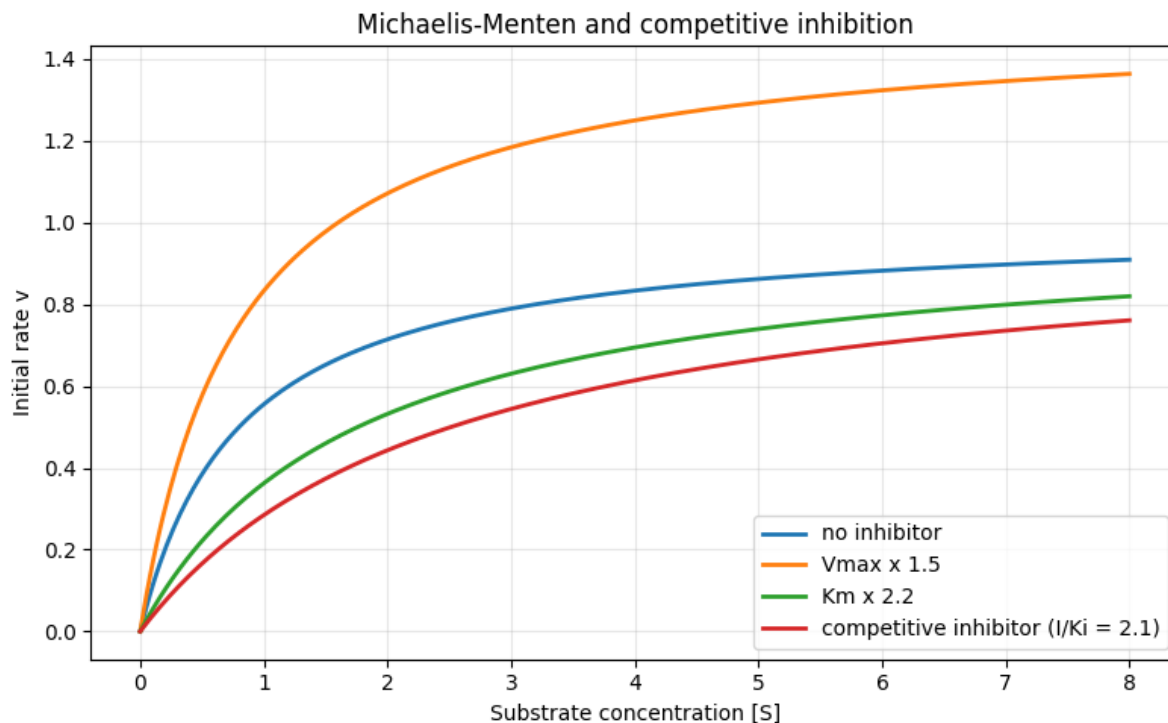
Editable parameters
Vmax = 1.0
Km = 0.8
I = 1.5
Ki = 0.7
S = np.linspace(0, 8, 300)

def michaelis(S, Vmax, Km):
 return Vmax * S / (Km + S)

v_base = michaelis(S, Vmax, Km)
v_high_vmax = michaelis(S, 1.5 * Vmax, Km)
v_high_km = michaelis(S, Vmax, 2.2 * Km)
Km_app = Km * (1 + I / Ki)
v_inhibited = michaelis(S, Vmax, Km_app)

plt.figure(figsize=(8, 5))
plt.plot(S, v_base, linewidth=2, label="no inhibitor")
plt.plot(S, v_high_vmax, linewidth=2, label="Vmax x 1.5")
plt.plot(S, v_high_km, linewidth=2, label="Km x 2.2")
plt.plot(S, v_inhibited, linewidth=2, label="competitive inhibitor (I/Ki = " + f"{I/Ki:.1f}")")
plt.xlabel("Substrate concentration [S]")
plt.ylabel("Initial rate v")
plt.title("Michaelis-Menten and competitive inhibition")
plt.grid(alpha=0.3)
plt.legend()
plt.tight_layout()
plt.show()

print(f"Base Vmax = {Vmax:.2f}, Km = {Km:.2f}")
print(f"Competitive inhibitor: apparent Km = {Km_app:.2f}")
```



Base  $V_{\max} = 1.00$ ,  $K_m = 0.80$   
 Competitive inhibitor: apparent  $K_m = 2.51$

### 12.2.1 Interpretation

Increasing  $V_{\max}$  raises the maximum rate. Increasing  $K_m$  shifts the curve to the right. Under competitive inhibition, the apparent effect is an increase in  $K_m$  without changing  $V_{\max}$ .

## 12.3 Interactive Periodic Table

The interactive table below summarizes a teaching selection of representative elements. It is not intended to be a complete database; it is a fast way to compare properties and chemical families during an explanation.

**Interactive table. Basic properties of selected elements.**

**Interactive periodic table:** see the HTML version to filter elements and compare properties. This page uses a teaching selection of representative elements.

The interactive table makes trends quick to compare: alkali metals have low ionization energies, halogens have high electronegativities, and noble gases usually have no defined electronegativity on this scale.

### 12.3.1 Activity

Select electronegativity and compare sodium, chlorine, and argon. Then explain why NaCl is interpreted very differently from a covalent molecule such as HCl.

## 12.4 Interactive IR Spectrum

An infrared spectrum links absorption bands with functional groups. The tool below is a teaching simulation: when a functional group is enabled, approximate bands are shown in the regions where they commonly appear.

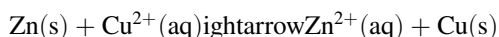
**Interactive figure. Approximate IR bands by functional group.**

**Interactive IR spectrum:** see the HTML version to enable functional groups. Typical bands: O--H 3200--3600  $\text{cm}^{-1}$ , C--H 2850--3000  $\text{cm}^{-1}$ , C=O 1650--1750  $\text{cm}^{-1}$ , C $\equiv$ N 2210--2260  $\text{cm}^{-1}$ .

The interactive figure helps separate two ideas: band position gives information about the bond, but width and intensity also depend on chemical environment and sample conditions.

## 12.5 Nernst Equation

The Nernst equation calculates the potential of an electrochemical cell under non-standard conditions. The Daniell cell is used as an example:



For this reaction,  $Q = [\text{Zn}^{2+}]/[\text{Cu}^{2+}]$  and  $n = 2$ .

```
import numpy as np
import matplotlib.pyplot as plt

Editable parameters
E0 = 1.10 # V, standard Daniell cell potential
n = 2
R = 8.314462618
F = 96485.33212
T_values = [278.15, 298.15, 318.15]
Q = np.logspace(-3, 3, 400)

def nernst(E0, T, n, Q):
 return E0 - (R * T) / (n * F) * np.log(Q)

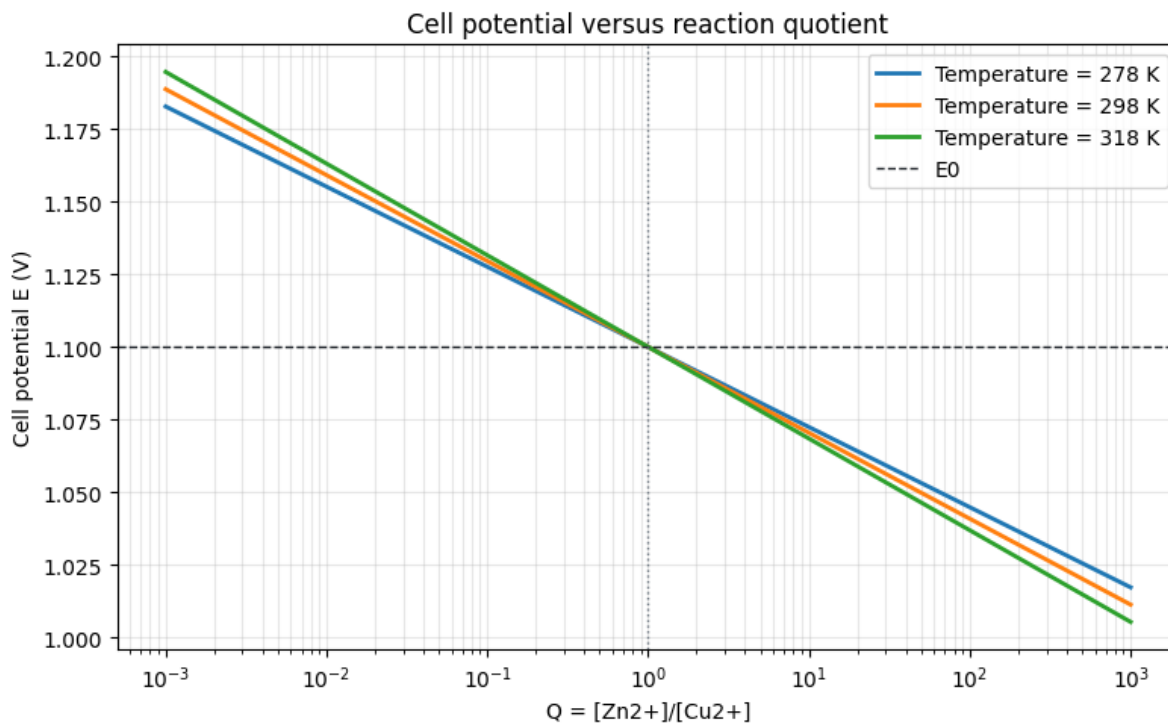
plt.figure(figsize=(8, 5))
for T in T_values:
 plt.semilogx(Q, nernst(E0, T, n, Q), linewidth=2, label=f"Temperature = {T:.0f} K")
plt.axhline(E0, color="#24292f", linestyle="--", linewidth=1, label="E0")
plt.axvline(1, color="#6e7781", linestyle=":", linewidth=1)
plt.xlabel("Q = [Zn2+]/[Cu2+]")
plt.ylabel("Cell potential E (V)")
plt.title("Cell potential versus reaction quotient")
plt.grid(alpha=0.3, which="both")
plt.legend()
plt.tight_layout()
```

(continues on next page)

(continued from previous page)

```
plt.show()

Cu2 = 0.010
Zn2 = 1.000
T = 298.15
Q_case = Zn2 / Cu2
E_case = nernst(E0, T, n, Q_case)
print(f"[Cu2+] = {Cu2:.3f} M, [Zn2+] = {Zn2:.3f} M, T = {T:.2f} K")
print(f"Q = {Q_case:.1f}")
print(f"E = {E_case:.3f} V")
```



```
[Cu2+] = 0.010 M, [Zn2+] = 1.000 M, T = 298.15 K
Q = 100.0
E = 1.041 V
```

### 12.5.1 Interpretation

When  $Q$  increases, the reaction is shifted further toward products and the potential decreases. Temperature changes the slope of the logarithmic term.



## **Part III**

# **Cross-disciplinary Examples**



## CROSS-DISCIPLINARY EXAMPLES

### **i** What are cross-disciplinary examples?

They are reusable teaching patterns for many degrees, not examples locked to a single discipline. The idea is to learn a technique —modeling a process, analyzing a survey, building a rubric, summarizing texts, simulating data, or solving a balance— and then adapt it to sciences, humanities, social sciences, health, engineering, or education.

## 13.1 Example families

The following table summarizes the main elements of this section.

**Table. Example families.**

| Example                                          | Cross-disciplinary pattern                        | Can be adapted to...                                                        |
|--------------------------------------------------|---------------------------------------------------|-----------------------------------------------------------------------------|
| <i>Growth and saturation models</i>              | Systems that grow toward a limit                  | Populations, cultural diffusion, technology adoption, learning, epidemics   |
| <i>Questionnaire analysis</i>                    | Surveys and opinion scales                        | Psychology, education, sociology, teaching satisfaction, market studies     |
| <i>Text corpus frequencies</i>                   | Counting, classifying, and summarizing terms      | Languages, history, law, communication, document analysis, NLP              |
| <i>Rubrics and learning sequences</i>            | Transparent assessment and planning               | Any course with essays, labs, projects, or presentations                    |
| <i>Simulated data and variable relationships</i> | Data exploration and correlations                 | Health, economics, education, environment, social sciences, laboratory work |
| <i>Balances and conservation</i>                 | Inputs, outputs, accumulation, and transformation | Chemistry, ecology, economics, logistics, energy, budgets, demography       |

### **i** How to use this section

Do not copy the discipline: copy the **pattern**. Change the vocabulary, the data, and the teaching question so it fits your course. That is where the cross-disciplinary value lives.

## 13.2 The common idea

All these examples follow the same philosophy:

1. describe a problem with a clear structure;
2. represent data, rules, or criteria as code or tables;
3. generate a reproducible teaching artifact;
4. allow AI to help adapt the example to another context.

This turns each page into a template: it is not “a Biology example” or “a Psychology example”, but a recipe that can travel across fields.

## 13.3 Growth and Saturation Models

The logistic model describes any process that grows quickly at first and slows down as it approaches a limit: a population, technology adoption, diffusion of an idea, accumulated learning, or the early spread of a disease.

$$\frac{dN}{dt} = rN \left(1 - \frac{N}{K}\right)$$

- $N$  = accumulated quantity of the phenomenon
- $r$  = intrinsic growth rate
- $K$  = limit or maximum capacity of the system

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import odeint

def logistic(N, t, r, K):
 return r * N * (1 - N / K)

t = np.linspace(0, 50, 500)
N0 = 10

params = [(0.1, 500), (0.3, 500), (0.3, 1000)]
labels = ['r=0.1, K=500', 'r=0.3, K=500', 'r=0.3, K=1000']

fig, ax = plt.subplots(figsize=(8, 5))
for (r, K), label in zip(params, labels):
 sol = odeint(logistic, N0, t, args=(r, K))
 ax.plot(t, sol, label=label)

ax.set_xlabel('Time')
ax.set_ylabel('Population (N)')
ax.set_title('Logistic growth curves')
ax.legend()
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

### 13.3.1 Applications

- **Biology:** population dynamics, bacterial growth
- **Medicine:** epidemic spread (simplified SIR model)
- **Economics:** adoption of new technologies

#### Try it yourself

Change the values of `r` and `K` in the `params` list and observe how the curve changes. What happens when `r` is very high?

## 13.4 Questionnaire Analysis

We generate synthetic data from a Likert-scale survey (1-5) and analyze the results. The same pattern works for teaching satisfaction, classroom climate, public opinion, market studies, or self-assessment.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

np.random.seed(42)

n_participants = 120
questions = [
 'Clarity of explanations',
 'Teaching materials',
 'Practical usefulness',
 'Pace of the class',
 'Teacher availability',
 'Fair assessment',
 'Class atmosphere',
 'Workload',
 'Online resources',
 'Would recommend the course'
]

data = {}
for q in questions:
 center = np.random.choice([2.5, 3.0, 3.5, 4.0])
 data[q] = np.clip(np.round(np.random.normal(center, 0.8, n_participants)), 1, 5).
 ↪astype(int)

df = pd.DataFrame(data)
df.head(10)
```

```
summary = df.agg(['mean', 'std']).T.round(2)
summary.columns = ['Mean', 'Std Deviation']
summary
```

```
fig, ax = plt.subplots(figsize=(10, 6))

frequencies = df.apply(lambda col: col.value_counts().reindex(range(1, 6), fill_
```

(continues on next page)

(continued from previous page)

```

↪value=0))
colors = ['#d73027', '#fc8d59', '#fee08b', '#91bfdb', '#4575b4']

x = np.arange(len(questions))
width = 0.15

for i, (val, color) in enumerate(zip(range(1, 6), colors)):
 ax.bar(x + i * width, frequencies.loc[val], width, label=str(val), color=color)

ax.set_xticks(x + 2 * width)
ax.set_xticklabels([q[:25] + '...' if len(q) > 25 else q for q in questions], ↪
 ↪rotation=45, ha='right')
ax.set_ylabel('Number of responses')
ax.set_title('Response distribution by question')
ax.legend(title='Value', bbox_to_anchor=(1.05, 1))
plt.tight_layout()
plt.show()

```

### 13.4.1 Applications

- **Psychology:** attitude and perception analysis
- **Education:** student satisfaction evaluation
- **Medicine:** quality of life surveys

#### Adapt this example

Replace the `questions` list with your own survey items and adjust `n_participants`. The code generates data and plots automatically.

## 13.5 Text Corpus Frequencies

In any discipline that works with documents, **term frequency** indicates how many times each word or concept appears in a corpus. It helps detect dominant topics, key vocabulary, and shifts in focus across texts.

#### Why does frequency matter?

The most frequent words or concepts often reveal the structure of a discourse. The same technique can be applied to novels, legal decisions, interviews, scientific articles, historical newspapers, or open-ended survey responses.

### 13.5.1 Frequency table: sample corpus

The following table summarizes the main elements of this section.

**Table. Frequency table: sample corpus.**

| Term | Grammatical category  | Frequency | Cumulative percentage |
|------|-----------------------|-----------|-----------------------|
| the  | Definite article      | 6 892     | 12.3 %                |
| of   | Preposition           | 5 417     | 22.0 %                |
| and  | Conjunction           | 4 756     | 30.5 %                |
| to   | Preposition           | 4 102     | 37.8 %                |
| a    | Indefinite article    | 3 945     | 44.8 %                |
| in   | Preposition           | 3 401     | 50.9 %                |
| is   | Verb                  | 2 980     | 56.2 %                |
| it   | Pronoun               | 2 654     | 60.9 %                |
| for  | Preposition           | 2 432     | 65.3 %                |
| that | Conjunction / Pronoun | 2 180     | 69.2 %                |

### 13.5.2 Teaching applications

- **Humanities:** topic analysis in literary, historical, or philosophical texts
- **Social sciences:** study of interviews, open-ended surveys, or public discourse
- **Health sciences:** review of frequent terms in reports or simulated anamnesis
- **Engineering and computer science:** foundations of text mining and NLP
- **Education:** design of materials graded by lexical or conceptual difficulty

#### How to adapt this table

Replace the frequencies with those from your own corpus. Tools such as AntConc, Voyant Tools, or Python compute frequencies automatically from almost any collection of texts.

## 13.6 Rubrics and Learning Sequences

### 13.6.1 Rubric: Research project

The following table summarizes the main elements of this section.

**Table. Rubric: Research project.**

| Criterion                | Excellent (4)                                            | Good (3)                                      | Sufficient (2)                                   | Insufficient (1)                        |
|--------------------------|----------------------------------------------------------|-----------------------------------------------|--------------------------------------------------|-----------------------------------------|
| <b>Problem statement</b> | Defines an original, relevant problem with clear context | Defines an adequate problem with some context | Defines a vague or poorly contextualized problem | No identifiable problem defined         |
| <b>Methodology</b>       | Rigorous, well-justified, replicable method              | Adequate method with partial justification    | Simple method without clear justification        | No defined method                       |
| <b>Data analysis</b>     | In-depth analysis with correct interpretation            | Correct analysis with basic interpretation    | Superficial analysis with minor errors           | No analysis or major errors             |
| <b>Conclusions</b>       | Well-founded conclusions answering the problem           | Coherent conclusions aligned with data        | Partial or poorly connected conclusions          | No conclusions or unrelated to the work |
| <b>Presentation</b>      | Impeccable structure, clear writing, no errors           | Good structure, minimal errors                | Acceptable structure, some errors                | Disorganized, frequent errors           |

#### Scoring

Minimum passing score: **10 / 20** (50 %). The total score is obtained by adding the values of each criterion.

### 13.6.2 Learning objective sequence

The following table summarizes the main elements of this section.

**Table. Learning objective sequence.**

| Phase            | Objective                         | Expected outcome                |
|------------------|-----------------------------------|---------------------------------|
| 1. Exploration   | Identify a topic of interest      | List of 3 candidate topics      |
| 2. Delimitation  | Formulate a research question     | Concrete, viable question       |
| 3. Design        | Select an appropriate methodology | Research protocol               |
| 4. Collection    | Gather relevant data              | Organized dataset               |
| 5. Analysis      | Interpret the results             | Findings report                 |
| 6. Communication | Present the final work            | Written document + oral defense |

### 13.6.3 Cross-disciplinary applications

- **Humanities:** assessment of essays, text commentaries, and presentations
- **Social sciences:** assessment of case studies, reports, and debates
- **Experimental sciences:** assessment of lab work and field notebooks
- **Engineering:** assessment of projects, prototypes, and technical documentation
- **Health:** assessment of simulated clinical cases and diagnostic reasoning

#### Adapt the rubric

Replace the criteria in the first column with those from your course and adjust the descriptors. The structure works for almost any assessable product: essay, lab activity, report, project, poster, presentation, or portfolio.



## 13.7 Simulated Data and Variable Relationships

We generate a simulated dataset and explore correlations between variables. Although the example uses clinical variables, the pattern works for economics, education, environment, laboratory work, or social sciences.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

np.random.seed(42)
n = 200

age = np.random.randint(30, 80, n)
cholesterol = 150 + 1.5 * age + np.random.normal(0, 25, n)
systolic_bp = 110 + 0.5 * age + np.random.normal(0, 10, n)
risk = (0.3 * (age - 30) + 0.1 * (cholesterol - 150) + np.random.normal(0, 5, n)).
 clip(0, 100)

df = pd.DataFrame({
 'Age': age,
 'Cholesterol (mg/dL)': cholesterol.round(0),
 'Systolic BP (mmHg)': systolic_bp.round(0),
 'Cardiovascular risk (%)': risk.round(1)
})

df.describe().round(1)
```

```
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

axes[0].scatter(df['Age'], df['Cholesterol (mg/dL)'], alpha=0.5, c='steelblue')
axes[0].set_xlabel('Age (years)')
axes[0].set_ylabel('Cholesterol (mg/dL)')
axes[0].set_title('Age vs Cholesterol')

scatter = axes[1].scatter(
 df['Cholesterol (mg/dL)'], df['Cardiovascular risk (%)'],
 alpha=0.5, c=df['Age'], cmap='RdYlBu_r'
)
axes[1].set_xlabel('Cholesterol (mg/dL)')
axes[1].set_ylabel('Cardiovascular risk (%)')
axes[1].set_title('Cholesterol vs Risk (color = Age)')
plt.colorbar(scatter, ax=axes[1], label='Age')

plt.tight_layout()
plt.show()
```

```
correlations = df.corr().round(2)
correlations
```

### 13.7.1 Applications

- **Medicine:** exploratory analysis of clinical data
- **Psychology:** correlation between well-being variables
- **Biology:** relationship between physiological variables

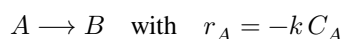
#### Important note

These data are **simulated** and do not represent real patients. In actual clinical research, always verify the origin and quality of the data.

## 13.8 Balances and Conservation

### 13.8.1 Problem statement

A solution of reactant A with concentration  $C_{A0} = 2.0$  mol/L is fed into a continuous stirred-tank reactor (CSTR) at a volumetric flow rate  $Q = 10$  L/min. The reaction is:



where  $k = 0.5 \text{ min}^{-1}$ . The reactor volume is  $V = 100$  L.

Determine the outlet concentration  $C_A$ .

### 13.8.2 Mass balance equation

The steady-state mass balance for component A is:

$$0 = Q C_{A0} - Q C_A + r_A V$$

Substituting the reaction rate expression:

$$0 = Q C_{A0} - Q C_A - k C_A V$$

### 13.8.3 Step-by-step solution

**Step 1:** Group terms with  $C_A$ .

$$Q C_{A0} = C_A (Q + kV)$$

**Step 2:** Solve for  $C_A$ .

$$C_A = \frac{Q C_{A0}}{Q + kV}$$

**Step 3:** Substitute numerical values.

$$C_A = \frac{10 \times 2.0}{10 + 0.5 \times 100} = \frac{20}{60} = 0.333 \text{ mol/L}$$

### 13.8.4 Conversion

The reactor conversion is defined as:

$$X = 1 - \frac{C_A}{C_{A0}} = 1 - \frac{0.333}{2.0} = 0.833 \text{ (83.3\%)}$$

#### Quick verification

Residence time:  $\tau = V/Q = 100/10 = 10$  min. For a first-order CSTR:  $X = k\tau/(1 + k\tau) = 5/6 = 0.833$ . The results match.

### 13.8.5 Cross-disciplinary applications

- **Chemical Engineering:** reactor design, stoichiometric calculations
- **Biology and environment:** bioreactor, ecosystem, and wastewater treatment models
- **Medicine:** pharmacokinetics and compartment models
- **Economics:** income, expenses, debt, and accumulation flows
- **Logistics:** inputs, outputs, and material storage
- **Energy:** power balances, consumption, and losses



## **Part IV**

# **Examples from Other Books**



## EXAMPLES FROM OTHER BOOKS

This section collects external books, Jupyter Books, TeachBooks, repositories, and teaching resources that can inspire the design of a new book.

The goal is not to provide a formal literature review, but to show real examples of structure, navigation, notebook use, exercises, AI policies, and web publication that can be adapted to courses in science and chemical science degrees.

**Table14.1:** Catalogue of external resources.

| Area or degree                             | Resource                                                              | Type                    | What a teacher can learn                                                                                                          | Link                 |
|--------------------------------------------|-----------------------------------------------------------------------|-------------------------|-----------------------------------------------------------------------------------------------------------------------------------|----------------------|
| Chemical Sciences                          | eChem: Computational Chemistry from Laptop to HPC                     | Jupyter Book            | How to organize an advanced computational chemistry book with notebooks, theory, molecular visualization, and complete workflows. | <a href="#">Open</a> |
| Chemical Sciences                          | PQ-Jupyter                                                            | Notebook repository     | Chemical process exercises with notebooks on balances, reactors, distillation, absorption, and extraction.                        | <a href="#">Open</a> |
| Statistics Degree                          | Learning from Data                                                    | Jupyter Book            | How to organize notes with examples on Bayesian statistics, regression, MCMC, Gaussian processes, and mini-projects.              | <a href="#">Open</a> |
| Physics Degree                             | ShowingPhysics                                                        | Jupyter Book            | A broad collection of physics demonstrations with clear navigation through pedagogy, inquiry, and conceptual-development blocks.  | <a href="#">Open</a> |
| Physics Degree                             | Quantum Mechanics I                                                   | Jupyter Book            | Quantum mechanics notes with notebooks, external material, references, and a brief guide to using the book.                       | <a href="#">Open</a> |
| Geology and Geological Engineering Degrees | Geo-Python                                                            | Teaching website        | A lesson-based course with exercises, Python programming, and an explicit policy for the use of AI tools.                         | <a href="#">Open</a> |
| Computer Science Degree                    | Python for Engineers                                                  | Teach-Book/Jupyter Book | Programming course with theory, exercises, part-based navigation, clear credits, and a teaching website maintained by TU Delft.   | <a href="#">Open</a> |
| Mathematics Degree                         | An Introduction to Python Jupyter Notebooks for College Math Teachers | Jupyter Book            | A mathematics book organized by areas, with notebooks, exercise solutions, and reusable material for teachers.                    | <a href="#">Open</a> |

When reviewing these examples, it is useful to focus on three aspects: how the table of contents is structured, what type of activities are proposed, and how text, code, visualizations, and exercises are combined.



**Part V**

**Basic Content**



## BASIC CONTENT

This section is your **quick reference** for the most common TeachBook elements. Each page shows the MyST code you need to write and explains what it produces when compiled.

### Note

All examples in this section work in both **HTML** and **PDF**, unless explicitly stated otherwise.

## 15.1 What you will find here

The following table summarizes the main elements of this section.

**Table. What you will find here.**

| Page                              | What you will learn                              |
|-----------------------------------|--------------------------------------------------|
| <i>MyST Markdown</i>              | Basic syntax for writing book pages              |
| <i>Figures and images</i>         | Inserting and referencing images                 |
| <i>Equations</i>                  | Inline and display math                          |
| <i>Citations and bibliography</i> | BibTeX bibliography                              |
| <i>Cross-references</i>           | Linking figures, tables, sections...             |
| <i>Tables</i>                     | Simple and formatted tables                      |
| <i>Admonitions and dropdowns</i>  | Notes, warnings, hints, collapsible blocks       |
| <i>Tabs and cards</i>             | Tabbed content (sphinx-design)                   |
| <i>Source code</i>                | Code blocks with syntax highlighting             |
| <i>Videos</i>                     | YouTube and local video (HTML + PDF)             |
| <i>Mermaid diagrams</i>           | Flowcharts, sequences, class diagrams...         |
| <i>HTML/PDF compatibility</i>     | What works where and how to ensure compatibility |

## 15.2 How to use this section

1. Find the page you need.
2. Copy the example that closest matches what you want.
3. Adapt the content to your case.
4. Build and verify the result.

### Tip

If you use an AI assistant, you can ask it to “generate a figure like the one in the *Figures* section but with my image” and save time.

## MYST MARKDOWN: THE BASICS

**MyST** (Mark up Your Scientific Text) is a Markdown extension designed for scientific and technical documents. It is the format used by Jupyter Book. If you know basic Markdown, you already know 80 % of MyST.

### 16.1 Essential Syntax

#### 16.1.1 Headings

```
Main title (H1)
Section (H2)
Subsection (H3)
```

#### 16.1.2 Text Formatting

```
Bold for strong emphasis.
Italic for terms or titles of works.
`inline code` for file names or commands.
```

#### 16.1.3 Lists

```
- Unordered list item
- Another item
 - Sub-item with indentation

1. First step
2. Second step
3. Third step
```

## 16.1.4 Links and Images

```
[Link text] (https://example.com)
```

```
! [Alt text] (path/to/image.png)
```

For images that you will explain or cite in the text, use the `{figure}` directive from the *Figures and images* section. This lets you add a caption, alternative text, and references with `{numref}`.

## 16.2 Directives

MyST adds **directives** that do not exist in standard Markdown:

### Note

This is a note. It appears highlighted in blue.

### Warning

This is a warning. It appears highlighted in red/orange.

### Tip

This is a tip. It appears highlighted in green.

You can create custom blocks with admonition:

```
```{admonition} My custom title
Block content goes here.
```
```

## 16.3 Math

Inline: the equation  $E = mc^2$  is written as `$E = mc^2$`.

Display:

$$\int_0^\infty e^{-x^2} dx = \frac{\sqrt{\pi}}{2}$$

Written as `$$ ... $$` on its own line.

## 16.4 Tables

The following table summarizes the main elements of this section.

**\*\*Table. Tables.\*\***

```
Column A	Column B	Column C
data 1	data 2	data 3
data 4	data 5	data 6
```

Result:

The following table summarizes the main elements of this section.

**Table. Tables.**

| Column A | Column B | Column C |
|----------|----------|----------|
| data 1   | data 2   | data 3   |
| data 4   | data 5   | data 6   |

## 16.5 Code Blocks

```
```python
def greet(name):
    return f"Hello, {name}!"
```
```

### Tip

Do not try to memorize all the syntax. Use an AI assistant to generate the MyST you need and learn as you go.





## FIGURES AND IMAGES

Figures are essential in any teaching material. MyST offers two main directives: `{image}` (simple image) and `{figure}` (image with number and caption).

### 17.1 Centered image with alt text

The `{image}` directive inserts an image without numbering:

```
``{image} ../../_static/logo.png
:alt: TeachBook project logo
:width: 60%
:align: center
``
```

Result:

ELABORACIÓN DE LIBROS  
ELECTRÓNICOS MEDIANTE CÓDIGO  
Y ASISTENTES DE  
INTELIGENCIA ARTIFICIAL



VNIVERSIDAD  
D SALAMANCA

## 17.2 Numbered figure with caption

The `{figure}` directive adds automatic numbering (Figure 1, Figure 2...):

```
```{figure} ../../_static/logo.png
:width: 50%
:align: center
:name: fig-figures-1

Official TeachBook project logo.
```
```

Result:

The [Fig. 17.1](#) visually summarizes this part of the explanation.



**Figure17.1:** Official TeachBook project logo.

## 17.3 Cross-reference to a figure

Use `{numref}` to reference figures by their number:

```
The {numref}`fig-logo-en` shows the project logo.
```

Result: The [Fig. 17.1](#) shows the project logo.

## 17.4 Margin figure

Add `:figclass: margin` to place an image in the margin (HTML only):

```
```{figure} ../../_static/logo.png
:figclass: margin
:width: 100%
:name: fig-figures-3
```

(continues on next page)

(continued from previous page)

Logo in the margin.
`\begin{margin}
\includegraphics[width=100px]{logo.png}
\end{margin}`

Result:

The Fig. 17.2 visually summarizes this part of the explanation.



Figure17.2: Logo in the margin.

17.5 Useful parameters

The following table summarizes the main elements of this section.

Table. Useful parameters.

Parameter	What it does	Example
<code>:width:</code>	Width (50%, 200px)	<code>:width: 80%</code>
<code>:align:</code>	Alignment (left, center, right)	<code>:align: center</code>
<code>:alt:</code>	Alt text (accessibility)	<code>:alt: Diagram</code>
<code>:name:</code>	Label for {numref}	<code>:name: fig-map</code>
<code>:figclass:</code>	CSS class (margin)	<code>:figclass: margin</code>

Tip

Always add `:alt:` for accessibility. It is the text that screen readers see and displays if the image fails to load.

EQUATIONS AND MATHEMATICS

MyST supports LaTeX math via the `dollarmath` extension, already enabled in this template. Works in **HTML and PDF**.

18.1 Inline math

Use `$...$` within a line of text:

```
Energy is calculated as $E = mc^2$, where $m$ is mass.
```

Result: Energy is calculated as $E = mc^2$, where m is mass.

18.2 Display equation (unnumbered)

Use `$$...$$` on its own line:

```
$$
\int_0^\infty e^{-x^2} \, dx = \frac{\sqrt{\pi}}{2}
$$
```

Result:

$$\int_0^\infty e^{-x^2} dx = \frac{\sqrt{\pi}}{2}$$

18.3 Numbered equation with label

Add a label (`label`) = before the equation to reference it:

```
$$
\frac{\partial u}{\partial t} = \alpha \nabla^2 u
$$ (eq-heat)
```

Result:

$$\frac{\partial u}{\partial t} = \alpha \nabla^2 u \tag{18.1}$$

18.4 Cross-reference to an equation

Use `{eq}` to cite an equation by its number:

Equation `{eq}`eq-heat`` describes heat diffusion.

Result: Equation (18.1) describes heat diffusion.

18.5 Common math symbols

The following table summarizes the main elements of this section.

Table. Common math symbols.

Symbol	LaTeX code	Symbol	LaTeX code
α	<code>\alpha</code>	β	<code>\beta</code>
γ	<code>\gamma</code>	δ	<code>\delta</code>
π	<code>\pi</code>	θ	<code>\theta</code>
λ	<code>\lambda</code>	σ	<code>\sigma</code>
$\sum$	<code>\sum</code>	$\prod$	<code>\prod</code>
$\int$	<code>\int</code>	$\oint$	<code>\oint</code>
∞	<code>\infty</code>	∇	<code>\nabla</code>
$\frac{a}{b}$	<code>\frac{a}{b}</code>	$\sqrt{x}$	<code>\sqrt{x}</code>
$\leq$	<code>\leq</code>	$\geq$	<code>\geq</code>
$\approx$	<code>\approx</code>	$\neq$	<code>\neq</code>
$\vec{v}$	<code>\vec{v}</code>	$\hat{x}$	<code>\hat{x}</code>

Tip

For complex math, ask your AI assistant to generate LaTeX code from a natural language description: “generate the Gauss integral formula in LaTeX.”

CITATIONS AND BIBLIOGRAPHY

This template includes `sphinxcontrib-bibtex`, which manages bibliographic references using BibTeX files.

19.1 Bibliography file

Bibliography entries are stored in `book/_static/references.bib`. Each entry follows this format:

```
@book{einstein1920,
  author   = {Albert Einstein},
  title    = {Relativity: The Special and General Theory},
  year     = {1920},
  publisher = {Henry Holt and Company}
}

@article{dirac1930,
  author   = {Paul A. M. Dirac},
  title    = {A Theory of Electrons and Protons},
  journal  = {Proceedings of the Royal Society A},
  year     = {1930},
  volume   = {126},
  pages    = {360--365}
}
```

19.2 Textual citation: `{cite:t}`

Inserts the author name followed by the year:

```
{cite:t}`einstein1920` formulated the theory of relativity.
```

Result: Einstein [Ein20] formulated the theory of relativity.

19.3 Parenthetical citation: `{cite:p}`

Inserts author and year in parentheses:

```
Quantum mechanics revolutionized physics {cite:p}`dirac1930`.
```

Result: Quantum mechanics revolutionized physics [Dir30].

19.4 Multiple citations

Separate multiple keys with commas:

```
Several authors contributed {cite:p}`einstein1920,dirac1930`.
```

Result: Several authors contributed [Dir30, Ein20].

19.5 Printing the bibliography

The template uses two bibliography levels:

- **Global:** the final Bibliography page gathers the entries cited in the book, not every entry from `book/_static/references.bib`.
- **Local:** an individual page can show only its own references in an HTML-only block.

19.5.1 Global book bibliography

The global bibliography is already prepared on the final Bibliography page:

```
```${bibliography}  
:cited:
```
```

Do not add this block to normal pages: it should exist only on the global bibliography page.

Note

If you add local bibliographies to web pages, some entries will also appear on the global Bibliography page. This is expected: the local bibliography helps with page-level reading, while the global page gathers the whole book bibliography. In PDF, only the final global bibliography is printed.

19.5.2 Local page bibliography

For a local bibliography on the web, cite with `{cite:t}` or `{cite:p}` in the normal page text and add a filtered `{bibliography}` directive inside `{only}` `html` at the end. This lets HTML show page-level references, while the PDF keeps only the final global bibliography.

Text with a local citation `{cite:p}`citation_key``.

```

````{only} html
Bibliography for this page

````{bibliography}
:filter: docname in docnames
````

```

You can also show only selected keys from the document in HTML:

```

````{only} html
````{bibliography}
:filter: docname in docnames and key in {"ziman2000", "hodson1996"}
````

```

In this local example, Ziman [Zim00] describes science as a social practice, not just as a collection of results. Laboratory work should be connected to explanation-building, not only to following instructions [Hod96].

19.6 Workflow

1. Add BibTeX entries to `_static/references.bib`.
2. Cite with `{cite:t}` or `{cite:p}` on any page.
3. If you want a local bibliography visible in HTML, add an `{only}` `html` block with a filtered `{bibliography}` directive at the end of the page.
4. In PDF, do not generate local bibliographies: the template creates a temporary `.bib` with the used citations and shows them only on the global Bibliography page.

Warning

Each citation key (e.g., `einstein1920`) must be unique across the entire `.bib` file. Duplicates will cause a build failure.

CROSS-REFERENCES

Cross-references allow you to link figures, tables, equations, and sections within the book. This is **essential** for long documents: if you rearrange chapters, links update automatically.

20.1 Label a section

Write `(label)=` just before the heading:

```
(my-topic)=  
## My Important Topic  
  
Content here...
```

20.2 Reference a section

Use `{ref}` to create a link with the heading text:

```
See the {ref}`my-topic` section for details.
```

Result: See the *Figures and Images* section for details.

20.3 Label and reference figures

Use `:name:` inside the `{figure}` directive and `{numref}` to cite by number:

```
````{figure} ../../_static/logo.png  
:width: 40%
:name: to cite by number:

Project logo.
````  
  
The {numref}`fig-logo-ref-en` shows the logo.
```



Figure20.1: Project logo.

Result:

The Fig. 20.1 visually summarizes this part of the explanation.

The Fig. 20.1 shows the logo.

20.4 Label and reference equations

Use `(eq-label)` = after the equation and `{eq}` to cite:

```
$$
F = ma
$$ (eq-newton-en)

Equation {eq}`eq-newton-en` is Newton's second law.
```

Result:

$$F = ma \quad (20.1)$$

Equation (20.1) is Newton's second law.

20.5 Label and reference tables

Use `:name:` inside `{table}` and `{numref}` to cite:

```
````{table} Physical constants
:name: tab-constants-en
:align: center

| Constant | Symbol | Value |
|-----|-----|-----|
| Speed of light | c | 3×10^8 m/s |
| Planck | h | 6.63×10^{-34} J·s |
````

The {numref}`tab-constants-en` lists fundamental constants.
```

Result:

Table20.1: Physical constants

| Constant | Symbol | Value |
|----------------|--------|----------------------------|
| Speed of light | c | 3×10^8 m/s |
| Planck | h | 6.63×10^{-34} J.s |

The [Table 20.1](#) lists fundamental constants.

20.6 Roles summary

The following table summarizes the main elements of this section.

Table. Roles summary.

| Role | What it references | Example |
|-----------------------|----------------------------------|--|
| <code>{ref}</code> | Sections (uses heading text) | <code>{ref}`my-topic`</code> |
| <code>{numref}</code> | Figures and tables (uses number) | <code>{numref}`fig-logo-ref-en`</code> |
| <code>{eq}</code> | Numbered equations | <code>{eq}`eq-newton-en`</code> |

Tip

Use descriptive label names: `fig-chromatogram`, `eq-bernoulli`, `tab-raw-data`. They are easier to remember than `fig1` or `eq2`.

TABLES

Tables are the most direct way to present structured data. MyST supports standard Markdown tables and the `{table}` directive for adding numbering.

21.1 Simple Markdown table

The following table summarizes the main elements of this section.

****Table. Simple Markdown table.****

```
Element	Symbol	Atomic number
Hydrogen	H	1
Helium	He	2
Lithium	Li	3
```

Result:

The following table summarizes the main elements of this section.

Table. Simple Markdown table.

| Element | Symbol | Atomic number |
|----------|--------|---------------|
| Hydrogen | H | 1 |
| Helium | He | 2 |
| Lithium | Li | 3 |

21.2 Numbered table with caption

Use `{table}` for automatic numbering and cross-referencing:

```
```{table} Common laboratory solvents
:name: tab-solvents
:align: center

| Solvent | Formula | Boiling point (°C) |
|-----|-----|-----|
| Water | H2O | 100 |
```

(continues on next page)

(continued from previous page)

```
| Ethanol | C2H5OH | 78
| Acetone | CH3COCH3 | 56
| Ether | C4H10O | 35
````
```

Result:

Table21.1: Common laboratory solvents

| Solvent | Formula | Boiling point (°C) |
|---------|-----------------------------------|--------------------|
| Water | H ₂ O | 100 |
| Ethanol | C ₂ H ₅ OH | 78 |
| Acetone | CH ₃ COCH ₃ | 56 |
| Ether | C ₄ H ₁₀ O | 35 |

21.3 List-table directive

MyST does not support per-column alignment in Markdown tables. Use `{list-table}` for more control:

```
````{list-table} Experimental data
:header-rows: 1
:name: tab-list-en
:align: center

* - Sample
 - pH
 - Conductivity (mS/cm)
* - A
 - 7.2
 - 1.45
* - B
 - 6.8
 - 2.10
* - C
 - 8.1
 - 0.95
````
```

Result:

Table21.2: Experimental data

| Sample | pH | Conductivity (mS/cm) |
|--------|-----|----------------------|
| A | 7.2 | 1.45 |
| B | 6.8 | 2.10 |
| C | 8.1 | 0.95 |

21.4 Tips for readable tables

- **Fewer than 6 columns:** wide tables are hard to read on screen.
- **Align numbers:** makes visual comparison easier.
- **Units in the header:** do not repeat units in each cell.
- **Long tables:** if you exceed 15 rows, consider using a chart instead.

Tip

The [Table 21.1](#) shows an example with units in the header. Notice how numeric columns make comparison easier.

ADMONITIONS AND DROPDOWNS

Admonitions are **highlighted boxes** that draw the reader's attention. They are perfect for hints, warnings, key definitions, and exercise solutions.

22.1 Note ({note})

```
```{note}
The exam will be held on June 15th in room 3.2.
```
```

Result:

Note

The exam will be held on June 15th in room 3.2.

22.2 Warning ({warning})

```
```{warning}
Do not mix hydrochloric acid with bleach. It produces toxic gas.
```
```

Result:

Warning

Do not mix hydrochloric acid with bleach. It produces toxic gas.

22.3 Tip ({tip})

```
```{tip}
To memorize the periodic table, use a color association technique.
```
```

Result:



Tip

To memorize the periodic table, use a color association technique.

22.4 Important ({important})

```
```{important}
The submission deadline is Friday at 23:59. Late submissions are not accepted.
```
```

Result:



Important

The submission deadline is Friday at 23:59. Late submissions are not accepted.

22.5 Caution ({caution})

```
```{caution}
This reagent is corrosive. Wear gloves and safety goggles.
```
```

Result:



Caution

This reagent is corrosive. Wear gloves and safety goggles.

22.6 Custom admonition

Use {admonition} with your own title:

```
```{admonition} Definition: covalent bond
A covalent bond is the union between two atoms that share one or more pairs of
↪electrons.
```
```

Result:

Definition: covalent bond

A covalent bond is the union between two atoms that share one or more pairs of electrons.

22.7 Dropdown (collapsible content)

Add `:class: dropdown` to hide content until the reader clicks:

```
```{admonition} Solution to exercise 3
:class: dropdown

The molar mass of water (H2O) is:

$$M = 2 \times 1.008 + 16.00 = 18.016 \text{ g/mol}$$

```
```

Result:

Solution to exercise 3

The molar mass of water (H₂O) is:

$$M = 2 \times 1.008 + 16.00 = 18.016 \text{ g/mol}$$

22.8 Teaching use cases

The following table summarizes the main elements of this section.

Table. Teaching use cases.

| Type | Common use |
|--------------------------|--------------------------------------|
| <code>{note}</code> | Complementary information, reminders |
| <code>{warning}</code> | Hazards, common errors |
| <code>{tip}</code> | Study tips, tricks |
| <code>{important}</code> | Deadlines, mandatory requirements |
| <code>{caution}</code> | Lab safety precautions |
| <code>dropdown</code> | Exercise solutions, lengthy steps |

Tip

Combine admonitions with math, code, or images inside the block. All MyST content works inside an admonition.

TABS AND CARDS

The **sphinx-design** extension (already installed) lets you organize content into tabs and cards. Ideal for comparing options, showing code in multiple languages, or presenting alternative steps.

23.1 Simple tabs ({tab-set} / {tab-item})

```
::::{tab-set}
:::{tab-item} Python
```python
print("Hello, world")
```

:::

:::{tab-item} R
```r
print("Hello, world")
```

:::

:::{tab-item} MATLAB
```matlab
disp('Hello, world')
```

:::
::::
```

Result:

Python

```
print("Hello, world")
```

R

```
print("Hello, world")
```

MATLAB

```
disp('Hello, world')
```

23.2 Grouped tabs ({tab-set} / {tab-item})

Tab groups allow synchronization:

```
::::{tab-set}
::{tab-item} Analytical method
Solving the differential equation directly.
:::

::{tab-item} Numerical method
Approximating the solution step by step with a computer.
:::
::::
```

Result:

Analytical method

Solving the differential equation directly.

Numerical method

Approximating the solution step by step with a computer.

23.3 Tabs for procedure steps

```
::::{tab-set}
::{tab-item} Step 1: Prepare the sample
Weigh 0.5 g of the solid compound with 0.001 g precision.
:::

::{tab-item} Step 2: Dissolve
Add 50 mL of solvent and stir for 5 minutes.
:::

::{tab-item} Step 3: Measure
Read absorbance at 280 nm using the spectrophotometer.
:::
::::
```

Result:

Step 1: Prepare the sample

Weigh 0.5 g of the solid compound with 0.001 g precision.

Step 2: Dissolve

Add 50 mL of solvent and stir for 5 minutes.

Step 3: Measure

Read absorbance at 280 nm using the spectrophotometer.

23.4 Cards ({card})

Cards present information in visual blocks:

```
```{card} Lab 1: Spectroscopy
Determination of protein concentration using the Bradford method.
+++
Week 3
```
```

Result:

Lab 1: Spectroscopy Determination of protein concentration using the Bradford method.

Week 3

Warning

Tabs and cards work in **HTML** but **not in PDF**. In PDF, the content of all tabs is shown sequentially.

CODE BLOCKS

Code blocks are fundamental in science: they show computational formulas, analysis scripts, and step-by-step procedures.

24.1 Syntax highlighting

Add the language after the backticks:

```
```python
import numpy as np

data = np.array([3.2, 4.1, 3.8, 4.5, 3.9])
mean = np.mean(data)
print(f"Mean: {mean:.2f}")
```
```

Result:

```
import numpy as np

data = np.array([3.2, 4.1, 3.8, 4.5, 3.9])
mean = np.mean(data)
print(f"Mean: {mean:.2f}")
```

24.2 Line numbers

Use the `{code-block}` directive with `:linenos:` to show line numbers. This option does not work inside a normal block written as ````python`; in that case it is interpreted as one more line of code.

```
```{code-block} python
:linenos:

def calculate_std(data):
 n = len(data)
 mean = sum(data) / n
 variance = sum((x - mean) ** 2 for x in data) / n
 return variance ** 0.5
```
```

Result:

```

1 def calculate_std(data):
2     n = len(data)
3     mean = sum(data) / n
4     variance = sum((x - mean) ** 2 for x in data) / n
5     return variance ** 0.5

```

24.3 Highlight specific lines

Use `:emphasize-lines: inside {code-block}` to highlight specific lines:

```

```{code-block} python
:emphasize-lines: 3

def read_data(filename):
 with open(filename, 'r') as f:
 lines = f.readlines() # This line will be highlighted
 return [float(l.strip()) for l in lines]
```

```

Result:

```

def read_data(filename):
    with open(filename, 'r') as f:
        lines = f.readlines()
    return [float(l.strip()) for l in lines]

```

24.4 Other languages

Highlighting works with many languages:

```

# Bash example
curl -O https://example.com/data.csv
head -5 data.csv

```

```

-- SQL example
SELECT name, grade
FROM students
WHERE grade >= 5.0
ORDER BY grade DESC;

```

24.5 Common languages in science

The following table summarizes the main elements of this section.

Table. Common languages in science.

| Language | When to use it |
|-------------|------------------------------------|
| python | Data analysis, numerical computing |
| r | Statistics, biostatistics |
| matlab | Engineering, signal processing |
| sql | Database queries |
| bash | Task automation |
| json / yaml | Configuration files |

Tip

This template includes `sphinx-copybutton`: a **Copy** button appears in the corner of each code block. Readers can copy code with one click.

VIDEOS

Videos greatly enrich teaching materials. However, videos **cannot be embedded in PDF**, so you must always provide a text alternative.

25.1 YouTube video (HTML + PDF)

ALWAYS use this dual pattern with `{raw} html` and `{raw} latex`:

```
```{raw} html
<iframe width="560" height="315" src="https://www.youtube.com/embed/dQw4w9WgXcQ"
↪frameborder="0" allowfullscreen></iframe>
```

```{raw} latex
\begin{center}
\textbf{Video:} \url{https://www.youtube.com/watch?v=dQw4w9WgXcQ}
\end{center}
```
```

Result:

Video: <https://www.youtube.com/watch?v=dQw4w9WgXcQ>

25.2 How to get a YouTube video ID

1. Open the video in your browser.
2. The URL has this format: `https://www.youtube.com/watch?v=VIDEO_ID`
3. Use VIDEO_ID in the embed URL: `https://www.youtube.com/embed/VIDEO_ID`

For example, if the URL is `https://www.youtube.com/watch?v=abc123`, the iframe uses `https://www.youtube.com/embed/abc123`.

25.3 Local video (HTML5)

For videos hosted in `_static/`:

```
```{raw} html
<video width="560" controls>
 <source src="_static/my_video.mp4" type="video/mp4">
 Your browser does not support HTML5 video.
</video>
```

```{raw} latex
\begin{center}
\textbf{Local video:} see the digital version to play it.
\end{center}
```
```

Recommended path for local book videos:

- `book/_static/videos/my_video.mp4`

25.4 Externally generated videos

The base installation of this template avoids heavy video-generation dependencies. If you need an animation, a screen recording, or a recorded explanation, generate the `.mp4` with the external tool of your choice and save the result in `book/_static/videos/`.

Then insert it with the local video pattern above and always add the `{raw} latex` block so the PDF keeps a textual reference.

25.5 Video with description

It is good practice to add context before the video:

The following video shows the crystallization process of copper sulfate:

[insert video block here]

Duration: 4:32 | Topic: Crystallization

25.6 Recommended complete pattern

The following video shows the experimental setup:

```
```{raw} html
<iframe width="560" height="315" src="https://www.youtube.com/embed/VIDEO_ID"
frameborder="0" allowfullscreen></iframe>
```

```{raw} latex
```

(continues on next page)



(continued from previous page)

```
\begin{center}
\textbf{Video: Experimental setup}\\
\url{https://www.youtube.com/watch?v=VIDEO_ID}
\end{center}
```\n`
```

****Duration:**** 5:15 | ****Topic:**** Chemical kinetics

Warning

NEVER use only `{raw}` html without the `{raw}` latex block. The PDF will contain no reference to the video and the reader will lose information.

DIAGRAMS WITH KROKI (MERMAID)

Kroki converts text into SVG diagrams during the book build. If you use `:type: mermaid`, you can write Mermaid syntax and get a diagram that works in both **HTML** and **PDF**.

26.1 Flowchart (flowchart)

```
```{kroki}
:type: mermaid
:align: center

%%{init: {"themeVariables": {"fontSize": "12px", "fontFamily": "Arial"}}}%
flowchart LR
 A[Sample] --> B[Preparation]
 B --> C[Measurement]
 C --> D[Analysis]
 D --> E[Results]
```
```

Result:

As shown in Fig. 26.1, the diagram is versioned as a static image.

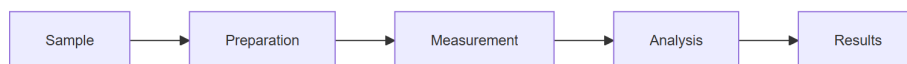


Figure 26.1: Diagram: Flowchart (flowchart).

```
```{kroki}
:type: mermaid
:align: center

%%{init: {"themeVariables": {"fontSize": "12px", "fontFamily": "Arial"}}}%
sequenceDiagram
 participant S as Student
 participant T as Teacher
 participant L as Laboratory
 S->>T: Submit report
 T->>S: Corrections
 S->>L: Book equipment
 L->>S: Confirmation
```
```

Result:

As shown in Fig. 26.2, the diagram is versioned as a static image.

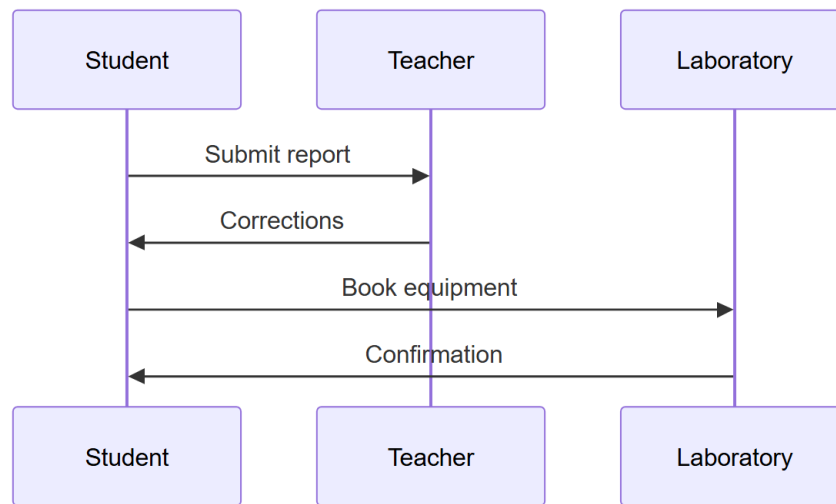


Figure26.2: Diagram: Sequence diagram (sequenceDiagram).

```

```{kroki}
:type: mermaid
:align: center

%%{init: {"themeVariables": {"fontSize": "11px", "fontFamily": "Arial"}}}%
classDiagram
 direction LR
 class Experiment {
 +name
 +run()
 }
 class Sample {
 +type
 +prepare()
 }
 Experiment --> Sample : uses
```

```

Result:

As shown in Fig. 26.3, the diagram is versioned as a static image.

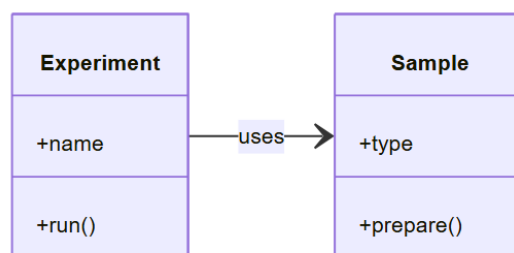


Figure26.3: Diagram: Class diagram (classDiagram).

```

```{kroki}
:type: mermaid
:align: center

%%{init: {"themeVariables": {"fontSize": "11px", "fontFamily": "Arial"}}}%
erDiagram
 STUDENT ||--o{ ENROLLMENT : makes
    ```

```

Result:

As shown in Fig. 26.4, the diagram is versioned as a static image.

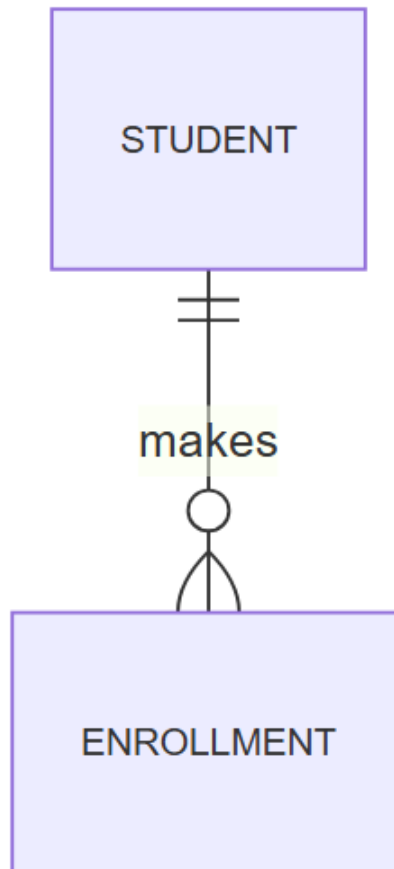


Figure26.4: Diagram: Entity-relationship diagram (erDiagram).

```

```{kroki}
:type: mermaid
:align: center

%%{init: {"themeVariables": {"fontSize": "11px", "fontFamily": "Arial"}}}%
stateDiagram-v2
 direction LR
 [*] --> Pending
 Pending --> InProgress : start
    ```

```

(continues on next page)

(continued from previous page)

```
InProgress --> Completed : finish
InProgress --> Error : failure
Error --> Pending : retry
Completed --> [*]
...
```

Result:

As shown in Fig. 26.5, the diagram is versioned as a static image.

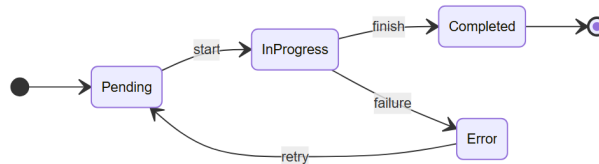


Figure 26.5: Diagram: State diagram (stateDiagram-v2).

The following table summarizes the main elements of this section.

Table. Flowchart directions.

Code	Direction
flowchart LR	Left to right
flowchart RL	Right to left
flowchart TB	Top to bottom
flowchart BT	Bottom to top

💡 Tip

Always use {kroki} with :type: mermaid instead of {mermaid}. This makes the diagram work in both HTML and PDF without manual fallbacks.

HTML AND PDF COMPATIBILITY

Your TeachBook is published in two formats: **HTML** (interactive web) and **PDF** (printable version). Not everything works the same in both. This page helps you write content that always works.

27.1 Compatibility table

The following table summarizes the main elements of this section.

Table. Compatibility table.

Element	HTML	PDF	Strategy
Text, lists, links	✓	✓	Use freely
Images (<code>{image}</code> , <code>{figure}</code>)	✓	✓	Use freely
LaTeX math ($\$ \dots \$$, $\$ \$ \dots \$ \$$)	✓	✓	Use freely
Tables	✓	✓	Use freely
Admonitions (<code>{note}</code> , <code>{tip}</code> ...)	✓	✓	Use freely
Dropdowns (<code>:class: dropdown</code>)	✓	✓ expanded	In PDF it appears expanded
Syntax-highlighted code	✓	✓	Use freely
BibTeX citations	✓	✓	Use freely
Cross-references	✓	✓	Use freely
Kroki diagrams (Mermaid, PlantUML, etc.)	✓	✓	Use <code>{kroki}</code> with <code>:type: mermaid</code>
Videos (iframe/YouTube)	✓	✗	Use <code>{raw} latex</code> with URL
Tabs (<code>{tabbed}</code> , <code>{tab-set}</code>)	✓	✗	In PDF: sequential content
Interactive HTML (<code><details></code>)	✓	✗	Use <code>{raw} latex</code> alternative
Thebe (live code)	✓	✗	Code visible as text in PDF

27.2 Universal pattern: HTML + LaTeX fallback

For any content that only works in HTML, use this pattern:

```
```${raw} html
<!-- Interactive content here -->
<div>...</div>
```

```${raw} latex
\begin{center}
```

(continues on next page)

(continued from previous page)

```
Alternative text for the PDF.
\end{center}
```
```

27.3 Example: video with fallback

```
```{raw} html
<iframe width="560" height="315" src="https://www.youtube.com/embed/VIDEO_ID"
 frameborder="0" allowfullscreen></iframe>
```

```{raw} latex
\begin{center}
\textbf{Video:} \url{https://www.youtube.com/watch?v=VIDEO_ID}
\end{center}
```
```

27.4 Example: diagram with Kroki

```
```{kroki}
:type: mermaid
:align: center

flowchart LR
 A --> B --> C
```
```

27.5 Practical rules

1. **If it is text, image, equation, or table:** do not worry, it works in both.
2. **If it is interactive (video, custom HTML):** ALWAYS add a text alternative.
3. **If it is a diagram:** use Kroki. It works in both HTML and PDF.
4. **If it is a dropdown:** in PDF it appears expanded (no problem).
5. **If it uses tabs:** in PDF everything appears sequentially (verify it still makes sense).

Tip

Before publishing, always build both formats (`build_book.py` for HTML and `export_pdf.py` for PDF) and verify the content is coherent in both.

Part VI

Interactivity and Assessment

INTERACTIVITY AND ASSESSMENT

This section covers the different ways to add **interactive content** and **assessment** to your TeachBook.

28.1 What will you find here?

Interactivity in a TeachBook ranges from simple dropdowns to live executable code in the browser. This section covers: The following table summarizes the main elements of this section.

Table. What will you find here?.

| Resource | Type | Works in PDF? |
|--------------------------|----------------------------------|---------------|
| Thebe + Binder | Live code with a remote kernel | No |
| Interactive notebooks | Code executed at build time | Partial |
| Widgets (ipywidgets) | Interactive controls | No |
| Quizzes (jupyterquiz) | Self-correcting questions | No |
| Parametric questions | Exercises with random values | Partial |
| Exercises with solutions | Dropdowns with answers | Yes |
| Interactive HTML | Pure JS elements (no frameworks) | No |
| H5P (optional) | External interactive content | No |

Tip

Don't try to use everything at once. Start with **exercises with solutions** (simple and PDF-compatible) and add interactivity as you need it.

Warning

Features marked as experimental may change in future versions. Always verify that interactive content works correctly in your environment before publishing.

28.2 Thebe + Binder: Live Code

Thebe allows readers to run Python code from the web version of the book. In this project it connects to a **remote Binder kernel**.

Warning

Thebe is an **HTML-only** feature. It is not available in PDF export.

Warning

This feature is **optional** and requires additional configuration of a Jupyter server or Binder. You do not need it for the basic TeachBook setup.

28.2.1 How does it work?

1. Click the “**Live Code**” button below.
 2. Binder starts a remote environment (this may take a few seconds the first time).
 3. Code cells become editable and executable — try it!
-

28.2.2 Try it: executable code

Click the button to activate interactive mode. Then you can edit and run the cells.

```
import numpy as np

# Create an array from 0 to 2*pi
x = np.linspace(0, 2 * np.pi, 100)

# Calculate sine and cosine
y_sin = np.sin(x)
y_cos = np.cos(x)

print(f"Mean of sine: {np.mean(y_sin):.4f}")
print(f"Mean of cosine: {np.mean(y_cos):.4f}")
print(f"Max value of sine: {np.max(y_sin):.4f}")
print("It works! Try changing np.sin to another function.")
```

```
import matplotlib.pyplot as plt
import numpy as np

# Modifiable parameters
frequency = 2      # Hz
amplitude = 1.0    # Scale
phase = 0          # Radians

t = np.linspace(0, 2*np.pi, 200)
```

(continues on next page)

(continued from previous page)

```
signal = amplitude * np.sin(frequency * t + phase)

plt.figure(figsize=(8, 3))
plt.plot(t, signal, 'b-', linewidth=2, label=f'{amplitude}*sin({frequency}t + {phase})')
plt.title('Interactive Sine Wave')
plt.xlabel('Time (rad)')
plt.ylabel('Amplitude')
plt.grid(True, alpha=0.3)
plt.legend()
plt.tight_layout()
plt.show()
```

28.2.3 How to add this to your pages

1. Activation button

Add this directive where you want the “Live Code” button to appear:

```
``{thebe-button}
``
```

2. Executable cell

Use code-block with the thebe class:

```
``{code-block} python
:class: thebe

import numpy as np
print("Hello from the browser!")
``
```

You may also find examples written with the {code-cell} directive:

```
``{code-cell} python
import numpy as np
print(np.sqrt(16))
``
```

That syntax turns the block into an executable cell when Thebe is configured.

3. _config.yml configuration

Your `_config.yml` must include the `sphinx-thebe` extension:

```
sphinx:
  extra_extensions:
    - sphinx_thebe
  config:
    thebe_config:
      always_load: false
      repository_url: https://github.com/binder-examples/jupyter-stacks-datascience
      repository_branch: master
```

This configuration uses **Thebe + Binder**, meaning the code runs in a remote Binder environment.

Simpler alternative

If you don't want to set up Thebe, you can use **Google Colab**. Upload your notebook to Colab and share the link in your TeachBook. Students can run the code in the cloud with no installation.

Summary

Thebe is powerful but optional. Start with static content and notebooks, and consider Thebe when your students need to experiment with code in real time.

28.3 Interactive Notebooks

A **Jupyter Notebook** is a document that mixes explanatory text and executable code in cells. It is ideal for teaching: you can explain a concept and show the calculation in the same document.

This notebook demonstrates how to create interactive content that students can modify. Change the variable values and re-run the cells to see the results.

28.3.1 Basic example: $y = x^2$

This first example computes and plots the function $y = x^2$. It shows the minimal notebook structure: one cell with data, one cell with a plot, and a short interpretation.

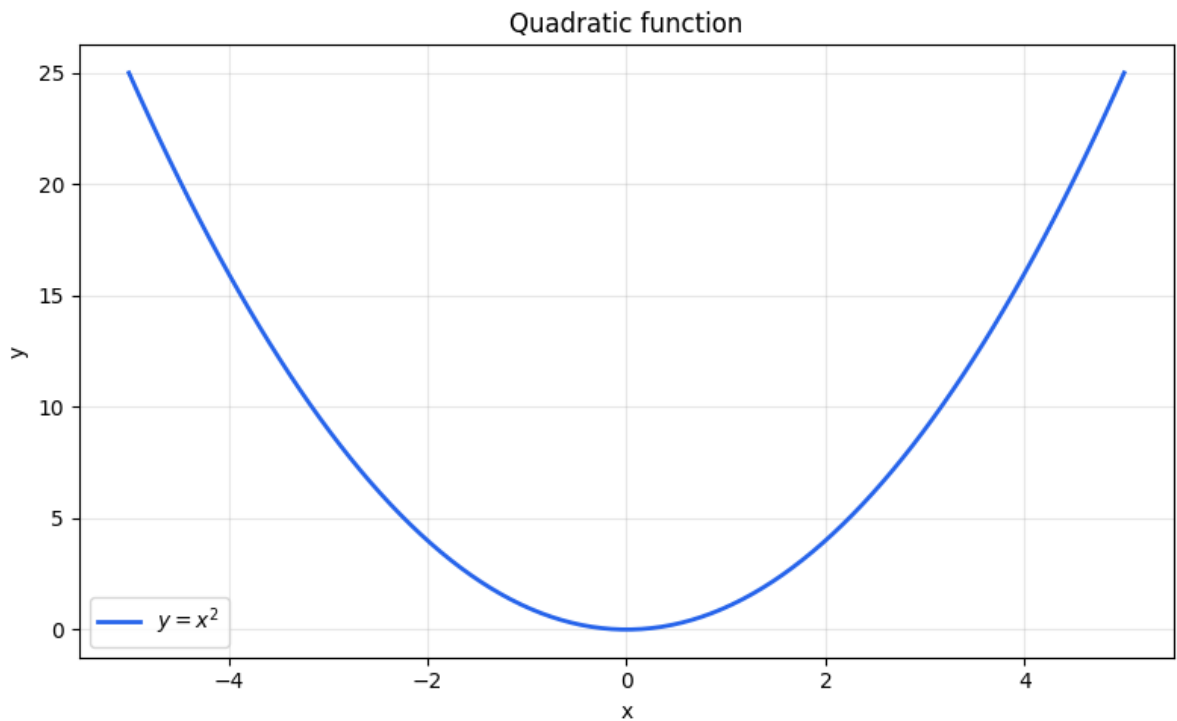
```
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(-5, 5, 100)
y = x ** 2

print(f"Minimum value of y: {y.min()}")
print(f"Value at x=3: {3**2}")
```

```
Minimum value of y: 0.002550760126517668
Value at x=3: 9
```

```
fig, ax = plt.subplots(figsize=(8, 5))
ax.plot(x, y, label=r"$y = x^2$", color="#2563eb", linewidth=2)
ax.set_xlabel("x")
ax.set_ylabel("y")
ax.set_title("Quadratic function")
ax.legend()
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```



28.3.2 Observations

- The function $y = x^2$ is a parabola with its vertex at the origin.
- Change the exponent in $y = x ** 2$ to $x ** 3$ and see how the plot changes.
- Notebooks let you experiment directly: modify the values and re-run the cell.

```
import numpy as np
import matplotlib.pyplot as plt

amplitude = 2.0
frequency = 1.5
phase = np.pi / 4

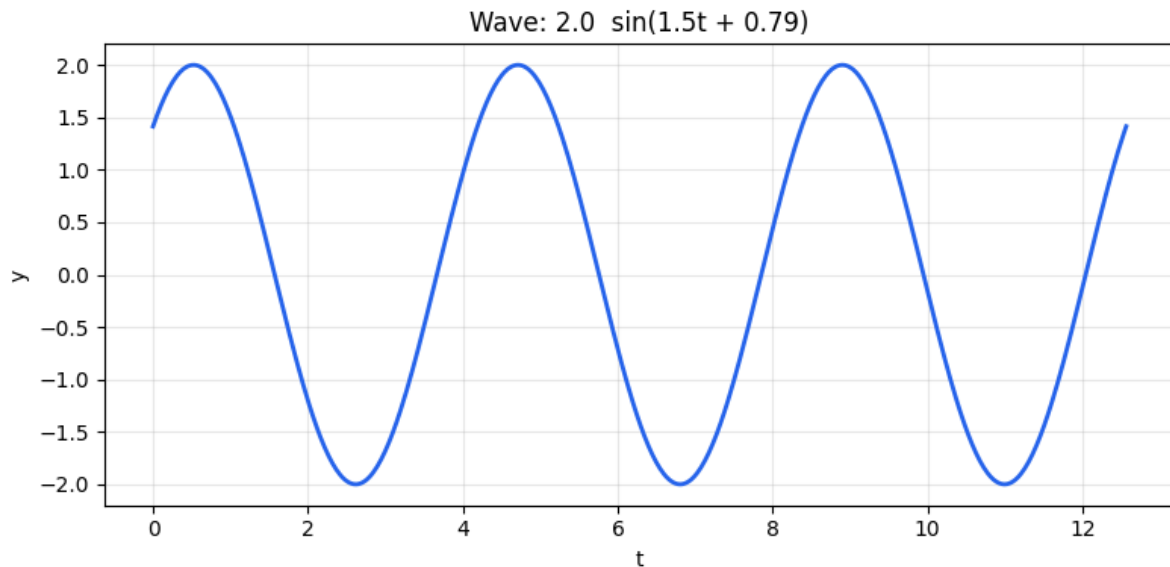
t = np.linspace(0, 4 * np.pi, 500)
y = amplitude * np.sin(frequency * t + phase)

plt.figure(figsize=(8, 4))
plt.plot(t, y, color='#2563eb', linewidth=2)
```

(continues on next page)

(continued from previous page)

```
plt.title(f'Wave: {amplitude} sin({frequency}t + {phase:.2f})')
plt.xlabel('t')
plt.ylabel('y')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```



28.3.3 Wave superposition

Modify the frequencies of the two waves to see phenomena like constructive and destructive interference.

```
freq1 = 3.0
freq2 = 3.2

t = np.linspace(0, 10, 1000)
wave1 = np.sin(2 * np.pi * freq1 * t)
wave2 = np.sin(2 * np.pi * freq2 * t)

fig, axes = plt.subplots(3, 1, figsize=(8, 8), sharex=True)

axes[0].plot(t, wave1, color='#2563eb')
axes[0].set_ylabel('Wave 1')
axes[0].set_title(f'f1 = {freq1} Hz')

axes[1].plot(t, wave2, color='#dc2626')
axes[1].set_ylabel('Wave 2')
axes[1].set_title(f'f2 = {freq2} Hz')

axes[2].plot(t, wave1 + wave2, color='#16a34a')
axes[2].set_ylabel('Sum')
axes[2].set_title('Wave 1 + Wave 2 (beats)')
axes[2].set_xlabel('Time (s)')

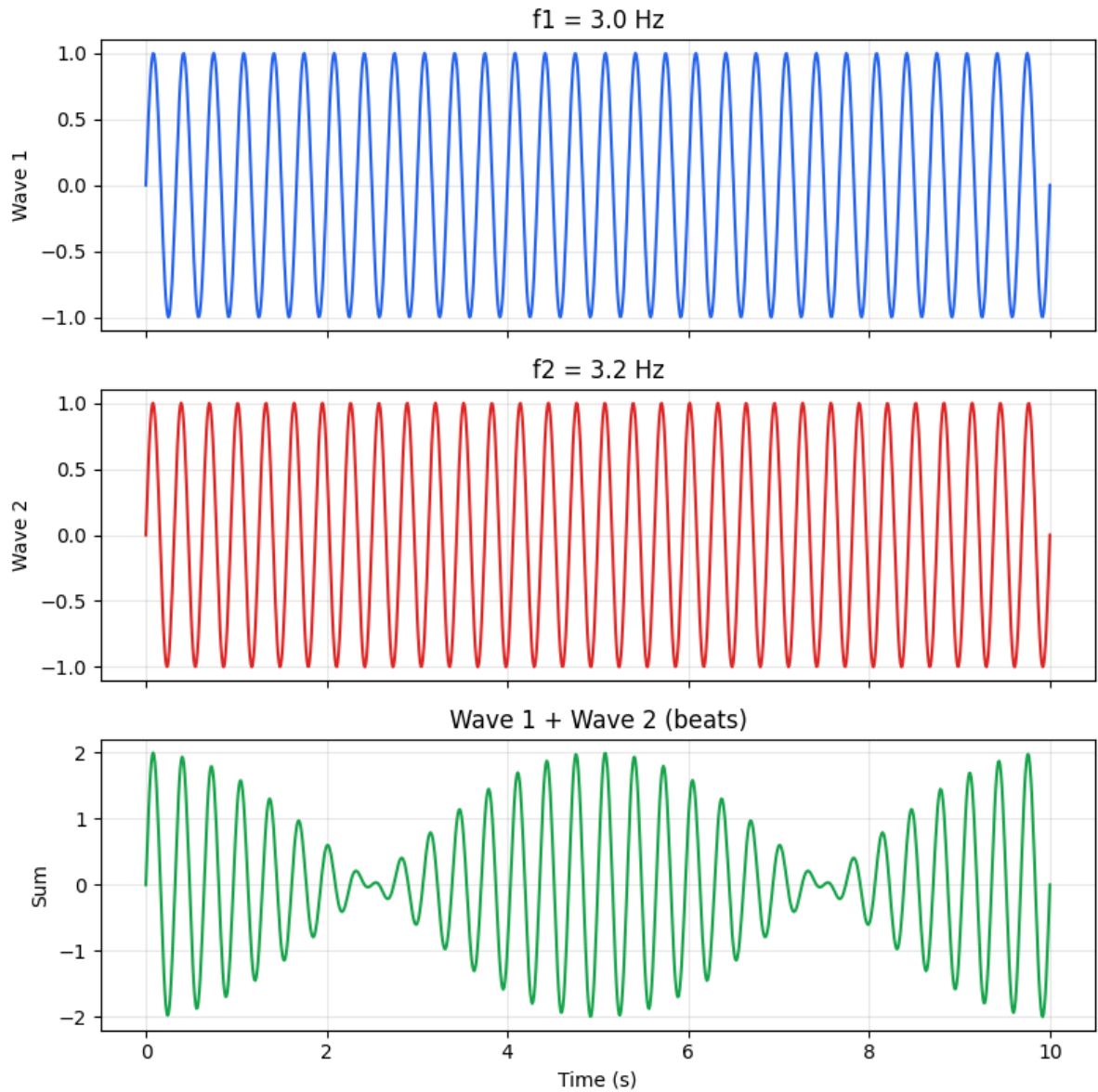
for ax in axes:
```

(continues on next page)

(continued from previous page)

```
ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```



28.3.4 Exercise

1. Set `freq1 = freq2 = 5.0`. What do you observe?
2. Set `freq1 = 10.0` and `freq2 = 10.5`. What is the beat frequency?
3. Change the amplitude of the second wave and observe the result.

28.4 Interactive Widgets (ipywidgets)

Widgets let you add interactive controls such as sliders, dropdowns, and buttons. They are very useful for letting students explore parameters visually.

```
import numpy as np
import matplotlib.pyplot as plt
from ipywidgets import interact, FloatSlider, Dropdown

@interact(freq=FloatSlider(min=0.5, max=10.0, step=0.1, value=2.0, description=
    ↪ 'Frequency'),
          amp=FloatSlider(min=0.1, max=5.0, step=0.1, value=1.0, description=
    ↪ 'Amplitude'))
def plot_wave(freq, amp):
    t = np.linspace(0, 2 * np.pi, 500)
    y = amp * np.sin(freq * t)
    plt.figure(figsize=(8, 4))
    plt.plot(t, y, color='#2563eb', linewidth=2)
    plt.ylim(-5.5, 5.5)
    plt.title(f'{amp:.1f} sin({freq:.1f} t)')
    plt.xlabel('t')
    plt.grid(True, alpha=0.3)
    plt.tight_layout()
    plt.show()

interactive(children=(FloatSlider(value=2.0, description='Frequency', max=10.0,
    ↪ min=0.5), FloatSlider(value=1.0,
```

28.4.1 Function selector

Use the dropdown to choose which function to plot.

```
@interact(func=Dropdown(options=['sin', 'cos', 'tan', 'exp', 'log'], value='sin',
    ↪ description='Function'))
def plot_func(func):
    t = np.linspace(-np.pi, np.pi, 500)
    funcs = {'sin': np.sin, 'cos': np.cos, 'tan': np.tan, 'exp': np.exp, 'log': np.
    ↪ log}
    y = funcs[func](t)
    y_clipped = np.clip(y, -5, 5)
    plt.figure(figsize=(8, 4))
    plt.plot(t, y_clipped, color='#16a34a', linewidth=2)
    plt.title(f'f(t) = {func}(t)')
    plt.xlabel('t')
    plt.ylim(-5.5, 5.5)
    plt.grid(True, alpha=0.3)
```

(continues on next page)

(continued from previous page)

```
plt.tight_layout()
plt.show()
```

```
interactive(children=(Dropdown(description='Function', options=('sin', 'cos', 'tan',
↵', 'exp', 'log'), value='sin...
```

28.5 Quizzes with jupyterquiz

jupyterquiz lets you embed multiple-choice and numeric questions directly in a notebook. Questions are automatically graded upon answering.

Requirement: pip install jupyterquiz

```
from jupyterquiz import display_quiz

physics_questions = [
    {
        "question": "What is the SI unit for force?",
        "type": "multiple_choice",
        "answers": [
            {"answer": "Joule (J)", "correct": False, "feedback": "The joule is a
↵unit of energy."},
            {"answer": "Newton (N)", "correct": True, "feedback": "Correct. 1 N = 1
↵kgm/s2."},
            {"answer": "Pascal (Pa)", "correct": False, "feedback": "The pascal is a
↵unit of pressure."},
            {"answer": "Watt (W)", "correct": False, "feedback": "The watt is a unit
↵of power."}
        ]
    },
    {
        "question": "What is the approximate acceleration due to gravity at Earth's
↵surface?",
        "type": "numeric",
        "answers": [
            {"type": "value", "value": 9.8, "correct": True, "feedback": "Correct. g
↵9.8 m/s2."},
            {"type": "range", "range": [9.78, 9.83], "correct": True, "feedback":
↵"Correct, within the expected range."},
            {"type": "value", "value": 10, "correct": False, "feedback": "A common
↵approximation, but not the standard value."}
        ]
    },
    {
        "question": "Newton's Second Law is expressed as:",
        "type": "multiple_choice",
        "answers": [
            {"answer": "F = ma", "correct": True, "feedback": "Correct. Force equals
↵mass times acceleration."},
            {"answer": "F = mv", "correct": False, "feedback": "mv is momentum."},
            {"answer": "F = mg", "correct": False, "feedback": "This is the weight
↵formula, a specific case."},
            {"answer": "E = mc2", "correct": False, "feedback": "This is Einstein's
↵mass-energy equivalence."}
        ]
    }
]
```

(continues on next page)

(continued from previous page)

```
]
    }
]

display_quiz(physics_questions)
```

```
<IPython.core.display.HTML object>
```

```
<IPython.core.display.Javascript object>
```

28.6 Parametric Questions

Parametric questions generate **random values** on each execution, so each student gets a different problem with the same structure. Re-run the cells to get new values.

```
import random
import math

a = random.randint(1, 10)
b = random.randint(1, 10)
c = random.randint(-5, 5)

print(f"Solve the quadratic equation: {a}x2 + {b}x + {c} = 0")
print(f"\nCalculate the discriminant = b2 - 4ac and the solutions.")

discriminant = b**2 - 4*a*c
print(f"\n--- Solution (expand to reveal) ---")
print(f"= {b}^2 - 4{a}{c} = {discriminant}")
if discriminant > 0:
    x1 = (-b + math.sqrt(discriminant)) / (2*a)
    x2 = (-b - math.sqrt(discriminant)) / (2*a)
    print(f"x1 = {x1:.4f}")
    print(f"x2 = {x2:.4f}")
elif discriminant == 0:
    x = -b / (2*a)
    print(f"Double root: x = {x:.4f}")
else:
    real = -b / (2*a)
    imag = math.sqrt(-discriminant) / (2*a)
    print(f"x1 = {real:.4f} + {imag:.4f}i")
    print(f"x2 = {real:.4f} - {imag:.4f}i")
```

```
Solve the quadratic equation: 10x2 + 5x + -1 = 0
```

```
Calculate the discriminant = b2 - 4ac and the solutions.
```

```
--- Solution (expand to reveal) ---
= 52 - 4(10)(-1) = 65
x1 = 0.1531
x2 = -0.6531
```

28.6.1 Parallel resistors

Calculate the equivalent resistance and total current.

```
R1 = random.choice([10, 22, 33, 47, 68, 100, 150, 220, 330, 470])
R2 = random.choice([10, 22, 33, 47, 68, 100, 150, 220, 330, 470])
R3 = random.choice([10, 22, 33, 47, 68, 100, 150, 220, 330, 470])
V = random.choice([5, 9, 12, 24])

print(f"Three resistors in parallel: R1 = {R1} , R2 = {R2} , R3 = {R3} ")
print(f"Applied voltage: V = {V} V")
print(f"\nCalculate the equivalent resistance Req and total current I.")

Req = 1 / (1/R1 + 1/R2 + 1/R3)
I_total = V / Req

print(f"\n--- Solution ---")
print(f"1/Req = 1/{R1} + 1/{R2} + 1/{R3}")
print(f"Req = {Req:.2f} ")
print(f"I = V/Req = {V}/{Req:.2f} = {I_total:.4f} A = {I_total*1000:.2f} mA")
```

```
Three resistors in parallel: R1 = 470 , R2 = 10 , R3 = 470
Applied voltage: V = 9 V

Calculate the equivalent resistance Req and total current I.

--- Solution ---
1/Req = 1/470 + 1/10 + 1/470
Req = 9.59
I = V/Req = 9/9.59 = 0.9383 A = 938.30 mA
```

28.7 Exercises with Solutions

Patterns for creating exercises with hidden solutions that students can reveal when ready. This approach works in both **HTML** and **PDF**.

28.7.1 Basic pattern

Use {admonition} for the exercise statement and {admonition} with :class: dropdown for the solution:

```
```{admonition} Exercise 1 – Basic derivative
:class: important

Calculate the derivative of $f(x) = 3x^4 - 2x^2 + 7x - 1$.
```

```{admonition} Solution
:class: dropdown

Apply differentiation rules term by term:

 $f'(x) = 12x^3 - 4x + 7$
```
```

i Exercise 1 — Basic derivative

Calculate the derivative of $f(x) = 3x^4 - 2x^2 + 7x - 1$.

i Solution

Apply differentiation rules term by term:

$$f'(x) = 12x^3 - 4x + 7$$

28.7.2 Exercises with difficulty levels

Add difficulty hints to guide the student:

i Exercise 2 — Integral (level: basic)

Calculate $\int_0^3 (2x + 1) dx$.

i Solution

$$\int_0^3 (2x + 1) dx = [x^2 + x]_0^3 = (9 + 3) - 0 = 12$$

i Exercise 3 — Integral (level: intermediate)

Calculate $\int_0^\pi x \sin(x) dx$. *Hint: integration by parts.*

i Solution

Use integration by parts with $u = x$ and $dv = \sin(x) dx$:

$$\int_0^\pi x \sin(x) dx = [-x \cos(x)]_0^\pi + \int_0^\pi \cos(x) dx = \pi + 0 = \pi$$

i Exercise 4 — Integral (level: advanced)

Calculate $\int_{-\infty}^\infty e^{-x^2} dx$. *Hint: use polar coordinates.*

i Solution

Let $I = \int_{-\infty}^{\infty} e^{-x^2} dx$. Then:

$$I^2 = \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} e^{-(x^2+y^2)} dx dy$$

Converting to polar: $I^2 = \int_0^{2\pi} \int_0^{\infty} e^{-r^2} r dr d\theta = 2\pi \cdot \frac{1}{2} = \pi$

Therefore: $I = \sqrt{\pi}$

i Tip

Dropdowns appear expanded in PDF export, which is desirable for the printed version. On the web, the student must click to reveal the solution.

28.8 Interactive HTML (no frameworks)

You can create interactive elements using **plain HTML and JavaScript**, without React, Vue, or any framework. All code is self-contained on the page itself.

Warning

Content in `{raw} html` blocks does **not appear in PDF**. Always add a `{raw} latex` block with alternative text for the printed version.

28.8.1 When to use interactive HTML

Use interactive HTML when you need simple behavior on the web: dropdowns, small controls, styled tables, or self-contained calculators. You do not need React, Vue, or any framework.

The basic directive is:

```
```${raw} html
<your HTML here>
```
```

Everything you write inside is inserted as-is into the web page. To keep the book working in PDF, pair that block with a `{raw} latex` alternative containing the essential information.

28.8.2 Collapsible content with `<details>`

Hint: Remember that the derivative of e^x is e^x .

28.8.3 Slider with live output

Temperature converter: $T_{\text{F}} = T_{\text{C}} \times \frac{9}{5} + 32$

28.8.4 Simple calculator

Ohm's Law calculator: $V = I \times R$

Rules for interactive HTML

1. **No JS frameworks:** do not use React, Vue, Angular, or npm.
2. **No external files:** all JS goes inline inside the `{raw} html` block.
3. **Always include LaTeX fallback:** add a `{raw} latex` block with essential content for PDF.
4. **Inline styles:** use `style=""` on tags to avoid depending on external CSS.

28.9 H5P (Optional)

H5P is an interactive content platform that offers quizzes, drag-and-drop activities, video interactions, and much more.

Warning

H5P is **optional** and **not a dependency** of this template. It requires an external server to host the content ([H5P.com](https://h5p.com), Moodle, WordPress with H5P plugin, etc.). It is not self-contained.

28.9.1 What does H5P offer?

The following table summarizes the main elements of this section.

Table. What does H5P offer?.

| Activity type | Description |
|--------------------------|--------------------------------------|
| Quizzes | Multiple choice, fill in the blanks |
| Drag & Drop | Drag elements to correct zones |
| Interactive video | Add pauses with questions on a video |
| Interactive presentation | Slides with integrated questions |
| Summary | Text with interspersed questions |

28.9.2 How to embed H5P in your TeachBook

Create your content on the H5P platform and use an `iframe` to display it:

```
```{raw} html
<iframe src="https://H5P_URL_HERE" width="800" height="600" frameborder="0"
 ↪allowfullscreen="allowfullscreen" style="border: 1px solid #e5e7eb; border-radius:
 ↪8px; "></iframe>
```

```{raw} latex
\textbf{Interactive H5P activity:} Available in the web version of the book.
Link: \url{https://H5P_URL_HERE}
```
```

28.9.3 Compatible platforms

- **H5P.com**: Official service (paid for advanced use).
- **Moodle**: Free plugin, ideal if your university already uses Moodle.
- **WordPress**: Free H5P plugin.
- **Lumi**: Desktop application to create H5P without a server.

Recommended alternative

If you don't need the complexity of H5P, **exercises with solutions** (using `{admonition}` with dropdown) cover most assessment needs and work in both HTML and PDF, with no external server required.

Part VII

Web Publication

WEB PUBLICATION

Web publication turns the generated book into a static site that can be opened from any browser. This template supports two publication routes for different needs:

- **GitHub Pages:** the recommended option for getting started. GitHub builds the book, generates the PDFs, and publishes the website automatically when changes are pushed to the main branch.
- **Own server:** the right option when the book must live under an institutional domain, for example `libro.usal.es`, or on university-managed hosting.

Both routes use the same technical output: the `book/_build/html` folder generated by `scripts/build_book.py`. The difference is where that folder is copied and which service serves the files publicly.

Important

Never write usernames, passwords, or access credentials inside the repository. Deployments to own servers must use GitHub Actions secrets.

The following sections summarize the two publication flows supported by the template.

29.1 GitHub Pages

GitHub Pages is the default deployment route for this template. It is the simplest flow because it does not require an own server: GitHub Actions builds the book and GitHub Pages serves the resulting website.

29.1.1 What the workflow does

The `.github/workflows/deploy.yml` file runs when changes are pushed to `main` and can also be started manually from the **Actions** tab. In each run it:

1. prepares Python 3.12;
2. installs the project environment with `scripts/setup_env.py`;
3. checks UTF-8 encoding and static assets;
4. installs the lightweight LaTeX toolchain;
5. generates the PDFs for all languages;
6. builds the multi-language HTML site;
7. publishes `book/_build/html` to GitHub Pages.

This order matters: PDFs are generated first and the website is built afterwards, so download links always point to current files.

29.1.2 Initial activation

In a new repository created from the template, the recommended setup is:

1. open **Settings** in GitHub;
2. go to **Pages**;
3. choose **Source: GitHub Actions**;
4. save the configuration;
5. push the first change to `main`.

After the first push, GitHub runs the `deploy-book` workflow. If it finishes successfully, the public URL appears on the GitHub Pages settings page and in the action summary.

29.1.3 Public and private repositories

The workflow is configured to read repository contents with minimum permissions. In private repositories, publication depends on the GitHub Pages features available for the account or organization. If GitHub Pages is not available for that repository, the book can still be built, but it will not be published through this route.

29.1.4 When to use GitHub Pages

Use GitHub Pages when:

- you want to publish quickly without requesting a server;
- the book can live under a GitHub URL;
- each change in `main` should update the website automatically;
- you are using the template for the first time and need the lowest-configuration path.

29.2 Own Server

Publishing to an own server lets you deploy the same book under an institutional domain, for example `libro.usal.es`, using SFTP. This option is useful when the website must integrate with university infrastructure or when you need control over the final domain.

29.2.1 What the server needs

The server must allow SFTP access with username and password, and it must provide a public directory where the generated website can be copied. In many hosting setups this directory is called `public_html`, although the exact name depends on the institutional configuration.

The SFTP deployment does not configure the domain by itself. There are two separate pieces:

- **Server:** receives the HTML, CSS, JavaScript, image, and PDF files.
- **Domain:** points an address such as `libro.usal.es` to the correct server.

The template builds and uploads the files. DNS and hosting configuration depend on the institution's web service.

29.2.2 Required secrets

The `.github/workflows/sftp-deploy.yml` workflow uses GitHub Actions secrets. They must be configured in **Settings → Secrets and variables → Actions**:

| Secret | Purpose |
|----------------------------|--|
| <code>SFTP_SERVER</code> | SFTP server name or connection domain. |
| <code>SFTP_USERNAME</code> | SFTP username. |
| <code>SFTP_PASSWORD</code> | SFTP password. |
| <code>SFTP_PORT</code> | SFTP port. Optional; if missing, 22 is used. |

Warning

Do not write these values in `.yml` files, Markdown, notebooks, or scripts. They must exist only as repository secrets.

29.2.3 Running the deployment

The own-server deployment is manual. To run it:

1. open the **Actions** tab in GitHub;
2. select **sftp-deploy-book**;
3. click **Run workflow**;
4. enter the remote directory, `public_html` by default;
5. confirm the run.

The workflow builds the complete book, generates the PDFs, and uploads `book/_build/html` to the selected remote directory.

29.2.4 Remote directory cleanup

The SFTP deployment synchronizes the server with cleanup: remote files that no longer exist in the local build are deleted. This prevents old pages from remaining published by accident.

Before deployment, the workflow validates that the remote directory is not a dangerous path such as `/`, `.`, `..`, or an absolute path. Even so, check the `remote_dir` value carefully, because that directory will be made to match `book/_build/html`.

29.2.5 When to use an own server

Use an own server when:

- you need to publish under an institutional domain;
- the book must integrate with an existing website;
- the university provides SFTP hosting;
- you want to separate final publication from the GitHub domain.

Part VIII

Information

ABOUT

You can cite this book as:

Álvaro Lozano Murciego and André Sales Mendes (2025). *Creating Electronic Books with Code and Artificial Intelligence Assistants*. https://elloza.com/teachbook_sciences_template/ . Source files at https://github.com/elloza/teachbook_sciences_template . CC BY 4.0.

You can also cite individual pages of the book as:

<Page title>. In Álvaro Lozano Murciego and André Sales Mendes (2025). *Creating Electronic Books with Code and Artificial Intelligence Assistants*. https://elloza.com/teachbook_sciences_template/ . Source files at https://github.com/elloza/teachbook_sciences_template . CC BY 4.0.

Because this book is expected to evolve over time, it is recommended to cite the source repository as well whenever the exact context of the material matters.

30.1 How this book is made

This website is written in Markdown files and Jupyter notebooks, and converted to HTML using tools from [TeachBooks](#) and [Jupyter Book](#).

The source files are stored in a public GitHub repository:

- https://github.com/elloza/teachbook_sciences_template

The published version of the book can be viewed at:

- https://elloza.com/teachbook_sciences_template/

30.2 How to reproduce the website

There are two main options:

1. **On GitHub:** fork the repository, enable GitHub Pages with source GitHub Actions, and run the deployment workflow.
2. **Locally:** clone the repository, install dependencies, and build the book with the project scripts.

30.3 About the authors

30.3.1 Authors

- Álvaro Lozano Murciego
- André Sales Mendes

30.3.2 Institution

Faculties of Sciences and Chemical Sciences, University of Salamanca (USAL)

CREDITS AND LICENSES

31.1 Project credits

This book and the course template are built with [TeachBooks](#), on top of the [Jupyter Book](#) and [MyST Markdown](#) ecosystem.

TeachBooks provides documentation, components and working patterns that make it easier to create open, interactive and reproducible teaching books.

31.2 Acknowledgements

We thank the [TeachBooks](#) project and [Delft University of Technology \(TU Delft\)](#) for their work on open tools for educational books. The [credits page in the TeachBooks Manual](#) is used as a reference for explicitly acknowledging the tools, communities and institutional support on which this material builds.

31.3 Book license

This book is distributed under the [Creative Commons Attribution 4.0 International \(CC BY 4.0\)](#) license.

This allows the material to be shared and adapted, as long as attribution is provided.

31.4 External resources

Unless explicitly stated on a specific page, the content of this book is authored by the project authors.

If any external resources are reused in a page, they should be acknowledged on that page together with their corresponding attribution and license.

31.5 Code and tools

This project uses external tools and libraries, including:

- [TeachBooks](#)
- [Jupyter Book](#)
- [MyST Markdown](#)

Each tool keeps its own license.

HOW TO CITE

If you use this material, please cite it as follows:

32.1 Text Citation

Faculties of Sciences and Chemical Sciences (2025). *Creating Electronic Books with Code and Artificial Intelligence Assistants*. University of Salamanca. Available at: [Repository URL]

32.2 BibTeX

```
@book{creating_electronic_books_ai_2025,  
  author = {Faculties of Sciences and Chemical Sciences},  
  title = {Creating Electronic Books with Code and Artificial Intelligence Assistants}  
  ↩,  
  year = {2025},  
  publisher = {University of Salamanca},  
  url = {https://github.com/user/repo}  
}
```

32.3 DOI

(Pending assignment via Zenodo)

BIBLIOGRAPHY

- [ProjectJupyterBC+25] Project Jupyter, Evan Bolyen, J. Gregory Caporaso, Rowan Cockett, Danny Garside, Chris Holdgraf, Angus Hollands, Julia Kent, Franklin Koch, Jim Madge, Matthew McKay, Mike Morrison, Fernando Pérez, Steve Purves, Min Ragan Kelley, Brian E. J. Rose, Malvika Sharan, Brigitta M. Sipőcz, Stéfan J. Walt, and Kirstie Jane Whitaker. Jupyter Book 2 and the MyST Document Stack: A Modular, Extensible, Web-Native Stack for Authoring and Publishing Computational Narratives. In *Proceedings of the 24th Python in Science Conference*, 173–193. 2025. URL: <https://doi.org/10.25080/hwcj9957>, doi:10.25080/hwcj9957.
- [TeachBooksDTeam25] TeachBooks Development Team. TeachBooks Manual. 2025. CC BY 4.0. URL: <https://teachbooks.io/manual/intro.html> (visited on 2026-05-22).
- [ABucicS25] Noga Alon, Matija Bucić, and Lisa Sauermann. Unit and distinct distances in typical norms. *Geometric and Functional Analysis*, 35:1–42, 2025.
- [Bra96] Peter Brass. Erdős distance problems in normed spaces. *Computational Geometry*, 6(4):195–214, 1996.
- [BMP05] Peter Brass, William O. J. Moser, and János Pach. *Research Problems in Discrete Geometry*. Springer, New York, 2005.
- [BilkaBF+10] Ondřej Bílka, Kevin Buchin, Radoslav Fulek, Masashi Kiyomi, Yoshio Okamoto, Shin-ichi Tanigawa, and Csaba D. Tóth. A tight lower bound for convexly independent subsets of the minkowski sums of planar point sets. *Electronic Journal of Combinatorics*, 17(1):Note 35, 2010.
- [Dav00] Harold Davenport. *Multiplicative Number Theory*. Volume 74 of Graduate Texts in Mathematics. Springer, New York, third edition, 2000. Revised by Hugh L. Montgomery.
- [DdSMS99] John D. Dixon, Marcus P. F. du Sautoy, Avinoam Mann, and Dan Segal. *Analytic Pro-p Groups*. Volume 61 of Cambridge Studies in Advanced Mathematics. Cambridge University Press, Cambridge, second edition, 1999.
- [EPRothvossS08] Friedrich Eisenbrand, János Pach, Thomas Rothvoß, and Nir B. Sopher. Convexly independent subsets of the minkowski sum of planar point sets. *Electronic Journal of Combinatorics*, 15(1):Note 8, 2008.
- [ErdHos46] Paul Erdős. On sets of distances of n points. *American Mathematical Monthly*, 53(5):248–250, 1946.
- [ErdHosF97] Paul Erdős and Peter C. Fishburn. Minimum planar sets with maximum equidistance counts. *Computational Geometry*, 7(4):207–218, 1997.
- [GS64] E. S. Golod and I. R. Shafarevich. On the class field tower. *Izvestiya Akademii Nauk SSSR. Seriya Matematicheskaya*, 28(2):261–272, 1964. English translation: American Mathematical Society Translations, Series 2, 48 (1965), 91–102.
- [GS65] E. S. Golod and I. R. Shafarevich. On class field towers. In *Fourteen Papers on Logic, Algebra, Complex Variables and Topology*, volume 48 of American Mathematical Society Translations, Series 2, pages 91–102. American Mathematical Society, Providence, RI, 1965.
- [GST25] Josef Greilhuber, Carl Schildkraut, and Jonathan Tidor. More unit distances in arbitrary norms. *Bulletin of the London Mathematical Society*, 57(9):2885–2901, 2025.

- [GK15] Larry Guth and Nets Hawk Katz. On the erdős distinct distances problem in the plane. *Annals of Mathematics*, 181(1):155–190, 2015.
- [HM01] Farshid Hajir and Christian Maire. Asymptotically good towers of global fields. In *European Congress of Mathematics, Vol. II (Barcelona, 2000)*, volume 202 of Progress in Mathematics, pages 207–218. Birkhäuser, Basel, 2001.
- [HMR21] Farshid Hajir, Christian Maire, and Ravi Ramakrishna. Cutting towers of number fields. *Annales Mathématiques du Québec*, 45(2):321–345, 2021.
- [Koc02] Helmut Koch. *Galois Theory of p -Extensions*. Springer Monographs in Mathematics. Springer, Berlin and Heidelberg, 2002.
- [KHovariSosTuran54] Tamás Kővári, Vera T. Sós, and Pál Turán. On a problem of k. zarankiewicz. *Colloquium Mathematicum*, 3:50–57, 1954.
- [Lan94] Serge Lang. *Algebraic Number Theory*. Volume 110 of Graduate Texts in Mathematics. Springer, New York, second edition, 1994.
- [Matouvsek11] Jiří Matoušek. The number of unit distances is almost linear for most norms. *Advances in Mathematics*, 226(3):2618–2628, 2011.
- [Neu99] Jürgen Neukirch. *Algebraic Number Theory*. Volume 322 of Grundlehren der mathematischen Wissenschaften. Springer, Berlin and Heidelberg, 1999. Translated from the German by Norbert Schappacher.
- [NSW08] Jürgen Neukirch, Alexander Schmidt, and Kay Wingberg. *Cohomology of Number Fields*. Volume 323 of Grundlehren der mathematischen Wissenschaften. Springer, Berlin and Heidelberg, second edition, 2008.
- [RZ10] Luis Ribes and Pavel Zalesskii. *Profinite Groups*. Volume 40 of Ergebnisse der Mathematik und ihrer Grenzgebiete. 3. Folge. Springer, Berlin and Heidelberg, second edition, 2010.
- [Sha63] Igor R. Shafarevich. Extensions à points de ramification donnés. *Publications Mathématiques de l'IHÉS*, 18:71–92, 1963. In Russian.
- [Sha66] Igor R. Shafarevich. Extensions with given points of ramification. *American Mathematical Society Translations, Series 2*, 59:128–149, 1966.
- [SSzemerédiT84] Joel H. Spencer, Endre Szemerédi, and William T. Trotter. Unit distances in the euclidean plane. In Béla Bollobás, editor, *Graph Theory and Combinatorics*, pages 293–303. Academic Press, London, 1984.
- [Szekely97] László A. Székely. Crossing numbers and hard erdős problems in discrete geometry. *Combinatorics, Probability and Computing*, 6(3):353–358, 1997.
- [Tsc26] N. Tschebotareff. Die bestimmung der dichtigkeit einer menge von primzahlen, welche zu einer gegebenen substitutionsklasse gehören. *Mathematische Annalen*, 95(1):191–228, 1926.
- [Val05] Pavel Valtr. Strictly convex norms allowing many unit distances and related touching questions. 2005. Manuscript.
- [Was97] Lawrence C. Washington. *Introduction to Cyclotomic Fields*. Volume 83 of Graduate Texts in Mathematics. Springer, New York, second edition, 1997.
- [AgostonPálvolgyi22] Péter Ágoston and Dömötör Pálvolgyi. An improved constant factor for the unit distance problem. *Studia Scientiarum Mathematicarum Hungarica*, 2022.
- [OpenAI26] OpenAI. Planar Point Sets with Many Unit Distances. 2026. Manuscript supplied as course material.
- [ABucicS25] Noga Alon, Matija Bucić, and Lisa Sauermann. Unit and distinct distances in typical norms. *Geometric and Functional Analysis*, 35:1–42, 2025.
- [Bra96] Peter Brass. Erdős distance problems in normed spaces. *Computational Geometry*, 6(4):195–214, 1996.
- [BMP05] Peter Brass, William O. J. Moser, and János Pach. *Research Problems in Discrete Geometry*. Springer, New York, 2005.

- [BilkaBF+10] Ondřej Bílka, Kevin Buchin, Radoslav Fulek, Masashi Kiyomi, Yoshio Okamoto, Shin-ichi Tanigawa, and Csaba D. Tóth. A tight lower bound for convexly independent subsets of the minkowski sums of planar point sets. *Electronic Journal of Combinatorics*, 17(1):Note 35, 2010.
- [Dav00] Harold Davenport. *Multiplicative Number Theory*. Volume 74 of Graduate Texts in Mathematics. Springer, New York, third edition, 2000. Revised by Hugh L. Montgomery.
- [Dir30] Paul A. M. Dirac. A theory of electrons and protons. *Proceedings of the Royal Society A*, 126:360–365, 1930.
- [DdSMS99] John D. Dixon, Marcus P. F. du Sautoy, Avinoam Mann, and Dan Segal. *Analytic Pro- p Groups*. Volume 61 of Cambridge Studies in Advanced Mathematics. Cambridge University Press, Cambridge, second edition, 1999.
- [Ein20] Albert Einstein. *Relativity: The Special and General Theory*. Henry Holt and Company, 1920.
- [EPRothvossS08] Friedrich Eisenbrand, János Pach, Thomas Rothvoß, and Nir B. Sopher. Convexly independent subsets of the minkowski sum of planar point sets. *Electronic Journal of Combinatorics*, 15(1):Note 8, 2008.
- [ErdHos46] Paul Erdős. On sets of distances of n points. *American Mathematical Monthly*, 53(5):248–250, 1946.
- [ErdHosF97] Paul Erdős and Peter C. Fishburn. Minimum planar sets with maximum equidistance counts. *Computational Geometry*, 7(4):207–218, 1997.
- [GS64] E. S. Golod and I. R. Shafarevich. On the class field tower. *Izvestiya Akademii Nauk SSSR. Seriya Matematicheskaya*, 28(2):261–272, 1964. English translation: American Mathematical Society Translations, Series 2, 48 (1965), 91–102.
- [GS65] E. S. Golod and I. R. Shafarevich. On class field towers. In *Fourteen Papers on Logic, Algebra, Complex Variables and Topology*, volume 48 of American Mathematical Society Translations, Series 2, pages 91–102. American Mathematical Society, Providence, RI, 1965.
- [GST25] Josef Greilhuber, Carl Schildkraut, and Jonathan Tidor. More unit distances in arbitrary norms. *Bulletin of the London Mathematical Society*, 57(9):2885–2901, 2025.
- [GK15] Larry Guth and Nets Hawk Katz. On the erdős distinct distances problem in the plane. *Annals of Mathematics*, 181(1):155–190, 2015.
- [HM01] Farshid Hajir and Christian Maire. Asymptotically good towers of global fields. In *European Congress of Mathematics, Vol. II (Barcelona, 2000)*, volume 202 of Progress in Mathematics, pages 207–218. Birkhäuser, Basel, 2001.
- [HMR21] Farshid Hajir, Christian Maire, and Ravi Ramakrishna. Cutting towers of number fields. *Annales Mathématiques du Québec*, 45(2):321–345, 2021.
- [Hod96] Derek Hodson. Laboratory work as scientific method: three decades of confusion and distortion. *Journal of Curriculum Studies*, 28(2):115–135, 1996.
- [Koc02] Helmut Koch. *Galois Theory of p -Extensions*. Springer Monographs in Mathematics. Springer, Berlin and Heidelberg, 2002.
- [KHovariSosTuran54] Tamás Kővári, Vera T. Sós, and Pál Turán. On a problem of k. zarankiewicz. *Colloquium Mathematicum*, 3:50–57, 1954.
- [Lan94] Serge Lang. *Algebraic Number Theory*. Volume 110 of Graduate Texts in Mathematics. Springer, New York, second edition, 1994.
- [Matouvsek11] Jiří Matoušek. The number of unit distances is almost linear for most norms. *Advances in Mathematics*, 226(3):2618–2628, 2011.
- [Neu99] Jürgen Neukirch. *Algebraic Number Theory*. Volume 322 of Grundlehren der mathematischen Wissenschaften. Springer, Berlin and Heidelberg, 1999. Translated from the German by Norbert Schappacher.
- [NSW08] Jürgen Neukirch, Alexander Schmidt, and Kay Wingberg. *Cohomology of Number Fields*. Volume 323 of Grundlehren der mathematischen Wissenschaften. Springer, Berlin and Heidelberg, second edition, 2008.

- [RZ10] Luis Ribes and Pavel Zalesskii. *Profinite Groups*. Volume 40 of *Ergebnisse der Mathematik und ihrer Grenzgebiete. 3. Folge*. Springer, Berlin and Heidelberg, second edition, 2010.
- [Sha63] Igor R. Shafarevich. Extensions à points de ramification donnés. *Publications Mathématiques de l'IHÉS*, 18:71–92, 1963. In Russian.
- [Sha66] Igor R. Shafarevich. Extensions with given points of ramification. *American Mathematical Society Translations, Series 2*, 59:128–149, 1966.
- [SSzemerédiT84] Joel H. Spencer, Endre Szemerédi, and William T. Trotter. Unit distances in the euclidean plane. In Béla Bollobás, editor, *Graph Theory and Combinatorics*, pages 293–303. Academic Press, London, 1984.
- [Szekely97] László A. Székely. Crossing numbers and hard erdős problems in discrete geometry. *Combinatorics, Probability and Computing*, 6(3):353–358, 1997.
- [Tsc26] N. Tschebotareff. Die bestimmung der dichtigkeit einer menge von primzahlen, welche zu einer gegebenen substitutionsklasse gehören. *Mathematische Annalen*, 95(1):191–228, 1926.
- [Val05] Pavel Valtr. Strictly convex norms allowing many unit distances and related touching questions. 2005. Manuscript.
- [Was97] Lawrence C. Washington. *Introduction to Cyclotomic Fields*. Volume 83 of *Graduate Texts in Mathematics*. Springer, New York, second edition, 1997.
- [Zim00] John M. Ziman. *Real Science: What It Is, and What It Means*. Cambridge University Press, 2000.
- [AgostonPálvolgyi22] Péter Ágoston and Dömötör Pálvolgyi. An improved constant factor for the unit distance problem. *Studia Scientiarum Mathematicarum Hungarica*, 2022.
- [OpenAI26] OpenAI. Planar Point Sets with Many Unit Distances. 2026. Manuscript supplied as course material.
- [ProjectJupyterBC+25] Project Jupyter, Evan Bolyen, J. Gregory Caporaso, Rowan Cockett, Danny Garside, Chris Holdgraf, Angus Hollands, Julia Kent, Franklin Koch, Jim Madge, Matthew McKay, Mike Morrison, Fernando Pérez, Steve Purves, Min Ragan Kelley, Brian E. J. Rose, Malvika Sharan, Brigitta M. Sipőcz, Stéfan J. Walt, and Kirstie Jane Whitaker. Jupyter Book 2 and the MyST Document Stack: A Modular, Extensible, Web-Native Stack for Authoring and Publishing Computational Narratives. In *Proceedings of the 24th Python in Science Conference*, 173–193. 2025. URL: <https://doi.org/10.25080/hwcj9957>, doi:10.25080/hwcj9957.
- [TeachBooksDTeam25] TeachBooks Development Team. TeachBooks Manual. 2025. CC BY 4.0. URL: <https://teachbooks.io/manual/intro.html> (visited on 2026-05-22).